```
======================================================================
  MISSING CHILD FINDER - Full Source Code
  Generated: Fri, 27 Mar 2026 07:13:56 GMT
======================================================================

// ============================================================
// FILE: src/backend/main.mo
// ============================================================
import Map "mo:core/Map";
import Principal "mo:core/Principal";
import Iter "mo:core/Iter";
import Text "mo:core/Text";
import Runtime "mo:core/Runtime";
import MixinStorage "blob-storage/Mixin";
import MixinAuthorization "authorization/MixinAuthorization";
import AccessControl "authorization/access-control";



actor {
  include MixinStorage();
  let accessControlState = AccessControl.initState();
  include MixinAuthorization(accessControlState);

  public type UserProfile = {
    name : Text;
  };

  let userProfiles = Map.empty<Principal, UserProfile>();

  public query ({ caller }) func getCallerUserProfile() : async ?UserProfile {
    if (caller.isAnonymous()) {
      Runtime.trap("Unauthorized: Must be logged in");
    };
    userProfiles.get(caller);
  };

  public query ({ caller }) func getUserProfile(user : Principal) : async ?UserProfile {
    if (caller != user and not AccessControl.isAdmin(accessControlState, caller)) {
      Runtime.trap("Unauthorized: Can only view your own profile");
    };
    userProfiles.get(user);
  };

  public shared ({ caller }) func saveCallerUserProfile(profile : UserProfile) : async () {
    if (caller.isAnonymous()) {
      Runtime.trap("Unauthorized: Must be logged in");
    };
    userProfiles.add(caller, profile);
  };

  public type Status = {
    #active;
    #underReview;
    #found;
```

```
  #closed;
};

public type ChildRecord = {
  name : Text;
  age : Nat;
  lastLocation : Text;
  contactNumber : Text;
  lastSeenPlace : Text;
  photoId : Text;
  status : Status;
};

public type Alert = {
  contactNumber : Text;
  message : Text;
};

public type DashboardStats = {
  totalCases : Nat;
  activeCases : Nat;
  foundCases : Nat;
};

let records = Map.empty<Text, ChildRecord>();
let alerts = Map.empty<Text, Alert>();

public shared ({ caller }) func registerCase(newRecord : ChildRecord) : async () {
  if (caller.isAnonymous()) {
    Runtime.trap("Unauthorized: Must be logged in to register cases");
  };
  if (records.containsKey(newRecord.contactNumber)) {
    Runtime.trap("Case with this contact number already exists");
  };
  records.add(newRecord.contactNumber, newRecord);
};

public query func getAllCases() : async [ChildRecord] {
  records.values().toArray();
};

public query func getCase(contactNumber : Text) : async ChildRecord {
  switch (records.get(contactNumber)) {
    case (null) { Runtime.trap("Case not found") };
    case (?record) { record };
  };
};

public shared ({ caller }) func updateCaseStatus(input : (Text, Status)) : async () {
  let (contactNumber, newStatus) = input;
  if (not (AccessControl.isAdmin(accessControlState, caller))) {
    Runtime.trap("Unauthorized: Only admins can update case status");
  };
  switch (records.get(contactNumber)) {
    case (null) { Runtime.trap("Case not found") };
```

```
    case (?record) {
      let updatedRecord = {
        record with
        status = newStatus;
      };
      records.add(contactNumber, updatedRecord);
    };
  };
};

public shared ({ caller }) func deleteCase(contactNumber : Text) : async () {
  if (not (AccessControl.isAdmin(accessControlState, caller))) {
    Runtime.trap("Unauthorized: Only admins can delete cases");
  };
  if (not records.containsKey(contactNumber)) {
    Runtime.trap("Case not found");
  };
  records.remove(contactNumber);
};

public shared ({ caller }) func updateStatusToFound(contactNumber : Text) : async () {
  if (caller.isAnonymous()) {
    Runtime.trap("Unauthorized: Must be logged in");
  };
  switch (records.get(contactNumber)) {
    case (null) { Runtime.trap("Case not found") };
    case (?record) {
      let updatedRecord = {
        record with
        status = #found;
      };
      records.add(contactNumber, updatedRecord);
    };
  };
};

public shared ({ caller }) func addAlert(input : (Text, Text)) : async () {
  if (caller.isAnonymous()) {
    Runtime.trap("Unauthorized: Must be logged in");
  };
  let (contactNumber, message) = input;
  let alert = { contactNumber; message };
  alerts.add(contactNumber, alert);
};

public query ({ caller }) func getAlerts() : async [Alert] {
  if (not (AccessControl.isAdmin(accessControlState, caller))) {
    Runtime.trap("Unauthorized: Only admins can view alerts");
  };
  alerts.values().toArray();
};

public query func getDashboardStats() : async DashboardStats {
  var active = 0;
  var found = 0;
```

```
    for (record in records.values()) {
      switch (record.status) {
        case (#active) { active += 1 };
        case (#underReview) { active += 1 };
        case (#found) { found += 1 };
        case (#closed) {};
      };
    };
    {
      totalCases = records.size();
      activeCases = active;
      foundCases = found;
    };
  };

  public shared ({ caller }) func claimAdminRole(password : Text) : async () {
    if (password == "admin123") {
      accessControlState.userRoles.add(caller, #admin);
      accessControlState.adminAssigned := true;
    } else {
      Runtime.trap("Invalid password for admin role");
    };
  };
};


// ============================================================
// FILE: src/backend/authorization/MixinAuthorization.mo
// ============================================================
import AccessControl "./access-control";
import Prim "mo:prim";
import Runtime "mo:core/Runtime";

mixin (accessControlState : AccessControl.AccessControlState) {
  // Initialize auth (first caller becomes admin, others become users)
  public shared ({ caller }) func _initializeAccessControlWithSecret(userSecret : Text) : async () {
    switch (Prim.envVar<system>("CAFFEINE_ADMIN_TOKEN")) {
      case (null) {
        Runtime.trap("CAFFEINE_ADMIN_TOKEN environment variable is not set");
      };
      case (?adminToken) {
        AccessControl.initialize(accessControlState, caller, adminToken, userSecret);
      };
    };
  };

  public query ({ caller }) func getCallerUserRole() : async AccessControl.UserRole {
    AccessControl.getUserRole(accessControlState, caller);
  };

  public shared ({ caller }) func assignCallerUserRole(user : Principal, role : AccessControl.UserRo
    // Admin-only check happens inside
    AccessControl.assignRole(accessControlState, caller, user, role);
  };
```

```
  public query ({ caller }) func isCallerAdmin() : async Bool {
    AccessControl.isAdmin(accessControlState, caller);
  };
};


// ============================================================
// FILE: src/backend/authorization/access-control.mo
// ============================================================
import Map "mo:core/Map";
import Principal "mo:core/Principal";
import Runtime "mo:core/Runtime";

module {
  public type UserRole = {
    #admin;
    #user;
    #guest;
  };

  public type AccessControlState = {
    var adminAssigned : Bool;
    userRoles : Map.Map<Principal, UserRole>;
  };

  public func initState() : AccessControlState {
    {
      var adminAssigned = false;
      userRoles = Map.empty<Principal, UserRole>();
    };
  };

  // First principal that calls this function becomes admin, all other principals become users.
  public func initialize(state : AccessControlState, caller : Principal, adminToken : Text, userProv
    if (caller.isAnonymous()) { return };
    switch (state.userRoles.get(caller)) {
      case (?_) {};
      case (null) {
        if (not state.adminAssigned and userProvidedToken == adminToken) {
          state.userRoles.add(caller, #admin);
          state.adminAssigned := true;
        } else {
          state.userRoles.add(caller, #user);
        };
      };
    };
  };

  public func getUserRole(state : AccessControlState, caller : Principal) : UserRole {
    if (caller.isAnonymous()) { return #guest };
    switch (state.userRoles.get(caller)) {
      case (?role) { role };
      case (null) {
        Runtime.trap("User is not registered");
      };
```

```
    };
  };

  public func assignRole(state : AccessControlState, caller : Principal, user : Principal, role : Us
    if (not (isAdmin(state, caller))) {
      Runtime.trap("Unauthorized: Only admins can assign user roles");
    };
    state.userRoles.add(user, role);
  };

  public func hasPermission(state : AccessControlState, caller : Principal, requiredRole : UserRole)
    let userRole = getUserRole(state, caller);
    if (userRole == #admin or requiredRole == #guest) { true } else {
      userRole == requiredRole;
    };
  };

  public func isAdmin(state : AccessControlState, caller : Principal) : Bool {
    getUserRole(state, caller) == #admin;
  };
};


// ============================================================
// FILE: src/backend/blob-storage/Mixin.mo
// ============================================================
import Nat "mo:core/Nat";
import Principal "mo:core/Principal";
import Map "mo:core/Map";
import Array "mo:core/Array";
import Text "mo:core/Text";
import Time "mo:core/Time";
import Storage "Storage";
import Prim "mo:prim";
import Runtime "mo:core/Runtime";

mixin () {
  type ExternalBlob = Storage.ExternalBlob;

  transient let _caffeineStorageState : Storage.State = Storage.new();

  type _CaffeineStorageRefillInformation = {
    proposed_top_up_amount : ?Nat;
  };

  type _CaffeineStorageRefillResult = {
    success : ?Bool;
    topped_up_amount : ?Nat;
  };

  type _CaffeineStorageCreateCertificateResult = {
    method : Text;
    blob_hash : Text;
  };
```

```
  public shared ({ caller }) func _caffeineStorageRefillCashier(refillInformation : ?_CaffeineStorag
    let cashier = await Storage.getCashierPrincipal();
    if (cashier != caller) {
      Runtime.trap("Unauthorized access");
    };
    await Storage.refillCashier(_caffeineStorageState, cashier, refillInformation);
  };

  public shared ({ caller }) func _caffeineStorageUpdateGatewayPrincipals() : async () {
    await Storage.updateGatewayPrincipals(_caffeineStorageState);
  };

  public query ({ caller }) func _caffeineStorageBlobIsLive(hash : Blob) : async Bool {
    Prim.isStorageBlobLive(hash);
  };

  public query ({ caller }) func _caffeineStorageBlobsToDelete() : async [Blob] {
    if (not Storage.isAuthorized(_caffeineStorageState, caller)) {
      Runtime.trap("Unauthorized access");
    };
    let deadBlobs = Prim.getDeadBlobs();
    switch (deadBlobs) {
      case (null) {
        [];
      };
      case (?deadBlobs) {
        deadBlobs.sliceToArray(0, 10000);
      };
    };
  };

  public shared ({ caller }) func _caffeineStorageConfirmBlobDeletion(blobs : [Blob]) : async () {
    if (not Storage.isAuthorized(_caffeineStorageState, caller)) {
      Runtime.trap("Unauthorized access");
    };
    Prim.pruneConfirmedDeadBlobs(blobs);
    // Trigger GC forcefully.
    type GC = actor {
      __motoko_gc_trigger : () -> async ();
    };
    let myGC = actor (debug_show (Prim.getSelfPrincipal<system>())) : GC;
    await myGC.__motoko_gc_trigger();
  };

  public shared ({ caller }) func _caffeineStorageCreateCertificate(blobHash : Text) : async _Caffei
    {
      method = "upload";
      blob_hash = blobHash;
    };
  };
};


// ============================================================
// FILE: src/backend/blob-storage/Storage.mo
```

```
// ==========================================================
import Runtime "mo:core/Runtime";
import Principal "mo:core/Principal";
import Array "mo:core/Array";
import Prim "mo:prim";
import Cycles "mo:core/Cycles";
import Nat "mo:core/Nat";

module {
  public type ExternalBlob = Blob;

  public type State = {
    var authorizedPrincipals : [Principal];
  };

  public func new() : State {
    let authorizedPrincipals : [Principal] = [];
    {
      var authorizedPrincipals;
    };
  };

  public func getCashierPrincipal() : async Principal {
    switch (Prim.envVar<system>("CAFFFEINE_STORAGE_CASHIER_PRINCIPAL")) {
      case (null) {
        Runtime.trap("CAFFFEINE_STORAGE_CASHIER_PRINCIPAL environment variable is not set");
      };
      case (?cashierPrincipal) {
        Principal.fromText(cashierPrincipal);
      };
    };
  };

  // Authorization functions
  public func updateGatewayPrincipals(registry : State) : async () {
    let cashierPrincipal = await getCashierPrincipal();
    let cashierActor = actor (cashierPrincipal.toText()) : actor {
      storage_gateway_principal_list_v1 : () -> async [Principal];
    };

    registry.authorizedPrincipals := await cashierActor.storage_gateway_principal_list_v1();
  };

  public func isAuthorized(registry : State, caller : Principal) : Bool {
    let authorized = registry.authorizedPrincipals.find(
      func(principal) {
        Principal.equal(principal, caller);
      }
    ) != null;
    authorized;
  };

  public func refillCashier(
    _registry : State,
    cashier : Principal,
```

```
    refillInformation : ?{
      proposed_top_up_amount : ?Nat;
    },
  ) : async {
    success : ?Bool;
    topped_up_amount : ?Nat;
  } {
    let currentBalance = Cycles.balance();
    let reservedCycles : Nat = 400_000_000_000;

    let currentFreeCyclesCount : Nat = Nat.sub(currentBalance, reservedCycles);

    let cyclesToSend : Nat = switch (refillInformation) {
      case (null) { currentFreeCyclesCount };
      case (?info) {
        switch (info.proposed_top_up_amount) {
          case (null) { currentFreeCyclesCount };
          case (?proposed) { Nat.min(proposed, currentFreeCyclesCount) };
        };
      };
    };

    let targetCanister = actor (cashier.toText()) : actor {
      account_top_up_v1 : ({ account : Principal }) -> async ();
    };

    await (with cycles = cyclesToSend) targetCanister.account_top_up_v1({
      account = Prim.getSelfPrincipal<system>();
    });

    {
      success = ?true;
      topped_up_amount = ?cyclesToSend;
    };
  };
};


// ============================================================
// FILE: src/frontend/src/main.tsx
// ============================================================
import ReactDOM from "react-dom/client";
import { InternetIdentityProvider } from "./hooks/useInternetIdentity";
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import App from "./App";
import "./index.css";

BigInt.prototype.toJSON = function () {
  return this.toString();
};

declare global {
  interface BigInt {
    toJSON(): string;
  }
```

```
}

const queryClient = new QueryClient();

ReactDOM.createRoot(document.getElementById("root")!).render(
  <QueryClientProvider client={queryClient}>
    <InternetIdentityProvider>
      <App />
    </InternetIdentityProvider>
  </QueryClientProvider>,
);


// ============================================================
// FILE: src/frontend/src/App.tsx
// ============================================================
import { Toaster } from "@/components/ui/sonner";
import {
  Navigate,
  Outlet,
  RouterProvider,
  createRootRoute,
  createRoute,
  createRouter,
} from "@tanstack/react-router";
import ProtectedRoute from "./components/ProtectedRoute";
import AccountPage from "./pages/AccountPage";
import AdminLoginPage from "./pages/AdminLoginPage";
import AdminPage from "./pages/AdminPage";
import DashboardPage from "./pages/DashboardPage";
import LandingPage from "./pages/LandingPage";
import LoginPage from "./pages/LoginPage";
import RegisterPage from "./pages/RegisterPage";
import SearchPage from "./pages/SearchPage";

const rootRoute = createRootRoute({
  component: () => (
    <>
      <Outlet />
      <Toaster />
    </>
  ),
});

const indexRoute = createRoute({
  getParentRoute: () => rootRoute,
  path: "/",
  component: LandingPage,
});

const loginRoute = createRoute({
  getParentRoute: () => rootRoute,
  path: "/login",
  component: LoginPage,
});
```

```
const adminLoginRoute = createRoute({
  getParentRoute: () => rootRoute,
  path: "/admin-login",
  component: AdminLoginPage,
});

const dashboardRoute = createRoute({
  getParentRoute: () => rootRoute,
  path: "/dashboard",
  component: () => (
    <ProtectedRoute>
      <DashboardPage />
    </ProtectedRoute>
  ),
});

const registerRoute = createRoute({
  getParentRoute: () => rootRoute,
  path: "/register",
  component: () => (
    <ProtectedRoute>
      <RegisterPage />
    </ProtectedRoute>
  ),
});

const searchRoute = createRoute({
  getParentRoute: () => rootRoute,
  path: "/search",
  component: SearchPage,
});

const accountRoute = createRoute({
  getParentRoute: () => rootRoute,
  path: "/account",
  component: () => (
    <ProtectedRoute>
      <AccountPage />
    </ProtectedRoute>
  ),
});

const adminRoute = createRoute({
  getParentRoute: () => rootRoute,
  path: "/admin",
  component: AdminPage,
});

const routeTree = rootRoute.addChildren([
  indexRoute,
  loginRoute,
  adminLoginRoute,
  dashboardRoute,
  registerRoute,
```

```
  searchRoute,
  accountRoute,
  adminRoute,
]);

const router = createRouter({ routeTree });

declare module "@tanstack/react-router" {
  interface Register {
    router: typeof router;
  }
}

export default function App() {
  return <RouterProvider router={router} />;
}


// ============================================================
// FILE: src/frontend/src/config.ts
// ============================================================
import {
  createActor,
  type backendInterface,
  type CreateActorOptions,
  ExternalBlob,
} from "./backend";
import { StorageClient } from "./utils/StorageClient";
import { HttpAgent } from "@icp-sdk/core/agent";

const DEFAULT_STORAGE_GATEWAY_URL = "https://blob.caffeine.ai";
const DEFAULT_BUCKET_NAME = "default-bucket";
const DEFAULT_PROJECT_ID = "0000000-0000-0000-0000-00000000000";

interface JsonConfig {
  backend_host: string;
  backend_canister_id: string;
  project_id: string;
  ii_derivation_origin: string;
}

interface Config {
  backend_host?: string;
  backend_canister_id: string;
  storage_gateway_url: string;
  bucket_name: string;
  project_id: string;
  ii_derivation_origin?: string;
}

let configCache: Config | null = null;

export async function loadConfig(): Promise<Config> {
  if (configCache) {
    return configCache;
```

```
  }
  const backendCanisterId = process.env.CANISTER_ID_BACKEND;
  const envBaseUrl = process.env.BASE_URL || "/";
  const baseUrl = envBaseUrl.endsWith("/") ? envBaseUrl : `${envBaseUrl}/`;
  try {
    const response = await fetch(`${baseUrl}env.json`);
    const config = (await response.json()) as JsonConfig;
    if (!backendCanisterId && config.backend_canister_id === "undefined") {
      console.error("CANISTER_ID_BACKEND is not set");
      throw new Error("CANISTER_ID_BACKEND is not set");
    }

    const fullConfig = {
      backend_host:
        config.backend_host === "undefined" ? undefined : config.backend_host,
      backend_canister_id: (config.backend_canister_id === "undefined"
        ? backendCanisterId
        : config.backend_canister_id) as string,
      storage_gateway_url: process.env.STORAGE_GATEWAY_URL ?? "nogateway",
      bucket_name: DEFAULT_BUCKET_NAME,
      project_id:
        config.project_id !== "undefined"
          ? config.project_id
          : DEFAULT_PROJECT_ID,
      ii_derivation_origin:
        config.ii_derivation_origin === "undefined"
          ? undefined
          : config.ii_derivation_origin,
    };
    configCache = fullConfig;
    return fullConfig;
  } catch {
    if (!backendCanisterId) {
      console.error("CANISTER_ID_BACKEND is not set");
      throw new Error("CANISTER_ID_BACKEND is not set");
    }
    const fallbackConfig = {
      backend_host: undefined,
      backend_canister_id: backendCanisterId,
      storage_gateway_url: DEFAULT_STORAGE_GATEWAY_URL,
      bucket_name: DEFAULT_BUCKET_NAME,
      project_id: DEFAULT_PROJECT_ID,
      ii_derivation_origin: undefined,
    };
    return fallbackConfig;
  }
}

function extractAgentErrorMessage(error: string): string {
  const errorString = String(error);
  const match = errorString.match(/with message:\s*'([^']+)'/s);
  return match ? match[1] : errorString;
}

function processError(e: unknown): never {
```

```
  if (e && typeof e === "object" && "message" in e) {
    throw new Error(extractAgentErrorMessage(`${e.message}`));
  }
  throw e;
}

async function maybeLoadMockBackend(): Promise<backendInterface | null> {
  if (import.meta.env.VITE_USE_MOCK !== "true") {
    return null;
  }

  try {
    // If VITE_USE_MOCK is enabled, try to load a mock backend module *if it exists*.
    // We use import.meta.glob so builds don't fail when the mock file is absent.
    const mockModules = import.meta.glob("./mocks/backend.{ts,tsx,js,jsx}");

    const path = Object.keys(mockModules)[0];
    if (!path) return null;

    const mod = (await mockModules[path]()) as {
      mockBackend?: backendInterface;
    };

    return mod.mockBackend ?? null;
  } catch {
    return null;
  }
}

export async function createActorWithConfig(
  options?: CreateActorOptions,
): Promise<backendInterface> {
  // Attempt to load mock backend if enabled
  const mock = await maybeLoadMockBackend();
  if (mock) {
    return mock;
  }

  const config = await loadConfig();
  const resolvedOptions = options ?? {};
  const agent = new HttpAgent({
    ...resolvedOptions.agentOptions,
    host: config.backend_host,
  });
  if (config.backend_host?.includes("localhost")) {
    await agent.fetchRootKey().catch((err) => {
      console.warn(
        "Unable to fetch root key. Check to ensure that your local replica is running",
      );
      console.error(err);
    });
  }
  const actorOptions = {
    ...resolvedOptions,
    agent: agent,
```

```
    processError,
  };

  const storageClient = new StorageClient(
    config.bucket_name,
    config.storage_gateway_url,
    config.backend_canister_id,
    config.project_id,
    agent,
  );

  const MOTOKO_DEDUPLICATION_SENTINEL = "!caf!";

  const uploadFile = async (file: ExternalBlob): Promise<Uint8Array> => {
    const { hash } = await storageClient.putFile(
      await file.getBytes(),
      file.onProgress,
    );
    return new TextEncoder().encode(MOTOKO_DEDUPLICATION_SENTINEL + hash);
  };

  const downloadFile = async (bytes: Uint8Array): Promise<ExternalBlob> => {
    const hashWithPrefix = new TextDecoder().decode(new Uint8Array(bytes));
    const hash = hashWithPrefix.substring(MOTOKO_DEDUPLICATION_SENTINEL.length);
    const url = await storageClient.getDirectURL(hash);
    return ExternalBlob.fromURL(url);
  };

  return createActor(
    config.backend_canister_id,
    uploadFile,
    downloadFile,
    actorOptions,
  );
}


// ============================================================
// FILE: src/frontend/src/hooks/useActor.ts
// ============================================================
import { useInternetIdentity } from "./useInternetIdentity";
import { useQuery, useQueryClient } from "@tanstack/react-query";
import { useEffect } from "react";
import { type backendInterface } from "../backend";
import { createActorWithConfig } from "../config";
import { getSecretParameter } from "../utils/urlParams";

const ACTOR_QUERY_KEY = "actor";
export function useActor() {
  const { identity } = useInternetIdentity();
  const queryClient = useQueryClient();
  const actorQuery = useQuery<backendInterface>({
    queryKey: [ACTOR_QUERY_KEY, identity?.getPrincipal().toString()],
    queryFn: async () => {
      const isAuthenticated = !!identity;
```

```
      if (!isAuthenticated) {
        // Return anonymous actor if not authenticated
        return await createActorWithConfig();
      }

      const actorOptions = {
        agentOptions: {
          identity,
        },
      };

      const actor = await createActorWithConfig(actorOptions);
      const adminToken = getSecretParameter("caffeineAdminToken") || "";
      await actor._initializeAccessControlWithSecret(adminToken);
      return actor;
    },
    // Only refetch when identity changes
    staleTime: Infinity,
    // This will cause the actor to be recreated when the identity changes
    enabled: true,
  });

  // When the actor changes, invalidate dependent queries
  useEffect(() => {
    if (actorQuery.data) {
      queryClient.invalidateQueries({
        predicate: (query) => {
          return !query.queryKey.includes(ACTOR_QUERY_KEY);
        },
      });
      queryClient.refetchQueries({
        predicate: (query) => {
          return !query.queryKey.includes(ACTOR_QUERY_KEY);
        },
      });
    }
  }, [actorQuery.data, queryClient]);

  return {
    actor: actorQuery.data || null,
    isFetching: actorQuery.isFetching,
  };
}


// ============================================================
// FILE: src/frontend/src/hooks/useInternetIdentity.ts
// ============================================================
import {
  type ReactNode,
  type PropsWithChildren,
  createContext,
  createElement,
  useCallback,
```

```
  useContext,
  useEffect,
  useMemo,
  useState,
} from "react";
import {
  AuthClient,
  type AuthClientCreateOptions,
  type AuthClientLoginOptions,
} from "@dfinity/auth-client";
import type { Identity } from "@icp-sdk/core/agent";
import { DelegationIdentity, isDelegationValid } from "@icp-sdk/core/identity";
import { loadConfig } from "../config";

export type Status =
  | "initializing"
  | "idle"
  | "logging-in"
  | "success"
  | "loginError";

export type InternetIdentityContext = {
  /** The identity is available after successfully loading the identity from local storage
   * or completing the login process. */
  identity?: Identity;

  /** Connect to Internet Identity to login the user. */
  login: () => void;

  /** Clears the identity from the state and local storage. Effectively "logs the user out". */
  clear: () => void;

  /** The loginStatus of the login process. Note: The login loginStatus is not affected when a store
   * identity is loaded on mount. */
  loginStatus: Status;

  /** `loginStatus === "initializing"` */
  isInitializing: boolean;

  /** `loginStatus === "idle"` */
  isLoginIdle: boolean;

  /** `loginStatus === "logging-in"` */
  isLoggingIn: boolean;

  /** `loginStatus === "success"` */
  isLoginSuccess: boolean;

  /** `loginStatus === "loginError"` */
  isLoginError: boolean;

  loginError?: Error;
};

const ONE_HOUR_IN_NANOSECONDS = BigInt(3_600_000_000_000);
```

```
const DEFAULT_IDENTITY_PROVIDER = process.env.II_URL;

type ProviderValue = InternetIdentityContext;
const InternetIdentityReactContext = createContext<ProviderValue | undefined>(
  undefined,
);

/**
 * Create the auth client with default options or options provided by the user.
 */
async function createAuthClient(
  createOptions?: AuthClientCreateOptions,
): Promise<AuthClient> {
  const config = await loadConfig();
  const options: AuthClientCreateOptions = {
    idleOptions: {
      // Default behaviour of this hook is not to logout and reload window on identity expiration
      disableDefaultIdleCallback: true,
      disableIdle: true,
      ...createOptions?.idleOptions,
    },
    loginOptions: {
      derivationOrigin: config.ii_derivation_origin,
    },
    ...createOptions,
  };
  const authClient = await AuthClient.create(options);
  return authClient;
}

/**
 * Helper function to set loginError state.
 */
function assertProviderPresent(
  context: ProviderValue | undefined,
): asserts context is ProviderValue {
  if (!context) {
    throw new Error(
      "InternetIdentityProvider is not present. Wrap your component tree with it.",
    );
  }
}

/**
 * Hook to access the internet identity as well as loginStatus along with
 * login and clear functions.
 */
export const useInternetIdentity = (): InternetIdentityContext => {
  const context = useContext(InternetIdentityReactContext);
  assertProviderPresent(context);
  return context;
};

/**
 * The InternetIdentityProvider component makes the saved identity available
```

```
 * after page reloads. It also allows you to configure default options
 * for AuthClient and login.
 *
 *
 * @example
 * ```tsx
 * <InternetIdentityProvider>
 *   <App />
 * </InternetIdentityProvider>
 * ```
 */
export function InternetIdentityProvider({
  children,
  createOptions,
}: PropsWithChildren<{
  /** The child components that the InternetIdentityProvider will wrap. This allows any child
   * component to access the authentication context provided by the InternetIdentityProvider. */
  children: ReactNode;

  /** Options for creating the {@link AuthClient}. See AuthClient documentation for list of options
   *
   * defaults to disabling the AuthClient idle handling (clearing identities
   * from store and reloading the window on identity expiry). If that behaviour is preferred, set th
   *
   * ```
   * const options = {
   *   idleOptions: {
   *     disableDefaultIdleCallback: false,
   *     disableIdle: false,
   *   },
   * }
   * ```
   */
  createOptions?: AuthClientCreateOptions;
}>) {
  const [authClient, setAuthClient] = useState<AuthClient | undefined>(
    undefined,
  );
  const [identity, setIdentity] = useState<Identity | undefined>(undefined);
  const [loginStatus, setStatus] = useState<Status>("initializing");
  const [loginError, setError] = useState<Error | undefined>(undefined);

  const setErrorMessage = useCallback((message: string) => {
    setStatus("loginError");
    setError(new Error(message));
  }, []);

  const handleLoginSuccess = useCallback(() => {
    const latestIdentity = authClient?.getIdentity();
    if (!latestIdentity) {
      setErrorMessage("Identity not found after successful login");
      return;
    }
    setIdentity(latestIdentity);
    setStatus("success");
```

```
}, [authClient, setErrorMessage]);

const handleLoginError = useCallback(
  (maybeError?: string) => {
    setErrorMessage(maybeError ?? "Login failed");
  },
  [setErrorMessage],
);

const login = useCallback(() => {
  if (!authClient) {
    setErrorMessage(
      "AuthClient is not initialized yet, make sure to call `login` on user interaction e.g. click
    );
    return;
  }

  const currentIdentity = authClient.getIdentity();
  if (
    !currentIdentity.getPrincipal().isAnonymous() &&
    currentIdentity instanceof DelegationIdentity &&
    isDelegationValid(currentIdentity.getDelegation())
  ) {
    setErrorMessage("User is already authenticated");
    return;
  }

  const options: AuthClientLoginOptions = {
    identityProvider: DEFAULT_IDENTITY_PROVIDER,
    onSuccess: handleLoginSuccess,
    onError: handleLoginError,
    maxTimeToLive: ONE_HOUR_IN_NANOSECONDS * BigInt(24 * 30), // 30 days
  };

  setStatus("logging-in");
  void authClient.login(options);
}, [authClient, handleLoginError, handleLoginSuccess, setErrorMessage]);

const clear = useCallback(() => {
  if (!authClient) {
    setErrorMessage("Auth client not initialized");
    return;
  }

  void authClient
    .logout()
    .then(() => {
      setIdentity(undefined);
      setAuthClient(undefined);
      setStatus("idle");
      setError(undefined);
    })
    .catch((unknownError: unknown) => {
      setStatus("loginError");
      setError(
```

```
        unknownError instanceof Error
          ? unknownError
          : new Error("Logout failed"),
      );
    });
}, [authClient, setErrorMessage]);

useEffect(() => {
  let cancelled = false;
  void (async () => {
    try {
      setStatus("initializing");
      let existingClient = authClient;
      if (!existingClient) {
        existingClient = await createAuthClient(createOptions);
        if (cancelled) return;
        setAuthClient(existingClient);
      }
      const isAuthenticated = await existingClient.isAuthenticated();
      if (cancelled) return;
      if (isAuthenticated) {
        const loadedIdentity = existingClient.getIdentity();
        setIdentity(loadedIdentity);
      }
    } catch (unknownError) {
      setStatus("loginError");
      setError(
        unknownError instanceof Error
          ? unknownError
          : new Error("Initialization failed"),
      );
    } finally {
      if (!cancelled) setStatus("idle");
    }
  })();
  return () => {
    cancelled = true;
  };
}, [createOptions, authClient]);

const value = useMemo<ProviderValue>(
  () => ({
    identity,
    login,
    clear,
    loginStatus,
    isInitializing: loginStatus === "initializing",
    isLoginIdle: loginStatus === "idle",
    isLoggingIn: loginStatus === "logging-in",
    isLoginSuccess: loginStatus === "success",
    isLoginError: loginStatus === "loginError",
    loginError,
  }),
  [identity, login, clear, loginStatus, loginError],
);
```

```
  return createElement(InternetIdentityReactContext.Provider, {
    value,
    children,
  });
}


// ============================================================
// FILE: src/frontend/src/hooks/useQueries.ts
// ============================================================
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import {
  type Alert,
  type ChildRecord,
  type DashboardStats,
  Status,
  type UserProfile,
  UserRole,
} from "../backend";
import { useActor } from "./useActor";

export function useGetAllCases() {
  const { actor, isFetching } = useActor();
  return useQuery<ChildRecord[]>({
    queryKey: ["cases"],
    queryFn: async () => {
      if (!actor) return [];
      return actor.getAllCases();
    },
    enabled: !!actor && !isFetching,
    staleTime: 30_000,
  });
}

export function useGetDashboardStats() {
  const { actor, isFetching } = useActor();
  return useQuery<DashboardStats>({
    queryKey: ["dashboardStats"],
    queryFn: async () => {
      if (!actor) return { totalCases: 0n, activeCases: 0n, foundCases: 0n };
      return actor.getDashboardStats();
    },
    enabled: !!actor && !isFetching,
    staleTime: 30_000,
  });
}

export function useGetCallerProfile() {
  const { actor, isFetching } = useActor();
  return useQuery<UserProfile | null>({
    queryKey: ["callerProfile"],
    queryFn: async () => {
      if (!actor) return null;
      return actor.getCallerUserProfile();
```

```
    },
    enabled: !!actor && !isFetching,
  });
}

export function useGetCallerRole() {
  const { actor, isFetching } = useActor();
  return useQuery<UserRole>({
    queryKey: ["callerRole"],
    queryFn: async () => {
      if (!actor) return UserRole.guest;
      return actor.getCallerUserRole();
    },
    enabled: !!actor && !isFetching,
  });
}

export function useIsCallerAdmin() {
  const { actor, isFetching } = useActor();
  return useQuery<boolean>({
    queryKey: ["isAdmin"],
    queryFn: async () => {
      if (!actor) return false;
      return actor.isCallerAdmin();
    },
    enabled: !!actor && !isFetching,
  });
}

export function useGetAlerts() {
  const { actor, isFetching } = useActor();
  return useQuery<Alert[]>({
    queryKey: ["alerts"],
    queryFn: async () => {
      if (!actor) return [];
      try {
        return await actor.getAlerts();
      } catch {
        // Non-admin users will get an unauthorized error   return empty array
        return [];
      }
    },
    enabled: !!actor && !isFetching,
  });
}

export function useRegisterCase() {
  const { actor } = useActor();
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: async (record: ChildRecord) => {
      if (!actor)
        throw new Error("Backend not connected. Please wait and try again.");

      let lastError: unknown;
```

```
      for (let attempt = 0; attempt < 4; attempt++) {
        try {
          await actor.registerCase(record);
          return;
        } catch (err: unknown) {
          lastError = err;
          const msg = String((err as { message?: string })?.message ?? err);
          const isTransient =
            msg.includes("IC0508") ||
            msg.includes("is stopped") ||
            msg.includes("stopped") ||
            msg.includes("canister is stopping") ||
            msg.includes("temporarily unavailable");
          if (isTransient && attempt < 3) {
            await new Promise((resolve) =>
              setTimeout(resolve, 2000 * (attempt + 1)),
            );
            continue;
          }
          throw err;
        }
      }
      throw lastError;
    },
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ["cases"] });
      queryClient.invalidateQueries({ queryKey: ["dashboardStats"] });
    },
  });
}

export function useUpdateStatusToFound() {
  const { actor } = useActor();
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: async (contactNumber: string) => {
      if (!actor) throw new Error("Not authenticated");
      await actor.updateStatusToFound(contactNumber);
    },
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ["cases"] });
      queryClient.invalidateQueries({ queryKey: ["dashboardStats"] });
    },
  });
}

export function useUpdateCaseStatus() {
  const { actor } = useActor();
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: async ({
      contactNumber,
      status,
    }: {
      contactNumber: string;
```

```
      status: Status;
    }) => {
      if (!actor) throw new Error("Not authenticated");
      await actor.updateCaseStatus([contactNumber, status]);
    },
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ["cases"] });
      queryClient.invalidateQueries({ queryKey: ["dashboardStats"] });
    },
  });
}

export function useDeleteCase() {
  const { actor } = useActor();
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: async (contactNumber: string) => {
      if (!actor) throw new Error("Not authenticated");
      await actor.deleteCase(contactNumber);
    },
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ["cases"] });
      queryClient.invalidateQueries({ queryKey: ["dashboardStats"] });
    },
  });
}

export function useAddAlert() {
  const { actor } = useActor();
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: async ({
      contactNumber,
      message,
    }: {
      contactNumber: string;
      message: string;
    }) => {
      if (!actor) throw new Error("Not authenticated");
      await actor.addAlert([contactNumber, message]);
    },
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ["alerts"] });
    },
  });
}

export { Status };


// ============================================================
// FILE: src/frontend/src/hooks/use-mobile.tsx
// ============================================================
import { useEffect, useState } from "react";
```

```
export function useIsMobile(breakpoint = 768): boolean {
  const [isMobile, setIsMobile] = useState<boolean>(false);
  useEffect(() => {
    const update = () => setIsMobile(window.innerWidth < breakpoint);
    update();
    window.addEventListener("resize", update);
    return () => window.removeEventListener("resize", update);
  }, [breakpoint]);
  return isMobile;
}


// ============================================================
// FILE: src/frontend/src/utils/imageAnalysis.ts
// ============================================================
// PPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPP
// Missing Child Finder   Face Analysis Engine v3
// Models: face-api.js (CNN) ? BlazeFace ? YOLOv8-ONNX ? Age Progression
// v3: improved parallelism, tighter thresholds, faster model init
// PPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPPP

const TFJS_VERSION = "4.22.0";
const BLAZEFACE_VERSION = "0.0.7";
const ORT_VERSION = "1.18.0";

const TFJS_CDN = `https://cdn.jsdelivr.net/npm/@tensorflow/tfjs@${TFJS_VERSION}/dist/tf.min.js`;
const BLAZEFACE_CDN = `https://cdn.jsdelivr.net/npm/@tensorflow-models/blazeface@${BLAZEFACE_VERSION
const ORT_CDN = `https://cdn.jsdelivr.net/npm/onnxruntime-web@${ORT_VERSION}/dist/ort.min.js`;

const FACEAPI_CDNS = [
  "https://cdn.jsdelivr.net/npm/@vladmandic/face-api@1.7.13/dist/face-api.js",
  "https://unpkg.com/@vladmandic/face-api@1.7.13/dist/face-api.js",
  "https://cdn.jsdelivr.net/npm/@vladmandic/face-api/dist/face-api.js",
  "https://unpkg.com/@vladmandic/face-api/dist/face-api.js",
  "https://cdn.jsdelivr.net/npm/face-api.js@0.22.2/dist/face-api.min.js",
  "https://unpkg.com/face-api.js@0.22.2/dist/face-api.min.js",
];

const MODEL_URLS = [
  "https://cdn.jsdelivr.net/npm/@vladmandic/face-api/model",
  "https://unpkg.com/@vladmandic/face-api/model",
  "https://cdn.jsdelivr.net/npm/face-api.js/weights",
  "https://unpkg.com/face-api.js/weights",
  "https://raw.githubusercontent.com/vladmandic/face-api/master/model",
  "https://raw.githubusercontent.com/justadudewhohacks/face-api.js/master/weights",
];

const YOLOV8_ONNX_URLS = [
  "https://huggingface.co/spaces/nicehorse/yolov8-face-detection/resolve/main/yolov8n-face.onnx",
  "https://raw.githubusercontent.com/akanametov/yolo-face/main/weights/yolov8n-face.onnx",
];

const CANVAS_SIZE = 64;
const BINS = 32;
```

```
/**
 * CNN Euclidean distance thresholds (face-api.js 128-dim FaceNet descriptor):
 *   Same person, good photo:              0.20   0.50
 *   Same person, different lighting/angle: 0.40   0.65
 *   Different people:                      usually > 0.60, almost always > 0.72
 *
 * Using 0.74 gives the best recall without false positives.
 */
const CNN_THRESHOLD = 0.74;

//     Search face cache (avoids re-detecting the same search image N times)
const searchFaceCache = new Map<string, FaceAnalysis | null>();
export function clearSearchFaceCache() {
  searchFaceCache.clear();
}

//     Global model state
let faceApiLoaded = false;
let faceApiLoading: Promise<void> | null = null;

// eslint-disable-next-line @typescript-eslint/no-explicit-any
let blazefaceModel: any = null;
let blazefaceLoading: Promise<void> | null = null;

// eslint-disable-next-line @typescript-eslint/no-explicit-any
let yolov8Session: any = null;
let yolov8Loading: Promise<void> | null = null;

let modelLoadError: string | null = null;

declare global {
  interface Window {
    // eslint-disable-next-line @typescript-eslint/no-explicit-any
    faceapi: any;
    // eslint-disable-next-line @typescript-eslint/no-explicit-any
    tf: any;
    // eslint-disable-next-line @typescript-eslint/no-explicit-any
    blazeface: any;
    // eslint-disable-next-line @typescript-eslint/no-explicit-any
    ort: any;
  }
}

// eslint-disable-next-line @typescript-eslint/no-explicit-any
function getFaceApi(): any {
  return typeof window !== "undefined" ? window.faceapi : null;
}

//     Script loader
function loadScript(src: string, timeoutMs = 12000): Promise<void> {
  return new Promise((resolve, reject) => {
    if (document.querySelector(`script[src="${src}"]`)) {
      resolve();
      return;
    }
```

```
    const s = document.createElement("script");
    s.src = src;
    const timer = setTimeout(
      () => reject(new Error(`Timeout: ${src}`)),
      timeoutMs,
    );
    s.onload = () => {
      clearTimeout(timer);
      resolve();
    };
    s.onerror = () => {
      clearTimeout(timer);
      reject(new Error(`Failed: ${src}`));
    };
    document.head.appendChild(s);
  });
}

async function loadScriptWithFallback(
  cdns: string[],
  checkFn: () => boolean,
): Promise<void> {
  for (const cdn of cdns) {
    try {
      await loadScript(cdn);
      // Poll up to 3 seconds for the global to appear
      for (let i = 0; i < 30; i++) {
        if (checkFn()) return;
        await new Promise((r) => setTimeout(r, 100));
      }
      if (checkFn()) return;
    } catch {
      /* try next CDN */
    }
  }
  throw new Error(`All CDNs failed: ${cdns[0]}`);
}

//     1. face-api.js loader
async function loadFaceApiModels(modelUrl: string): Promise<void> {
  const api = getFaceApi();
  if (!api) throw new Error("face-api not loaded");
  await Promise.all([
    api.nets.tinyFaceDetector.loadFromUri(modelUrl),
    api.nets.ssdMobilenetv1.loadFromUri(modelUrl),
    api.nets.faceLandmark68Net.loadFromUri(modelUrl),
    api.nets.faceRecognitionNet.loadFromUri(modelUrl),
  ]);
}

async function initFaceApi(): Promise<void> {
  await loadScriptWithFallback(FACEAPI_CDNS, () => !!getFaceApi());
  for (const url of MODEL_URLS) {
    try {
      await loadFaceApiModels(url);
```

```
      return;
    } catch {
      /* try next */
    }
  }
  throw new Error("All face-api model CDNs failed");
}

//     2. BlazeFace loader
async function initBlazeFace(): Promise<void> {
  if (blazefaceModel) return;
  if (blazefaceLoading) return blazefaceLoading;
  blazefaceLoading = (async () => {
    try {
      if (!window.tf) {
        await loadScript(TFJS_CDN);
        for (let i = 0; i < 30; i++) {
          if (window.tf) break;
          await new Promise((r) => setTimeout(r, 100));
        }
      }
      await loadScript(BLAZEFACE_CDN);
      for (let i = 0; i < 30; i++) {
        if (window.blazeface) break;
        await new Promise((r) => setTimeout(r, 100));
      }
      if (window.blazeface?.load) {
        blazefaceModel = await window.blazeface.load();
      }
    } catch {
      blazefaceLoading = null;
    }
  })();
  return blazefaceLoading;
}

//     3. YOLOv8-ONNX loader
async function initYolov8(): Promise<void> {
  if (yolov8Session) return;
  if (yolov8Loading) return yolov8Loading;
  yolov8Loading = (async () => {
    try {
      if (!window.ort) {
        await loadScript(ORT_CDN);
        for (let i = 0; i < 30; i++) {
          if (window.ort) break;
          await new Promise((r) => setTimeout(r, 100));
        }
      }
      if (!window.ort) {
        yolov8Loading = null;
        return;
      }
      for (const url of YOLOV8_ONNX_URLS) {
        try {
```

```
          const session = await window.ort.InferenceSession.create(url, {
            executionProviders: ["wasm"],
            graphOptimizationLevel: "all",
          });
          if (session) {
            yolov8Session = session;
            return;
          }
        } catch {
          /* try next */
        }
      }
      yolov8Loading = null;
    } catch {
      yolov8Loading = null;
    }
  })();
  return yolov8Loading;
}

//     Master init
export async function ensureModelsLoaded(): Promise<void> {
  if (faceApiLoaded) return;
  if (faceApiLoading) return faceApiLoading;

  faceApiLoading = (async () => {
    try {
      await initFaceApi();
      faceApiLoaded = true;
      modelLoadError = null;
      // Load optional models non-blocking in parallel
      Promise.all([
        initBlazeFace().catch(() => {}),
        initYolov8().catch(() => {}),
      ]);
    } catch (e) {
      modelLoadError = (e as Error).message ?? "Model load failed";
      faceApiLoading = null;
    }
  })();

  return faceApiLoading;
}

export function areModelsLoaded(): boolean {
  return faceApiLoaded;
}
export function getModelLoadError(): string | null {
  return modelLoadError;
}

//     Image helpers
export async function loadImageToCanvas(
  src: string,
  width = CANVAS_SIZE,
```

```
  height = CANVAS_SIZE,
): Promise<ImageData> {
  return new Promise((resolve, reject) => {
    const img = new Image();
    img.crossOrigin = "anonymous";
    img.onload = () => {
      const c = document.createElement("canvas");
      c.width = width;
      c.height = height;
      const ctx = c.getContext("2d");
      if (!ctx) return reject(new Error("No canvas context"));
      ctx.drawImage(img, 0, 0, width, height);
      resolve(ctx.getImageData(0, 0, width, height));
    };
    img.onerror = () => reject(new Error("Failed to load image"));
    img.src = src;
  });
}

function loadHTMLImage(src: string): Promise<HTMLImageElement> {
  return new Promise((resolve, reject) => {
    const img = new Image();
    img.crossOrigin = "anonymous";
    img.onload = () => resolve(img);
    img.onerror = () => reject(new Error("Failed to load image"));
    img.src = src;
  });
}

/**
 * Resize image to targetSize x targetSize with contrast normalization.
 */
async function preprocessImage(
  src: string,
  targetSize: number,
): Promise<HTMLImageElement> {
  const imageData = await loadImageToCanvas(src, targetSize, targetSize);
  const data = imageData.data;

  let sumR = 0;
  let sumG = 0;
  let sumB = 0;
  const n = imageData.width * imageData.height;
  for (let i = 0; i < data.length; i += 4) {
    sumR += data[i];
    sumG += data[i + 1];
    sumB += data[i + 2];
  }
  const meanR = sumR / n;
  const meanG = sumG / n;
  const meanB = sumB / n;

  for (let i = 0; i < data.length; i += 4) {
    data[i] = Math.min(
      255,
```

```
      Math.max(0, Math.round((data[i] - meanR) * 1.1 + 128)),
    );
    data[i + 1] = Math.min(
      255,
      Math.max(0, Math.round((data[i + 1] - meanG) * 1.1 + 128)),
    );
    data[i + 2] = Math.min(
      255,
      Math.max(0, Math.round((data[i + 2] - meanB) * 1.1 + 128)),
    );
  }

  const canvas = document.createElement("canvas");
  canvas.width = targetSize;
  canvas.height = targetSize;
  canvas.getContext("2d")!.putImageData(imageData, 0, 0);
  return loadHTMLImage(canvas.toDataURL("image/jpeg", 0.95));
}

//     Histogram helpers
export function extractHistogram(imageData: ImageData): number[] {
  const hist = new Array(BINS * 3).fill(0);
  const data = imageData.data;
  const total = imageData.width * imageData.height;
  for (let i = 0; i < data.length; i += 4) {
    hist[Math.floor((data[i] / 256) * BINS)]++;
    hist[BINS + Math.floor((data[i + 1] / 256) * BINS)]++;
    hist[BINS * 2 + Math.floor((data[i + 2] / 256) * BINS)]++;
  }
  return hist.map((v) => v / total);
}

export function histogramIntersection(h1: number[], h2: number[]): number {
  let intersection = 0;
  let sum = 0;
  for (let i = 0; i < h1.length; i++) {
    intersection += Math.min(h1[i], h2[i]);
    sum += h1[i];
  }
  return sum > 0 ? intersection / sum : 0;
}

//     Age Progression                                                          P
/**
 * Lightweight pixel-level age simulation applied to a registered (case) photo.
 * ageFactor > 0 = age forward ? ageFactor < 0 = de-age
 */
export function applyAgeProgression(
  imageData: ImageData,
  ageFactor: number,
): ImageData {
  const out = new ImageData(
    new Uint8ClampedArray(imageData.data),
    imageData.width,
    imageData.height,
```

```
  );
  const d = out.data;
  if (ageFactor > 0) {
    const rBoost = ageFactor * 18;
    const gDrop = ageFactor * 6;
    for (let i = 0; i < d.length; i += 4) {
      d[i] = Math.min(255, d[i] + rBoost);
      d[i + 1] = Math.min(255, Math.max(0, d[i + 1] - gDrop));
      d[i] = Math.round(Math.min(255, Math.max(0, (d[i] - 128) * 1.04 + 128)));
      d[i + 1] = Math.round(
        Math.min(255, Math.max(0, (d[i + 1] - 128) * 1.04 + 128)),
      );
      d[i + 2] = Math.round(
        Math.min(255, Math.max(0, (d[i + 2] - 128) * 1.04 + 128)),
      );
    }
  } else if (ageFactor < 0) {
    const f = Math.abs(ageFactor);
    const bBoost = f * 10;
    const rDrop = f * 7;
    for (let i = 0; i < d.length; i += 4) {
      d[i] = Math.min(255, Math.max(0, d[i] - rDrop));
      d[i + 2] = Math.min(255, d[i + 2] + bBoost);
      d[i] = Math.round(Math.min(255, Math.max(0, (d[i] - 128) * 0.97 + 128)));
      d[i + 1] = Math.round(
        Math.min(255, Math.max(0, (d[i + 1] - 128) * 0.97 + 128)),
      );
      d[i + 2] = Math.round(
        Math.min(255, Math.max(0, (d[i + 2] - 128) * 0.97 + 128)),
      );
    }
  }
  return out;
}

function imageDataToDataUrl(id: ImageData): string {
  const c = document.createElement("canvas");
  c.width = id.width;
  c.height = id.height;
  c.getContext("2d")!.putImageData(id, 0, 0);
  return c.toDataURL("image/jpeg", 0.9);
}

//     YOLOv8 face crop
interface FaceBox {
  x: number;
  y: number;
  w: number;
  h: number;
  conf: number;
}

async function detectFaceYolov8(
  imgEl: HTMLImageElement,
): Promise<FaceBox | null> {
```

```
if (!yolov8Session) return null;
try {
  const ort = window.ort;
  if (!ort) return null;
  const INPUT_SIZE = 640;
  const canvas = document.createElement("canvas");
  canvas.width = INPUT_SIZE;
  canvas.height = INPUT_SIZE;
  canvas.getContext("2d")!.drawImage(imgEl, 0, 0, INPUT_SIZE, INPUT_SIZE);
  const imageData = canvas
    .getContext("2d")!
    .getImageData(0, 0, INPUT_SIZE, INPUT_SIZE);
  const inputArray = new Float32Array(3 * INPUT_SIZE * INPUT_SIZE);
  for (let y = 0; y < INPUT_SIZE; y++) {
    for (let x = 0; x < INPUT_SIZE; x++) {
      const idx = (y * INPUT_SIZE + x) * 4;
      const pixel = y * INPUT_SIZE + x;
      inputArray[pixel] = imageData.data[idx] / 255;
      inputArray[INPUT_SIZE * INPUT_SIZE + pixel] =
        imageData.data[idx + 1] / 255;
      inputArray[2 * INPUT_SIZE * INPUT_SIZE + pixel] =
        imageData.data[idx + 2] / 255;
    }
  }
  const tensor = new ort.Tensor("float32", inputArray, [
    1,
    3,
    INPUT_SIZE,
    INPUT_SIZE,
  ]);
  const feeds: Record<string, unknown> = {};
  feeds[yolov8Session.inputNames[0]] = tensor;
  const results = await yolov8Session.run(feeds);
  const output = results[yolov8Session.outputNames[0]];
  if (!output) return null;
  const data = output.data as Float32Array;
  const dims = output.dims as number[];
  const isChannelFirst = dims[1] === 5;
  const numDetections = isChannelFirst ? dims[2] : dims[1];
  const scaleX = imgEl.naturalWidth / INPUT_SIZE;
  const scaleY = imgEl.naturalHeight / INPUT_SIZE;
  let bestBox: FaceBox | null = null;
  for (let i = 0; i < numDetections; i++) {
    let cx: number;
    let cy: number;
    let w: number;
    let h: number;
    let conf: number;
    if (isChannelFirst) {
      cx = data[0 * numDetections + i];
      cy = data[1 * numDetections + i];
      w = data[2 * numDetections + i];
      h = data[3 * numDetections + i];
      conf = data[4 * numDetections + i];
    } else {
```

```
        cx = data[i * 5];
        cy = data[i * 5 + 1];
        w = data[i * 5 + 2];
        h = data[i * 5 + 3];
        conf = data[i * 5 + 4];
      }
      if (conf < 0.35) continue;
      const box: FaceBox = {
        x: (cx - w / 2) * scaleX,
        y: (cy - h / 2) * scaleY,
        w: w * scaleX,
        h: h * scaleY,
        conf,
      };
      if (!bestBox || box.w * box.h > bestBox.w * bestBox.h) bestBox = box;
    }
    return bestBox;
  } catch {
    return null;
  }
}

async function cropFaceYolov8(
  imgEl: HTMLImageElement,
): Promise<HTMLImageElement | null> {
  const box = await detectFaceYolov8(imgEl);
  if (!box) return null;
  try {
    const pad = 0.25;
    const cx = Math.max(0, box.x - box.w * pad);
    const cy = Math.max(0, box.y - box.h * pad);
    const cw = Math.min(imgEl.naturalWidth - cx, box.w * (1 + 2 * pad));
    const ch = Math.min(imgEl.naturalHeight - cy, box.h * (1 + 2 * pad));
    if (cw < 20 || ch < 20) return null;
    const canvas = document.createElement("canvas");
    canvas.width = 512;
    canvas.height = 512;
    canvas.getContext("2d")!.drawImage(imgEl, cx, cy, cw, ch, 0, 0, 512, 512);
    return loadHTMLImage(canvas.toDataURL("image/jpeg", 0.95));
  } catch {
    return null;
  }
}

//     BlazeFace face crop
async function cropFaceBlazeFace(
  imgEl: HTMLImageElement,
): Promise<HTMLImageElement | null> {
  if (!blazefaceModel) return null;
  try {
    // eslint-disable-next-line @typescript-eslint/no-explicit-any
    const preds = await (blazefaceModel as any).estimateFaces(imgEl, false);
    if (!preds || preds.length === 0) return null;
    // eslint-disable-next-line @typescript-eslint/no-explicit-any
    const best = preds.reduce((a: any, b: any) =>
```

```
      (b.bottomRight[0] - b.topLeft[0]) * (b.bottomRight[1] - b.topLeft[1]) >
      (a.bottomRight[0] - a.topLeft[0]) * (a.bottomRight[1] - a.topLeft[1])
        ? b
        : a,
    );
    const x1 = best.topLeft[0] as number;
    const y1 = best.topLeft[1] as number;
    const x2 = best.bottomRight[0] as number;
    const y2 = best.bottomRight[1] as number;
    const w = x2 - x1;
    const h = y2 - y1;
    const pad = 0.3;
    const cx = Math.max(0, x1 - w * pad);
    const cy = Math.max(0, y1 - h * pad);
    const cw = Math.min(imgEl.naturalWidth - cx, w * (1 + 2 * pad));
    const ch = Math.min(imgEl.naturalHeight - cy, h * (1 + 2 * pad));
    if (cw < 20 || ch < 20) return null;
    const canvas = document.createElement("canvas");
    canvas.width = 512;
    canvas.height = 512;
    canvas.getContext("2d")!.drawImage(imgEl, cx, cy, cw, ch, 0, 0, 512, 512);
    return loadHTMLImage(canvas.toDataURL("image/jpeg", 0.95));
  } catch {
    return null;
  }
}

//     Landmark geometry
// eslint-disable-next-line @typescript-eslint/no-explicit-any
function landmarkGeometryDescriptor(landmarks: any): Float32Array | null {
  try {
    const pts = landmarks.positions as { x: number; y: number }[];
    if (!pts || pts.length < 68) return null;
    const keyIdx = [
      0, 4, 8, 12, 16, 17, 19, 21, 22, 24, 26, 27, 28, 29, 30, 31, 33, 35, 36,
      37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 50, 52, 54, 56, 58, 60,
      62, 64, 66,
    ];
    const kp = keyIdx.map((i) => pts[i]);
    const xs = kp.map((p) => p.x);
    const ys = kp.map((p) => p.y);
    const minX = Math.min(...xs);
    const maxX = Math.max(...xs);
    const minY = Math.min(...ys);
    const maxY = Math.max(...ys);
    const W = Math.max(maxX - minX, 1);
    const H = Math.max(maxY - minY, 1);
    const norm = kp.map((p) => ({ x: (p.x - minX) / W, y: (p.y - minY) / H }));
    const vec = new Float32Array(norm.length * 2);
    for (let i = 0; i < norm.length; i++) {
      vec[i * 2] = norm[i].x;
      vec[i * 2 + 1] = norm[i].y;
    }
    return vec;
  } catch {
```

```
    return null;
  }
}

function landmarkDistance(a: Float32Array, b: Float32Array): number {
  if (a.length !== b.length) return 1;
  let sum = 0;
  for (let i = 0; i < a.length; i++) {
    const d = a[i] - b[i];
    sum += d * d;
  }
  return Math.sqrt(sum / a.length);
}

function descriptorDistance(a: Float32Array, b: Float32Array): number {
  let sum = 0;
  for (let i = 0; i < a.length; i++) {
    const d = a[i] - b[i];
    sum += d * d;
  }
  return Math.sqrt(sum);
}

//     Distance ’ score mapping
/**
 * Maps a Euclidean CNN descriptor distance [0..2] to a 0-100 score.
 * Distances above CNN_THRESHOLD yield 0 (not a match).
 */
function cnnDistanceToScore(distance: number): number {
  if (distance <= 0.2) return 100;
  if (distance <= 0.3) return Math.round(90 + ((0.3 - distance) / 0.1) * 10);
  if (distance <= 0.45) return Math.round(75 + ((0.45 - distance) / 0.15) * 15);
  if (distance <= 0.55) return Math.round(65 + ((0.55 - distance) / 0.1) * 10);
  if (distance <= 0.65) return Math.round(55 + ((0.65 - distance) / 0.1) * 10);
  if (distance <= CNN_THRESHOLD)
    return Math.round(
      40 + ((CNN_THRESHOLD - distance) / (CNN_THRESHOLD - 0.65)) * 15,
    );
  return 0;
}

//     face-api.js analysis
interface FaceAnalysis {
  descriptor: Float32Array;
  landmarkDesc: Float32Array | null;
  // eslint-disable-next-line @typescript-eslint/no-explicit-any
  detection: any;
}

/**
 * Try to extract a face descriptor from imgEl using face-api.js.
 * Runs parallel detection strategy groups for maximum speed.
 * Returns null only if absolutely no face found.
 */
async function extractFaceAnalysis(
```

```
  imgEl: HTMLImageElement,
): Promise<FaceAnalysis | null> {
  const api = getFaceApi();
  if (!api) return null;

  // eslint-disable-next-line @typescript-eslint/no-explicit-any
  const tryDetect = async (
    opts: any,
    src?: HTMLImageElement,
  ): Promise<FaceAnalysis | null> => {
    const img = src ?? imgEl;
    try {
      const all = await api
        .detectAllFaces(img, opts)
        .withFaceLandmarks()
        .withFaceDescriptors();
      if (all && all.length > 0) {
        // eslint-disable-next-line @typescript-eslint/no-explicit-any
        const best = all.reduce(
          (p: any, c: any) =>
            (c.detection?.box?.width ?? 0) * (c.detection?.box?.height ?? 0) >
            (p.detection?.box?.width ?? 0) * (p.detection?.box?.height ?? 0)
              ? c
              : p,
          all[0],
        );
        if (best?.descriptor) {
          return {
            descriptor: best.descriptor,
            landmarkDesc: best.landmarks
              ? landmarkGeometryDescriptor(best.landmarks)
              : null,
            detection: best.detection,
          };
        }
      }
    } catch {
      /* fall through */
    }
    try {
      const single = await api
        .detectSingleFace(img, opts)
        .withFaceLandmarks()
        .withFaceDescriptor();
      if (single?.descriptor) {
        return {
          descriptor: single.descriptor,
          landmarkDesc: single.landmarks
            ? landmarkGeometryDescriptor(single.landmarks)
            : null,
          detection: single.detection,
        };
      }
    } catch {
      /* fall through */
```

```
  }
  return null;
};

// Group 1: high-confidence parallel strategies
const [r1, r2] = await Promise.all([
  tryDetect(new api.SsdMobilenetv1Options({ minConfidence: 0.5 })),
  tryDetect(
    new api.TinyFaceDetectorOptions({ inputSize: 512, scoreThreshold: 0.4 }),
  ),
]);
if (r1) return r1;
if (r2) return r2;

// Group 2: medium-confidence
const [r3, r4] = await Promise.all([
  tryDetect(new api.SsdMobilenetv1Options({ minConfidence: 0.3 })),
  tryDetect(
    new api.TinyFaceDetectorOptions({ inputSize: 416, scoreThreshold: 0.25 }),
  ),
]);
if (r3) return r3;
if (r4) return r4;

// Group 3: permissive + enhanced crops all in parallel
const [blazeCrop, yoloCrop] = await Promise.all([
  cropFaceBlazeFace(imgEl),
  cropFaceYolov8(imgEl),
]);
const group3Results = await Promise.all([
  tryDetect(new api.SsdMobilenetv1Options({ minConfidence: 0.15 })),
  tryDetect(
    new api.TinyFaceDetectorOptions({ inputSize: 608, scoreThreshold: 0.15 }),
  ),
  blazeCrop
    ? tryDetect(
        new api.SsdMobilenetv1Options({ minConfidence: 0.3 }),
        blazeCrop,
      )
    : Promise.resolve(null),
  blazeCrop
    ? tryDetect(
        new api.TinyFaceDetectorOptions({
          inputSize: 512,
          scoreThreshold: 0.25,
        }),
        blazeCrop,
      )
    : Promise.resolve(null),
  yoloCrop
    ? tryDetect(
        new api.SsdMobilenetv1Options({ minConfidence: 0.3 }),
        yoloCrop,
      )
    : Promise.resolve(null),
```

```
    yoloCrop
      ? tryDetect(
          new api.TinyFaceDetectorOptions({
            inputSize: 512,
            scoreThreshold: 0.25,
          }),
          yoloCrop,
        )
      : Promise.resolve(null),
  ]);
  return group3Results.find((r) => r !== null) ?? null;
}

//     Single-extract helper for search image
export async function extractSearchFaceOnce(
  searchDataUrl: string,
): Promise<FaceAnalysis | null> {
  if (searchFaceCache.has(searchDataUrl)) {
    return searchFaceCache.get(searchDataUrl)!;
  }
  const img = await loadHTMLImage(searchDataUrl);
  const fa = await extractFaceAnalysis(img);
  searchFaceCache.set(searchDataUrl, fa ?? null);
  return fa ?? null;
}

//     Whole-image CNN fallback
async function wholeImageDescriptor(
  imgEl: HTMLImageElement,
): Promise<Float32Array | null> {
  const api = getFaceApi();
  if (!api) return null;
  try {
    const canvas = document.createElement("canvas");
    canvas.width = 224;
    canvas.height = 224;
    canvas.getContext("2d")!.drawImage(imgEl, 0, 0, 224, 224);
    const raw = await api.nets.faceRecognitionNet.computeFaceDescriptor(canvas);
    if (!raw) return null;
    return raw instanceof Float32Array ? raw : new Float32Array(raw);
  } catch {
    return null;
  }
}

//     Core pair comparison
async function compareFacePair(
  searchImg: HTMLImageElement,
  caseImg: HTMLImageElement,
  precomputedSearchFA?: FaceAnalysis | null,
): Promise<number> {
  const [searchFA, caseFA] = await Promise.all([
    precomputedSearchFA !== undefined
      ? Promise.resolve(precomputedSearchFA)
      : extractFaceAnalysis(searchImg),
```

```
    extractFaceAnalysis(caseImg),
  ]);

  if (searchFA && caseFA) {
    const cnnDist = descriptorDistance(searchFA.descriptor, caseFA.descriptor);
    let score = cnnDistanceToScore(cnnDist);

    if (score === 0) return 0;

    if (searchFA.landmarkDesc && caseFA.landmarkDesc) {
      const lmDist = landmarkDistance(
        searchFA.landmarkDesc,
        caseFA.landmarkDesc,
      );
      const lmScore =
        lmDist <= 0.04
          ? 95
          : lmDist <= 0.07
            ? Math.round(80 + ((0.07 - lmDist) / 0.03) * 15)
            : lmDist <= 0.12
              ? Math.round(55 + ((0.12 - lmDist) / 0.05) * 25)
              : lmDist <= 0.2
                ? Math.round(25 + ((0.2 - lmDist) / 0.08) * 30)
                : Math.max(0, Math.round(25 - ((lmDist - 0.2) / 0.2) * 25));
      score = Math.round(score * 0.8 + lmScore * 0.2);
    }

    return Math.max(0, Math.min(100, score));
  }

  // Whole-image CNN fallback
  try {
    const [dA, dB] = await Promise.all([
      searchFA
        ? Promise.resolve(searchFA.descriptor)
        : wholeImageDescriptor(searchImg),
      caseFA
        ? Promise.resolve(caseFA.descriptor)
        : wholeImageDescriptor(caseImg),
    ]);
    if (dA && dB) {
      const dist = descriptorDistance(dA, dB);
      return Math.min(55, cnnDistanceToScore(dist));
    }
  } catch {
    /* ignore */
  }

  return 0;
}

export async function detectFaceInImage(
  imgEl: HTMLImageElement,
): Promise<boolean> {
  const api = getFaceApi();
```

```
  if (!api) return false;
  try {
    const d = await api.detectSingleFace(
      imgEl,
      new api.TinyFaceDetectorOptions({ inputSize: 416, scoreThreshold: 0.3 }),
    );
    if (d) return true;
    const all = await api
      .detectAllFaces(
        imgEl,
        new api.SsdMobilenetv1Options({ minConfidence: 0.2 }),
      )
      .run();
    return !!(all && all.length > 0);
  } catch {
    return false;
  }
}

//     Age progression wrapper
async function compareWithAgeProgression(
  searchImg: HTMLImageElement,
  caseDataUrl: string,
  precomputedSearchFA?: FaceAnalysis | null,
): Promise<number> {
  // 3 age variants in parallel: original, +0.8 forward, -0.4 de-aged
  const ageFactors = [0, 0.8, -0.4];

  const getAgedImg = async (factor: number): Promise<HTMLImageElement> => {
    if (factor === 0) return loadHTMLImage(caseDataUrl);
    const imgData = await loadImageToCanvas(caseDataUrl, 320, 320);
    const aged = applyAgeProgression(imgData, factor);
    return loadHTMLImage(imageDataToDataUrl(aged));
  };

  const scores = await Promise.all(
    ageFactors.map(async (factor) => {
      try {
        const caseImg = await getAgedImg(factor);
        return compareFacePair(searchImg, caseImg, precomputedSearchFA);
      } catch {
        return 0;
      }
    }),
  );

  return Math.max(...scores, 0);
}

//     Public API
/**
 * Main entry point. Computes match score [0-100].
 *   score >= 65 ' MATCH FOUND
 *   score >= 40 ' POSSIBLE MATCH
 *   score >= 1  ' LOW SIMILARITY
```

```
 *   score == 0  ' NO FACIAL MATCH
 */
export async function computeMatchScore(
  searchSource: File | string,
  casePhotoUrl: string,
  onAgeProgressedDataUrl?: (dataUrl: string) => void,
): Promise<number> {
  try {
    let searchDataUrl: string;
    if (typeof searchSource === "string") {
      searchDataUrl = searchSource;
    } else {
      searchDataUrl = await new Promise<string>((resolve, reject) => {
        const reader = new FileReader();
        reader.onload = () => resolve(reader.result as string);
        reader.onerror = reject;
        reader.readAsDataURL(searchSource);
      });
    }

    if (onAgeProgressedDataUrl) {
      try {
        const imgData = await loadImageToCanvas(
          searchDataUrl,
          CANVAS_SIZE,
          CANVAS_SIZE,
        );
        const aged = applyAgeProgression(imgData, 0.6);
        onAgeProgressedDataUrl(imageDataToDataUrl(aged));
      } catch {
        /* non-critical */
      }
    }

    // Histogram fallback when models are not loaded
    if (!faceApiLoaded) {
      const [sd, cd] = await Promise.all([
        loadImageToCanvas(searchDataUrl),
        loadImageToCanvas(casePhotoUrl),
      ]);
      const raw = histogramIntersection(
        extractHistogram(sd),
        extractHistogram(cd),
      );
      return Math.round(
        Math.max(0, Math.min(100, ((raw - 0.3) / 0.5) * 100)) * 0.25,
      );
    }

    // Extract search face ONCE and cache
    const cachedSearchFA = searchFaceCache.has(searchDataUrl)
      ? searchFaceCache.get(searchDataUrl)!
      : await (async () => {
          const naturalImg = await loadHTMLImage(searchDataUrl);
          const fa = await extractFaceAnalysis(naturalImg);
```

```
          searchFaceCache.set(searchDataUrl, fa ?? null);
          return fa ?? null;
        })();

    let bestScore = 0;

    const runPass = async (searchImg: HTMLImageElement): Promise<void> => {
      const score = await compareWithAgeProgression(
        searchImg,
        casePhotoUrl,
        cachedSearchFA,
      );
      if (score > bestScore) bestScore = score;
    };

    // 3 resolution passes in parallel for speed
    const passImages = await Promise.all([
      loadHTMLImage(searchDataUrl),
      preprocessImage(searchDataUrl, 512).catch(() =>
        loadHTMLImage(searchDataUrl),
      ),
      preprocessImage(searchDataUrl, 320).catch(() =>
        loadHTMLImage(searchDataUrl),
      ),
    ]);

    await Promise.all(passImages.map((img) => runPass(img)));

    // If still no confident match, try 640px high-res pass
    if (bestScore < 65) {
      await runPass(
        await preprocessImage(searchDataUrl, 640).catch(() =>
          loadHTMLImage(searchDataUrl),
        ),
      );
    }

    return bestScore;
  } catch {
    return 0;
  }
}

export function computeFrameVariance(imageData: ImageData): number {
  const data = imageData.data;
  let sum = 0;
  let sumSq = 0;
  const pixels = data.length / 4;
  for (let i = 0; i < data.length; i += 4) {
    const g = 0.299 * data[i] + 0.587 * data[i + 1] + 0.114 * data[i + 2];
    sum += g;
    sumSq += g * g;
  }
  const mean = sum / pixels;
  return sumSq / pixels - mean * mean;
```

```
}

export async function preprocessImageForFace(
  src: string,
  targetSize = 320,
): Promise<HTMLImageElement> {
  return preprocessImage(src, targetSize);
}


// ============================================================
// FILE: src/frontend/src/utils/StorageClient.ts
// ============================================================
import { HttpAgent, isV3ResponseBody } from "@icp-sdk/core/agent";
import { IDL } from "@icp-sdk/core/candid";

type Headers = Record<string, string>;

const MAXIMUM_CONCURRENT_UPLOADS = 10;
const MAX_RETRIES = 3;
const BASE_DELAY_MS = 1000;
const MAX_DELAY_MS = 30000;

const GATEWAY_VERSION = "v1";

const HASH_ALGORITHM = "SHA-256";
const SHA256_PREFIX = "sha256:";
const DOMAIN_SEPARATOR_FOR_CHUNKS = new TextEncoder().encode("icfs-chunk/");
const DOMAIN_SEPARATOR_FOR_METADATA = new TextEncoder().encode(
  "icfs-metadata/",
);
const DOMAIN_SEPARATOR_FOR_NODES = new TextEncoder().encode("ynode/");

// Utility function for exponential backoff retry logic - retries on network/server errors only
async function withRetry<T>(operation: () => Promise<T>): Promise<T> {
  let lastError: Error | undefined;

  for (let attempt = 0; attempt <= MAX_RETRIES; attempt++) {
    try {
      return await operation();
    } catch (error) {
      lastError = error instanceof Error ? error : new Error(String(error));

      // Check if this error should be retried
      const shouldRetry = isRetriableError(error);

      // On the final attempt or non-retriable error, throw the error
      if (attempt === MAX_RETRIES || !shouldRetry) {
        if (!shouldRetry && attempt < MAX_RETRIES) {
          console.warn(
            `Non-retriable error encountered: ${lastError.message}. Not retrying.`,
          );
        }
        throw error;
      }
```

```
      // Calculate delay with exponential backoff and jitter
      const delay = Math.min(
        BASE_DELAY_MS * 2 ** attempt + Math.random() * 1000,
        MAX_DELAY_MS,
      );

      console.warn(
        `Request failed (attempt ${attempt + 1}/${MAX_RETRIES + 1}): ${lastError.message}. Retrying
      );

      await new Promise((resolve) => setTimeout(resolve, delay));
    }
  }

  // This should never happen due to the loop logic, but TypeScript needs it
  throw lastError || new Error("Unknown error occurred during retry attempts");
}

function isRetriableError(error: any): boolean {
  const errorMessage = error?.message?.toLowerCase() || "";

  // Don't retry client errors (4xx except specific ones)
  if (error?.response?.status) {
    const status = error.response.status;
    // Only retry these 4xx errors
    if (status === 408 || status === 429) return true;
    // Don't retry other 4xx client errors
    if (status >= 400 && status < 500) return false;
    // Retry 5xx server errors
    if (status >= 500) return true;
  }

  // Retry network/SSL errors
  if (
    errorMessage.includes("ssl") ||
    errorMessage.includes("tls") ||
    errorMessage.includes("network error") ||
    errorMessage.includes("connection") ||
    errorMessage.includes("timeout") ||
    errorMessage.includes("fetch")
  ) {
    return true;
  }

  // Don't retry validation/logic errors
  if (
    errorMessage.includes("validation") ||
    errorMessage.includes("invalid") ||
    errorMessage.includes("malformed") ||
    errorMessage.includes("unauthorized") ||
    errorMessage.includes("forbidden") ||
    errorMessage.includes("not found")
  ) {
    return false;
```

```
  }

  // Default to retry for unknown errors (conservative approach for network issues)
  return true;
}

// Hash validation utility
function validateHashFormat(hash: string, context: string): void {
  if (!hash) {
    throw new Error(`${context}: Hash cannot be empty`);
  }

  if (!hash.startsWith(SHA256_PREFIX)) {
    throw new Error(
      `${context}: Invalid hash format. Expected format: ${SHA256_PREFIX}<64-char-hex>, got: ${hash}
    );
  }

  const hexPart = hash.substring(SHA256_PREFIX.length); // Remove 'sha256:' prefix
  if (hexPart.length !== 64) {
    throw new Error(
      `${context}: Invalid hash format. Expected 64 hex characters after ${SHA256_PREFIX}, got ${hex
    );
  }

  if (!/^[0-9a-f]{64}$/i.test(hexPart)) {
    throw new Error(
      `${context}: Invalid hash format. Hash must contain only hex characters (0-9, a-f), got: ${has
    );
  }
}

class YHash {
  public readonly bytes: Uint8Array;

  constructor(bytes: Uint8Array) {
    if (bytes.length !== 32) {
      throw new Error(`YHash must be exactly 32 bytes, got ${bytes.length}`);
    }
    this.bytes = new Uint8Array(bytes);
  }

  static async fromNodes(
    left: YHash | null,
    right: YHash | null,
  ): Promise<YHash> {
    let leftBytes =
      left instanceof YHash
        ? left.bytes
        : new TextEncoder().encode("UNBALANCED");
    let rightBytes =
      right instanceof YHash
        ? right.bytes
        : new TextEncoder().encode("UNBALANCED");
    const combined = new Uint8Array(
```

```
    DOMAIN_SEPARATOR_FOR_NODES.length + leftBytes.length + rightBytes.length,
  );
  const arrays = [DOMAIN_SEPARATOR_FOR_NODES, leftBytes, rightBytes];
  let offset = 0;
  for (const data of arrays) {
    combined.set(data, offset);
    offset += data.length;
  }
  const hashBuffer = await crypto.subtle.digest(HASH_ALGORITHM, combined);
  return new YHash(new Uint8Array(hashBuffer));
}

static async fromChunk(data: Uint8Array): Promise<YHash> {
  return YHash.fromBytes(DOMAIN_SEPARATOR_FOR_CHUNKS, data);
}

static async fromHeaders(headers: Headers): Promise<YHash> {
  // For each key,value, generate the header line "key: value\n" where the key and value are trimm
  const headerLines: string[] = [];
  for (const [key, value] of Object.entries(headers)) {
    headerLines.push(`${key.trim()}: ${value.trim()}\n`);
  }
  // Sort the header lines alphabetically.
  headerLines.sort();

  // Hash the header lines, with the metadata domain separator.
  const hash = await YHash.fromBytes(
    DOMAIN_SEPARATOR_FOR_METADATA,
    new TextEncoder().encode(headerLines.join("")),
  );
  return hash;
}

static async fromBytes(
  domainSeparator: Uint8Array,
  data: Uint8Array,
): Promise<YHash> {
  const combined = new Uint8Array(domainSeparator.length + data.length);
  combined.set(domainSeparator);
  combined.set(data, domainSeparator.length);
  const hashBuffer = await crypto.subtle.digest(HASH_ALGORITHM, combined);
  return new YHash(new Uint8Array(hashBuffer));
}

public static fromHex(hexString: string): YHash {
  const bytes = new Uint8Array(
    hexString.match(/.{1,2}/g)!.map((byte) => parseInt(byte, 16)),
  );
  return new YHash(bytes);
}

public toShaString(): string {
  return `${SHA256_PREFIX}${this.toHex()}`;
}
```

```
  public toString(): string {
    throw new Error("toString is not supported for YHash");
  }

  private toHex(): string {
    return Array.from(this.bytes)
      .map((b: number) => b.toString(16).padStart(2, "0"))
      .join("");
  }
}

type TreeNode = {
  hash: YHash;
  left: TreeNode | null;
  right: TreeNode | null;
};

type TreeNodeJSON = {
  hash: string;
  left: TreeNodeJSON | null;
  right: TreeNodeJSON | null;
};

function nodeToJSON(node: TreeNode): TreeNodeJSON {
  return {
    hash: node.hash.toShaString(),
    left: node.left ? nodeToJSON(node.left) : null,
    right: node.right ? nodeToJSON(node.right) : null,
  };
}

type BlobHashTreeJSON = {
  tree_type: "DSBMTWH";
  chunk_hashes: string[];
  tree: TreeNodeJSON;
  headers: string[];
};

class BlobHashTree {
  public tree_type: "DSBMTWH";
  public chunk_hashes: YHash[];
  public tree: TreeNode;
  public headers: string[];

  constructor(
    chunk_hashes: YHash[],
    tree: TreeNode,
    headers: string[] | Headers | null = null,
  ) {
    this.tree_type = "DSBMTWH";
    this.chunk_hashes = chunk_hashes;
    this.tree = tree;

    if (headers == null) {
      this.headers = [];
```

```
  } else if (Array.isArray(headers)) {
    this.headers = headers;
  } else {
    this.headers = Object.entries(headers).map(
      ([key, value]) => `${key.trim()}: ${value.trim()}`,
    );
  }
  this.headers.sort();
}

public static async build(
  chunkHashes: YHash[],
  headers: Headers = {},
): Promise<BlobHashTree> {
  if (chunkHashes.length === 0) {
    // To match rust, we have the hash of nothing
    const hex =
      "8b8e620f084e48da0be2287fd12c5aaa4dbe14b468fd2e360f48d741fe7628a0";
    const bytes = new TextEncoder().encode(hex);
    chunkHashes.push(new YHash(bytes));
  }

  // Create leaf nodes for each chunk hash
  let level: TreeNode[] = chunkHashes.map((hash) => ({
    hash,
    left: null,
    right: null,
  }));

  // Build tree bottom-up
  while (level.length > 1) {
    const nextLevel: TreeNode[] = [];
    for (let i = 0; i < level.length; i += 2) {
      const left = level[i];
      const right = level[i + 1] || null;

      const parentHash = await YHash.fromNodes(
        left.hash,
        right ? right.hash : null,
      );
      nextLevel.push({
        hash: parentHash,
        left,
        right,
      });
    }
    level = nextLevel;
  }

  const chunksRoot = level[0];

  // If headers exist and have content, create combined tree
  if (headers && Object.keys(headers).length > 0) {
    const metadataRootHash = await YHash.fromHeaders(headers);
    const metadataRoot: TreeNode = {
```

```
        hash: metadataRootHash,
        left: null,
        right: null,
      };
      const combinedRootHash = await YHash.fromNodes(
        chunksRoot.hash,
        metadataRoot.hash,
      );
      const combinedRoot: TreeNode = {
        hash: combinedRootHash,
        left: chunksRoot,
        right: metadataRoot,
      };
      return new BlobHashTree(chunkHashes, combinedRoot, headers);
    }

    return new BlobHashTree(chunkHashes, chunksRoot, headers);
  }

  public toJSON(): BlobHashTreeJSON {
    return {
      tree_type: this.tree_type,
      chunk_hashes: this.chunk_hashes.map((h) => h.toShaString()),
      tree: nodeToJSON(this.tree),
      headers: this.headers,
    };
  }
}

interface UploadChunkParams {
  blobRootHash: YHash;
  chunkHash: YHash;
  chunkIndex: number;
  chunkData: Uint8Array;
  bucketName: string;
  owner: string;
  projectId: string;
  httpHeaders: Headers;
}

class StorageGatewayClient {
  constructor(private readonly storageGatewayUrl: string) {}

  public getStorageGatewayUrl(): string {
    return this.storageGatewayUrl;
  }

  public async uploadChunk(
    params: UploadChunkParams,
  ): Promise<{ isComplete: boolean }> {
    // Validate hash formats before sending to server (validation errors should not be retried)
    const blobHashString = params.blobRootHash.toShaString();
    const chunkHashString = params.chunkHash.toShaString();
    validateHashFormat(
      blobHashString,
```

```
    `uploadChunk[${params.chunkIndex}] blob_hash`,
  );
  validateHashFormat(
    chunkHashString,
    `uploadChunk[${params.chunkIndex}] chunk_hash`,
  );

  return await withRetry(async () => {
    // Use query parameters for metadata and raw bytes in body
    const queryParams = new URLSearchParams({
      owner_id: params.owner,
      blob_hash: blobHashString,
      chunk_hash: chunkHashString,
      chunk_index: params.chunkIndex.toString(),
      bucket_name: params.bucketName,
      project_id: params.projectId,
    });
    const url = `${this.storageGatewayUrl}/${GATEWAY_VERSION}/chunk/?${queryParams.toString()}`;

    const response = await fetch(url, {
      method: "PUT",
      headers: {
        "Content-Type": "application/octet-stream",
        "X-Caffeine-Project-ID": params.projectId,
      },
      body: params.chunkData as BodyInit,
    });

    if (!response.ok) {
      const errorText = await response.text();
      const error = new Error(
        `Failed to upload chunk ${params.chunkIndex}: ${response.status} ${response.statusText} -
      );
      // Add response status for retry logic
      (error as any).response = { status: response.status };
      throw error;
    }

    const result = (await response.json()) as {
      status: string;
    };
    return {
      isComplete: result.status === "blob_complete",
    };
  });
}

public async uploadBlobTree(
  blobHashTree: BlobHashTree,
  bucketName: string,
  numBlobBytes: number,
  owner: string,
  projectId: string,
  certificateBytes: Uint8Array,
): Promise<void> {
```

```
    // Validate all hashes in the tree before sending to server (validation errors should not be ret
    const treeJSON = blobHashTree.toJSON();
    validateHashFormat(treeJSON.tree.hash, "uploadBlobTree root hash");
    treeJSON.chunk_hashes.forEach((hash, index) => {
      validateHashFormat(hash, `uploadBlobTree chunk_hash[${index}]`);
    });

    return await withRetry(async () => {
      const url = `${this.storageGatewayUrl}/${GATEWAY_VERSION}/blob-tree/`;
      const requestBody = {
        blob_tree: treeJSON,
        bucket_name: bucketName,
        num_blob_bytes: numBlobBytes,
        owner: owner,
        project_id: projectId,
        headers: blobHashTree.headers,
        auth: {
          OwnerEgressSignature: Array.from(certificateBytes),
        },
      };

      const response = await fetch(url, {
        method: "PUT",
        headers: {
          "Content-Type": "application/json",
          "X-Caffeine-Project-ID": projectId,
        },
        body: JSON.stringify(requestBody),
      });

      if (!response.ok) {
        const errorText = await response.text();
        const error = new Error(
          `Failed to upload blob tree: ${response.status} ${response.statusText} - ${errorText}`,
        );
        // Add response status for retry logic
        (error as any).response = { status: response.status };
        throw error;
      }
    });
  }
}

export class StorageClient {
  private readonly storageGatewayClient: StorageGatewayClient;

  public constructor(
    private readonly bucket: string,
    storageGatewayUrl: string,
    private readonly backendCanisterId: string,
    private readonly projectId: string,
    private readonly agent: HttpAgent,
  ) {
    this.storageGatewayClient = new StorageGatewayClient(storageGatewayUrl);
  }
```

```
private async getCertificate(hash: string): Promise<Uint8Array> {
  const args = IDL.encode([IDL.Text], [hash]);
  const result = await this.agent.call(this.backendCanisterId, {
    methodName: "_caffeineStorageCreateCertificate",
    arg: args,
  });
  const respone = result.response.body;
  if (isV3ResponseBody(respone)) {
    console.log("Certificate:", respone.certificate);
    return respone.certificate;
  }
  throw new Error("Expected v3 response body");
}

public async putFile(
  blobBytes: Uint8Array,
  onProgress?: (percentage: number) => void,
): Promise<{ hash: string }> {
  // HTTP headers for fetch requests (used for the PUT request to gateway)
  const httpHeaders: Headers = {
    "Content-Type": "application/json",
  };
  // Create a Blob from the bytes
  const file = new Blob([new Uint8Array(blobBytes)], {
    type: "application/octet-stream",
  });
  // File metadata headers that will be stored with the blob tree
  const fileHeaders: Headers = {
    "Content-Type": "application/octet-stream",
    "Content-Length": file.size.toString(),
  };

  const { chunks, chunkHashes, blobHashTree } =
    await this.processFileForUpload(file, fileHeaders);
  const blobRootHash = blobHashTree.tree.hash;
  const hashString = blobRootHash.toShaString();

  const certificateBytes = await this.getCertificate(hashString);

  await this.storageGatewayClient.uploadBlobTree(
    blobHashTree,
    this.bucket,
    file.size,
    this.backendCanisterId,
    this.projectId,
    certificateBytes,
  );
  await this.parallelUpload(
    chunks,
    chunkHashes,
    blobRootHash,
    httpHeaders,
    onProgress,
  );
```

```
  return { hash: hashString };
}

public async getDirectURL(hash: string): Promise<string> {
  if (!hash) {
    throw new Error("Hash must not be empty");
  }
  validateHashFormat(hash, `getDirectURL for path '${hash}'`);
  return `${this.storageGatewayClient.getStorageGatewayUrl()}/${GATEWAY_VERSION}/blob/?blob_hash=$
}

private async processFileForUpload(
  file: Blob,
  headers: Headers,
): Promise<{
  chunks: Blob[];
  chunkHashes: YHash[];
  blobHashTree: BlobHashTree;
}> {
  const chunks = this.createFileChunks(file);
  const chunkHashes: YHash[] = [];
  for (let i = 0; i < chunks.length; i++) {
    const chunkData = new Uint8Array(await chunks[i].arrayBuffer());
    const hash = await YHash.fromChunk(chunkData);
    chunkHashes.push(hash);
  }
  const blobHashTree = await BlobHashTree.build(chunkHashes, headers);
  return { chunks, chunkHashes, blobHashTree };
}

private async parallelUpload(
  chunks: Blob[],
  chunkHashes: YHash[],
  blobRootHash: YHash,
  httpHeaders: Headers,
  onProgress: ((percentage: number) => void) | undefined,
): Promise<void> {
  let completedChunks = 0;
  const uploadSingleChunk = async (index: number): Promise<void> => {
    const chunkData = new Uint8Array(await chunks[index].arrayBuffer());
    const chunkHash = chunkHashes[index];
    await this.storageGatewayClient.uploadChunk({
      blobRootHash,
      chunkHash,
      chunkIndex: index,
      chunkData,
      bucketName: this.bucket,
      owner: this.backendCanisterId,
      projectId: this.projectId,
      httpHeaders,
    });
    // Use atomic increment to avoid race conditions
    const currentCompleted = ++completedChunks;
    if (onProgress != null) {
      const percentage =
```

```
          chunks.length === 0
            ? 100
            : Math.round((currentCompleted / chunks.length) * 100);
        onProgress(percentage);
      }
    };
    await Promise.all(
      Array.from(
        { length: MAXIMUM_CONCURRENT_UPLOADS },
        async (_, workerId) => {
          for (
            let i = workerId;
            i < chunks.length;
            i += MAXIMUM_CONCURRENT_UPLOADS
          ) {
            await uploadSingleChunk(i);
          }
        },
      ),
    );
  }

  private createFileChunks(file: Blob, chunkSize = 1024 * 1024): Blob[] {
    const chunks: Blob[] = [];
    const totalChunks = Math.ceil(file.size / chunkSize);
    for (let index = 0; index < totalChunks; index++) {
      const start = index * chunkSize;
      const end = Math.min(start + chunkSize, file.size);
      const chunk = file.slice(start, end);
      chunks.push(chunk);
    }
    return chunks;
  }
}


// ============================================================
// FILE: src/frontend/src/utils/urlParams.ts
// ============================================================
/**
 * Utility functions for parsing and managing URL parameters
 * Works with both hash-based and browser-based routing
 */

/**
 * Extracts a URL parameter from the current URL
 * Works with both query strings (?param=value) and hash-based routing (#/?param=value)
 *
 * @param paramName - The name of the parameter to extract
 * @returns The parameter value if found, null otherwise
 */
export function getUrlParameter(paramName: string): string | null {
  // Try to get from regular query string first
  const urlParams = new URLSearchParams(window.location.search);
  const regularParam = urlParams.get(paramName);
```

```
  if (regularParam !== null) {
    return regularParam;
  }

  // If not found, try to extract from hash (for hash-based routing)
  const hash = window.location.hash;
  const queryStartIndex = hash.indexOf("?");

  if (queryStartIndex !== -1) {
    const hashQuery = hash.substring(queryStartIndex + 1);
    const hashParams = new URLSearchParams(hashQuery);
    return hashParams.get(paramName);
  }

  return null;
}

/**
 * Stores a parameter in sessionStorage for persistence across navigation
 * Useful for maintaining state like admin tokens throughout the session
 *
 * @param key - The key to store the value under
 * @param value - The value to store
 */
export function storeSessionParameter(key: string, value: string): void {
  try {
    sessionStorage.setItem(key, value);
  } catch (error) {
    console.warn(`Failed to store session parameter ${key}:`, error);
  }
}

/**
 * Retrieves a parameter from sessionStorage
 *
 * @param key - The key to retrieve
 * @returns The stored value if found, null otherwise
 */
export function getSessionParameter(key: string): string | null {
  try {
    return sessionStorage.getItem(key);
  } catch (error) {
    console.warn(`Failed to retrieve session parameter ${key}:`, error);
    return null;
  }
}

/**
 * Gets a parameter from URL or sessionStorage (URL takes precedence)
 * If found in URL, also stores it in sessionStorage for future use
 *
 * @param paramName - The name of the parameter to retrieve
 * @param storageKey - Optional custom storage key (defaults to paramName)
 * @returns The parameter value if found, null otherwise
```

```
 */
export function getPersistedUrlParameter(
  paramName: string,
  storageKey?: string,
): string | null {
  const key = storageKey || paramName;

  // Check URL first
  const urlValue = getUrlParameter(paramName);
  if (urlValue !== null) {
    // Store in session for persistence
    storeSessionParameter(key, urlValue);
    return urlValue;
  }

  // Fall back to session storage
  return getSessionParameter(key);
}

/**
 * Removes a parameter from sessionStorage
 *
 * @param key - The key to remove
 */
export function clearSessionParameter(key: string): void {
  try {
    sessionStorage.removeItem(key);
  } catch (error) {
    console.warn(`Failed to clear session parameter ${key}:`, error);
  }
}

/**
 * Removes a specific parameter from the URL hash without reloading the page
 * Preserves route information and other parameters in the hash
 * Used to remove sensitive data from the address bar after extracting it
 *
 * @param paramName - The parameter to remove from the hash
 *
 * @example
 * // URL: https://app.com/#/dashboard?caffeineAdminToken=xxx&other=value
 * // After clearParamFromHash('caffeineAdminToken')
 * // URL: https://app.com/#/dashboard?other=value
 */
function clearParamFromHash(paramName: string): void {
  if (!window.history.replaceState) {
    return;
  }

  const hash = window.location.hash;
  if (!hash || hash.length <= 1) {
    return;
  }

  // Remove the leading #
```

```
  const hashContent = hash.substring(1);

  // Split route path from query string
  const queryStartIndex = hashContent.indexOf("?");

  if (queryStartIndex === -1) {
    // No query string in hash, nothing to remove
    return;
  }

  const routePath = hashContent.substring(0, queryStartIndex);
  const queryString = hashContent.substring(queryStartIndex + 1);

  // Parse and remove the specific parameter
  const params = new URLSearchParams(queryString);
  params.delete(paramName);

  // Reconstruct the URL
  const newQueryString = params.toString();
  let newHash = routePath;

  if (newQueryString) {
    newHash += `?${newQueryString}`;
  }

  // If we still have content in the hash, keep it; otherwise remove the hash entirely
  const newUrl =
    window.location.pathname +
    window.location.search +
    (newHash ? `#${newHash}` : "");
  window.history.replaceState(null, "", newUrl);
}

/**
 * Gets a secret from the URL hash fragment only (more secure than query params)
 * Hash fragments aren't sent to servers or logged in access logs
 * The hash is immediately cleared from the URL after extraction to prevent history leakage
 *
 * Usage: https://yourapp.com/#secret=xxx
 *
 * @param paramName - The name of the secret parameter
 * @returns The secret value if found (from hash or session), null otherwise
 */
export function getSecretFromHash(paramName: string): string | null {
  // Check session first to avoid unnecessary URL manipulation
  const existingSecret = getSessionParameter(paramName);
  if (existingSecret !== null) {
    return existingSecret;
  }

  // Try to extract from hash
  const hash = window.location.hash;
  if (!hash || hash.length <= 1) {
    return null;
  }
```

```
  // Remove the leading #
  const hashContent = hash.substring(1);
  const params = new URLSearchParams(hashContent);
  const secret = params.get(paramName);

  if (secret) {
    // Store in session for persistence
    storeSessionParameter(paramName, secret);
    // Immediately clear the secret parameter from URL to avoid history leakage
    clearParamFromHash(paramName);
    return secret;
  }

  return null;
}

/**
 * Gets a secret parameter with fallback chain: hash -> sessionStorage
 * This is the recommended way to handle sensitive parameters like admin tokens
 *
 * Security benefits over regular URL params:
 * - Hash fragments are not sent to the server
 * - Not logged in server access logs
 * - Not sent in HTTP Referer headers
 * - Automatically cleared from URL after extraction
 *
 * @param paramName - The name of the secret parameter
 * @returns The secret value if found, null otherwise
 */
export function getSecretParameter(paramName: string): string | null {
  return getSecretFromHash(paramName);
}


// ============================================================
// FILE: src/frontend/src/lib/utils.ts
// ============================================================
import type { ClassValue } from "clsx";
import { clsx } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]): string {
  return twMerge(clsx(inputs));
}


// ============================================================
// FILE: src/frontend/src/components/Navbar.tsx
// ============================================================
import { Avatar, AvatarFallback } from "@/components/ui/avatar";
import { Button } from "@/components/ui/button";
import {
  DropdownMenu,
  DropdownMenuContent,
```

```
  DropdownMenuItem,
  DropdownMenuTrigger,
} from "@/components/ui/dropdown-menu";
import { useQueryClient } from "@tanstack/react-query";
import { Link, useNavigate, useRouterState } from "@tanstack/react-router";
import { LogOut, Menu, Search, Shield, User, X } from "lucide-react";
import { useState } from "react";
import { useInternetIdentity } from "../hooks/useInternetIdentity";
import { useIsCallerAdmin } from "../hooks/useQueries";

const navLinks = [
  { href: "/dashboard", label: "Dashboard" },
  { href: "/register", label: "Register Child" },
  { href: "/search", label: "Search Cases" },
] as const;

export default function Navbar() {
  const routerState = useRouterState();
  const pathname = routerState.location.pathname;
  const { identity, clear } = useInternetIdentity();
  const queryClient = useQueryClient();
  const navigate = useNavigate();
  const [mobileOpen, setMobileOpen] = useState(false);
  const { data: isAdmin } = useIsCallerAdmin();

  const sessionMode = localStorage.getItem("sessionMode");
  const showAdminLink = isAdmin && sessionMode === "admin";

  const principalShort = identity
    ? `${identity.getPrincipal().toString().slice(0, 6)}...`
    : "Guest";

  const handleLogout = async () => {
    localStorage.removeItem("sessionMode");
    await clear();
    queryClient.clear();
  };

  return (
    <header className="sticky top-0 z-50 bg-card border-b border-border shadow-xs">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
        <div className="flex items-center justify-between h-16">
          {/* Brand */}
          <Link
            to="/dashboard"
            className="flex items-center gap-3 shrink-0"
            data-ocid="nav.link"
          >
            <div className="w-9 h-9 rounded-full bg-primary flex items-center justify-center shadow"
              <Search className="w-4 h-4 text-primary-foreground" />
            </div>
            <div className="leading-tight">
              <p className="text-sm font-bold text-foreground leading-none">
                Missing Child
              </p>
```

```
    <p className="text-xs text-muted-foreground leading-none mt-0.5">
      Finder System
    </p>
  </div>
</Link>

{/* Desktop nav */}
<nav className="hidden md:flex items-center gap-1">
  {navLinks.map((link) => (
    <Link
      key={link.href}
      to={link.href}
      data-ocid="nav.link"
      className={`px-4 py-2 text-sm font-medium rounded-md transition-colors ${
        pathname === link.href
          ? "text-primary border-b-2 border-primary rounded-none"
          : "text-muted-foreground hover:text-foreground hover:bg-muted"
      }`}
    >
      {link.label}
    </Link>
  ))}
  {showAdminLink && (
    <Link
      to="/admin"
      data-ocid="nav.link"
      className={`px-4 py-2 text-sm font-medium rounded-md transition-colors flex items-ce
        pathname === "/admin"
          ? "text-primary border-b-2 border-primary rounded-none"
          : "text-muted-foreground hover:text-foreground hover:bg-muted"
      }`}
    >
      <Shield className="w-3.5 h-3.5" />
      Admin Panel
    </Link>
  )}
</nav>

{/* Right side */}
<div className="flex items-center gap-3">
  {identity ? (
    <DropdownMenu>
      <DropdownMenuTrigger asChild>
        <button
          type="button"
          data-ocid="nav.dropdown_menu"
          className="flex items-center gap-2 px-3 py-1.5 rounded-full border border-border
        >
          <Avatar className="w-6 h-6">
            <AvatarFallback className="bg-primary text-primary-foreground text-xs">
              <User className="w-3 h-3" />
            </AvatarFallback>
          </Avatar>
          <span className="text-sm font-medium hidden sm:block">
            {principalShort}
```

```
            </span>
          </button>
        </DropdownMenuTrigger>
        <DropdownMenuContent align="end">
          <DropdownMenuItem
            onClick={() => navigate({ to: "/account" })}
            data-ocid="nav.link"
          >
            <User className="w-4 h-4 mr-2" />
            My Account
          </DropdownMenuItem>
          {showAdminLink && (
            <DropdownMenuItem
              onClick={() => navigate({ to: "/admin" })}
              data-ocid="nav.link"
            >
              <Shield className="w-4 h-4 mr-2" />
              Admin Panel
            </DropdownMenuItem>
          )}
          <DropdownMenuItem onClick={handleLogout} data-ocid="nav.link">
            <LogOut className="w-4 h-4 mr-2" />
            Logout
          </DropdownMenuItem>
        </DropdownMenuContent>
      </DropdownMenu>
    ) : (
      <Link to="/login">
        <Button size="sm" data-ocid="nav.link">
          Login
        </Button>
      </Link>
    )}

    {/* Mobile menu toggle */}
    <button
      type="button"
      className="md:hidden p-2 rounded-md hover:bg-muted"
      onClick={() => setMobileOpen((v) => !v)}
      aria-label="Toggle menu"
    >
      {mobileOpen ? (
        <X className="w-5 h-5" />
      ) : (
        <Menu className="w-5 h-5" />
      )}
    </button>
  </div>
</div>

{/* Mobile nav */}
{mobileOpen && (
  <nav className="md:hidden border-t border-border py-3 flex flex-col gap-1">
    {navLinks.map((link) => (
      <Link
```

```
                key={link.href}
                to={link.href}
                data-ocid="nav.link"
                onClick={() => setMobileOpen(false)}
                className={`px-4 py-2 text-sm font-medium rounded-md ${
                  pathname === link.href
                    ? "bg-primary/10 text-primary"
                    : "text-muted-foreground hover:text-foreground hover:bg-muted"
                }`}
              >
                {link.label}
              </Link>
            ))}
            {showAdminLink && (
              <Link
                to="/admin"
                data-ocid="nav.link"
                onClick={() => setMobileOpen(false)}
                className={`px-4 py-2 text-sm font-medium rounded-md flex items-center gap-1.5 ${
                  pathname === "/admin"
                    ? "bg-primary/10 text-primary"
                    : "text-muted-foreground hover:text-foreground hover:bg-muted"
                }`}
              >
                <Shield className="w-3.5 h-3.5" />
                Admin Panel
              </Link>
            )}
          </nav>
        )}
      </div>
    </header>
  );
}


// ============================================================
// FILE: src/frontend/src/components/ProtectedRoute.tsx
// ============================================================
import { Navigate } from "@tanstack/react-router";
import type { ReactNode } from "react";
import { useInternetIdentity } from "../hooks/useInternetIdentity";

interface ProtectedRouteProps {
  children: ReactNode;
}

export default function ProtectedRoute({ children }: ProtectedRouteProps) {
  const { identity, isInitializing } = useInternetIdentity();

  if (isInitializing) {
    return (
      <div className="min-h-screen flex items-center justify-center bg-background">
        <div className="text-center">
          <div className="inline-block w-10 h-10 border-4 border-primary/30 border-t-primary rounded
```

```
          <p className="text-muted-foreground text-sm">Loading...</p>
        </div>
      </div>
    );
  }

  if (!identity) {
    return <Navigate to="/login" />;
  }

  return <>{children}</>;
}


// ============================================================
// FILE: src/frontend/src/pages/LandingPage.tsx
// ============================================================
import { Navigate } from "@tanstack/react-router";
export default function LandingPage() {
  return <Navigate to="/login" />;
}


// ============================================================
// FILE: src/frontend/src/pages/LoginPage.tsx
// ============================================================
import { Alert, AlertDescription } from "@/components/ui/alert";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { Tabs, TabsContent, TabsList, TabsTrigger } from "@/components/ui/tabs";
import { useNavigate } from "@tanstack/react-router";
import { AlertCircle, Eye, EyeOff, Search, Shield } from "lucide-react";
import { motion } from "motion/react";
import { useState } from "react";
import { useActor } from "../hooks/useActor";
import { useInternetIdentity } from "../hooks/useInternetIdentity";

export default function LoginPage() {
  const navigate = useNavigate();
  const { login, loginStatus, identity } = useInternetIdentity();
  const { actor } = useActor();

  const [signInForm, setSignInForm] = useState({ username: "", password: "" });
  const [signUpForm, setSignUpForm] = useState({
    username: "",
    email: "",
    password: "",
    confirmPassword: "",
  });
  const [showPassword, setShowPassword] = useState(false);
  const [error, setError] = useState("");
  const [activeTab, setActiveTab] = useState("signin");
  const [adminPassword, setAdminPassword] = useState("");
  const [adminError, setAdminError] = useState("");
```

```
const [adminLoading, setAdminLoading] = useState(false);

// Only auto-redirect if NOT on the admin tab
if (identity && activeTab !== "admin") {
  navigate({ to: "/dashboard" });
  return null;
}

const isLoggingIn = loginStatus === "logging-in";

const handleSignIn = async (e: React.FormEvent) => {
  e.preventDefault();
  setError("");
  if (!signInForm.username.trim()) {
    setError("Username is required");
    return;
  }
  if (!signInForm.password.trim()) {
    setError("Password is required");
    return;
  }
  try {
    await login();
    localStorage.setItem("sessionMode", "user");
    navigate({ to: "/dashboard" });
  } catch (err: any) {
    setError(err?.message || "Login failed. Please try again.");
  }
};

const handleSignUp = async (e: React.FormEvent) => {
  e.preventDefault();
  setError("");
  if (!signUpForm.username.trim()) {
    setError("Username is required");
    return;
  }
  if (!signUpForm.email.trim()) {
    setError("Email is required");
    return;
  }
  if (!signUpForm.password.trim() || signUpForm.password.length < 6) {
    setError("Password must be at least 6 characters");
    return;
  }
  if (signUpForm.password !== signUpForm.confirmPassword) {
    setError("Passwords do not match");
    return;
  }
  try {
    await login();
    if (actor) {
      await actor.saveCallerUserProfile({ name: signUpForm.username });
    }
    localStorage.setItem("sessionMode", "user");
```

```
    navigate({ to: "/dashboard" });
  } catch (err: any) {
    setError(err?.message || "Sign up failed. Please try again.");
  }
};

const handleAdminLogin = async (e: React.FormEvent) => {
  e.preventDefault();
  setAdminError("");
  if (!adminPassword.trim()) {
    setAdminError("Admin password is required");
    return;
  }
  if (!actor) {
    setAdminError("Not connected. Please wait and try again.");
    return;
  }
  setAdminLoading(true);
  try {
    await actor.claimAdminRole(adminPassword);
    localStorage.setItem("sessionMode", "admin");
    navigate({ to: "/admin" });
  } catch (_err: unknown) {
    setAdminError("Invalid admin password. Please try again.");
  } finally {
    setAdminLoading(false);
  }
};

return (
  <div className="min-h-screen bg-background flex items-center justify-center px-4 py-12">
    <motion.div
      initial={{ opacity: 0, y: 20 }}
      animate={{ opacity: 1, y: 0 }}
      transition={{ duration: 0.4 }}
      className="w-full max-w-md"
    >
      {/* Logo + Brand */}
      <div className="text-center mb-8">
        <div className="inline-flex items-center justify-center w-16 h-16 rounded-full bg-primary
          <Search className="w-7 h-7 text-primary-foreground" />
        </div>
        <h1 className="text-2xl font-bold text-foreground">
          Missing Child Finder
        </h1>
        <p className="text-muted-foreground text-sm mt-1">
          Secure portal for authorized personnel
        </p>
      </div>

      {/* Card */}
      <div className="bg-card rounded-xl shadow-card border border-border p-8">
        <Tabs
          value={activeTab}
          onValueChange={(v) => {
```

```
      setActiveTab(v);
      setError("");
      setAdminError("");
    }}
  >
    <TabsList
      className="grid grid-cols-3 w-full mb-6"
      data-ocid="login.tab"
    >
      <TabsTrigger value="signin" data-ocid="login.tab">
        Sign In
      </TabsTrigger>
      <TabsTrigger value="signup" data-ocid="login.tab">
        Sign Up
      </TabsTrigger>
      <TabsTrigger value="admin" data-ocid="login.tab">
        <Shield className="w-3.5 h-3.5 mr-1" />
        Admin
      </TabsTrigger>
    </TabsList>

    {error && activeTab !== "admin" && (
      <Alert
        variant="destructive"
        className="mb-4"
        data-ocid="login.error_state"
      >
        <AlertCircle className="h-4 w-4" />
        <AlertDescription>{error}</AlertDescription>
      </Alert>
    )}

    <TabsContent value="signin">
      <form onSubmit={handleSignIn} className="space-y-4">
        <div className="space-y-1.5">
          <Label htmlFor="signin-username">Username</Label>
          <Input
            id="signin-username"
            data-ocid="login.input"
            placeholder="Enter your username"
            value={signInForm.username}
            onChange={(e) =>
              setSignInForm((p) => ({ ...p, username: e.target.value }))
            }
            autoComplete="username"
          />
        </div>
        <div className="space-y-1.5">
          <Label htmlFor="signin-password">Password</Label>
          <div className="relative">
            <Input
              id="signin-password"
              data-ocid="login.input"
              type={showPassword ? "text" : "password"}
              placeholder="Enter your password"
```

```
          value={signInForm.password}
          onChange={(e) =>
            setSignInForm((p) => ({
              ...p,
              password: e.target.value,
            }))
          }
          autoComplete="current-password"
          className="pr-10"
        />
        <button
          type="button"
          className="absolute right-3 top-1/2 -translate-y-1/2 text-muted-foreground hov
          onClick={() => setShowPassword((v) => !v)}
        >
          {showPassword ? (
            <EyeOff className="w-4 h-4" />
          ) : (
            <Eye className="w-4 h-4" />
          )}
        </button>
      </div>
    </div>
    <Button
      type="submit"
      className="w-full bg-primary text-primary-foreground hover:bg-primary/90"
      disabled={isLoggingIn}
      data-ocid="login.submit_button"
    >
      {isLoggingIn ? (
        <span className="flex items-center gap-2">
          <span className="w-4 h-4 border-2 border-white/30 border-t-white rounded-full
          Signing in...
        </span>
      ) : (
        "Sign In"
      )}
    </Button>
  </form>
</TabsContent>

<TabsContent value="signup">
  <form onSubmit={handleSignUp} className="space-y-4">
    <div className="space-y-1.5">
      <Label htmlFor="signup-username">Username</Label>
      <Input
        id="signup-username"
        data-ocid="register.input"
        placeholder="Choose a username"
        value={signUpForm.username}
        onChange={(e) =>
          setSignUpForm((p) => ({ ...p, username: e.target.value }))
        }
        autoComplete="username"
      />
```

```
</div>
<div className="space-y-1.5">
  <Label htmlFor="signup-email">Email Address</Label>
  <Input
    id="signup-email"
    data-ocid="register.input"
    type="email"
    placeholder="Enter your email"
    value={signUpForm.email}
    onChange={(e) =>
      setSignUpForm((p) => ({ ...p, email: e.target.value }))
    }
    autoComplete="email"
  />
</div>
<div className="space-y-1.5">
  <Label htmlFor="signup-password">Password</Label>
  <div className="relative">
    <Input
      id="signup-password"
      data-ocid="register.input"
      type={showPassword ? "text" : "password"}
      placeholder="Create a password"
      value={signUpForm.password}
      onChange={(e) =>
        setSignUpForm((p) => ({
          ...p,
          password: e.target.value,
        }))
      }
      autoComplete="new-password"
      className="pr-10"
    />
    <button
      type="button"
      className="absolute right-3 top-1/2 -translate-y-1/2 text-muted-foreground hov
      onClick={() => setShowPassword((v) => !v)}
    >
      {showPassword ? (
        <EyeOff className="w-4 h-4" />
      ) : (
        <Eye className="w-4 h-4" />
      )}
    </button>
  </div>
</div>
<div className="space-y-1.5">
  <Label htmlFor="signup-confirm">Confirm Password</Label>
  <Input
    id="signup-confirm"
    data-ocid="register.input"
    type="password"
    placeholder="Confirm your password"
    value={signUpForm.confirmPassword}
    onChange={(e) =>
```

```
          setSignUpForm((p) => ({
            ...p,
            confirmPassword: e.target.value,
          }))
        }
        autoComplete="new-password"
      />
    </div>
    <Button
      type="submit"
      className="w-full bg-secondary text-secondary-foreground hover:bg-secondary/90"
      disabled={isLoggingIn}
      data-ocid="register.submit_button"
    >
      {isLoggingIn ? (
        <span className="flex items-center gap-2">
          <span className="w-4 h-4 border-2 border-white/30 border-t-white rounded-full
          Creating account...
        </span>
      ) : (
        "Create Account"
      )}
    </Button>
  </form>
</TabsContent>

<TabsContent value="admin">
  <div className="space-y-5">
    <div className="flex items-center gap-3 p-4 rounded-lg bg-muted/50 border border-bor
      <Shield className="w-5 h-5 text-primary shrink-0" />
      <div>
        <p className="text-sm font-medium text-foreground">
          Admin Access
        </p>
        <p className="text-xs text-muted-foreground mt-0.5">
          Requires Internet Identity authentication first.
        </p>
      </div>
    </div>

    {adminError && (
      <Alert variant="destructive" data-ocid="login.error_state">
        <AlertCircle className="h-4 w-4" />
        <AlertDescription>{adminError}</AlertDescription>
      </Alert>
    )}

    {!identity ? (
      <div className="space-y-3">
        <p className="text-sm text-muted-foreground">
          Step 1: Authenticate with Internet Identity
        </p>
        <Button
          className="w-full"
          onClick={() => {
```

```
                setAdminError("");
                login();
              }}
              disabled={isLoggingIn}
              data-ocid="login.primary_button"
            >
              {isLoggingIn ? (
                <span className="flex items-center gap-2">
                  <span className="w-4 h-4 border-2 border-white/30 border-t-white rounded-f
                  Connecting...
                </span>
              ) : (
                "Sign in with Internet Identity"
              )}
            </Button>
          </div>
        ) : (
          <form onSubmit={handleAdminLogin} className="space-y-4">
            <p className="text-sm text-muted-foreground">
              Step 2: Enter your admin password
            </p>
            <div className="space-y-1.5">
              <Label htmlFor="admin-password">Admin Password</Label>
              <Input
                id="admin-password"
                data-ocid="login.input"
                type="password"
                placeholder="Enter admin password"
                value={adminPassword}
                onChange={(e) => setAdminPassword(e.target.value)}
                autoComplete="current-password"
              />
            </div>
            <Button
              type="submit"
              className="w-full bg-primary text-primary-foreground hover:bg-primary/90"
              disabled={adminLoading}
              data-ocid="login.submit_button"
            >
              {adminLoading ? (
                <span className="flex items-center gap-2">
                  <span className="w-4 h-4 border-2 border-white/30 border-t-white rounded-f
                  Verifying...
                </span>
              ) : (
                "Access Admin Panel"
              )}
            </Button>
          </form>
        )}
      </div>
    </TabsContent>
  </Tabs>
</div>
```

```
        <p className="text-center text-xs text-muted-foreground mt-6">
          Authorized personnel only. Unauthorized access is prohibited.
        </p>
      </motion.div>
    </div>
  );
}


// ============================================================
// FILE: src/frontend/src/pages/DashboardPage.tsx
// ============================================================
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Skeleton } from "@/components/ui/skeleton";
import { useNavigate } from "@tanstack/react-router";
import {
  AlertTriangle,
  CheckCircle,
  Clock,
  Eye,
  Plus,
  Search,
  Users,
} from "lucide-react";
import { motion } from "motion/react";
import { useState } from "react";
import Navbar from "../components/Navbar";
import {
  Status,
  useGetAllCases,
  useGetDashboardStats,
} from "../hooks/useQueries";

export default function DashboardPage() {
  const navigate = useNavigate();
  const { data: stats, isLoading: statsLoading } = useGetDashboardStats();
  const { data: cases, isLoading: casesLoading } = useGetAllCases();
  const [search, setSearch] = useState("");

  const filteredCases = (cases ?? []).filter(
    (c) =>
      c.name.toLowerCase().includes(search.toLowerCase()) ||
      c.lastLocation.toLowerCase().includes(search.toLowerCase()) ||
      c.contactNumber.includes(search),
  );

  const kpiCards = [
    {
      label: "Total Cases",
      value: stats ? Number(stats.totalCases) : "-",
      icon: Users,
      color: "text-primary",
      bg: "bg-primary/10",
```

```
  },
  {
    label: "Active Cases",
    value: stats ? Number(stats.activeCases) : "-",
    icon: AlertTriangle,
    color: "text-destructive",
    bg: "bg-destructive/10",
  },
  {
    label: "Found / Resolved",
    value: stats ? Number(stats.foundCases) : "-",
    icon: CheckCircle,
    color: "text-success",
    bg: "bg-success/10",
  },
  {
    label: "Pending Matches",
    value: stats ? Math.max(0, Number(stats.activeCases)) : "-",
    icon: Clock,
    color: "text-warning",
    bg: "bg-warning/10",
  },
];

return (
  <div className="min-h-screen bg-background flex flex-col">
    <Navbar />

    {/* Hero strip */}
    <div className="bg-card border-b border-border">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-6">
        <div className="flex flex-col sm:flex-row sm:items-center justify-between gap-4">
          <div>
            <h1 className="text-2xl sm:text-3xl font-bold text-foreground">
              Active Case Dashboard
            </h1>
            <p className="text-muted-foreground text-sm mt-1">
              Monitor and manage missing child cases in real time
            </p>
          </div>
          <div className="relative w-full sm:w-72">
            <Search className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4 text-muted-foregro
            <Input
              data-ocid="dashboard.search_input"
              placeholder="Quick search cases..."
              className="pl-9"
              value={search}
              onChange={(e) => setSearch(e.target.value)}
            />
          </div>
        </div>
      </div>
    </div>

    <main className="flex-1 max-w-7xl mx-auto w-full px-4 sm:px-6 lg:px-8 py-6">
```

```
{/* Welcome + CTA */}
<div className="flex items-center justify-between mb-6">
  <p className="text-foreground font-semibold text-lg">Welcome Back</p>
  <Button
    data-ocid="dashboard.primary_button"
    onClick={() => navigate({ to: "/register" })}
    className="bg-primary text-primary-foreground hover:bg-primary/90 gap-2"
  >
    <Plus className="w-4 h-4" />
    Add Child Case
  </Button>
</div>

{/* KPI Cards */}
<div className="grid grid-cols-2 lg:grid-cols-4 gap-4 mb-6">
  {kpiCards.map((card, i) => (
    <motion.div
      key={card.label}
      initial={{ opacity: 0, y: 12 }}
      animate={{ opacity: 1, y: 0 }}
      transition={{ delay: i * 0.08 }}
      className="bg-card rounded-xl border border-border shadow-card p-5"
      data-ocid="dashboard.card"
    >
      <div className="flex items-center justify-between mb-3">
        <span className="text-xs font-medium text-muted-foreground uppercase tracking-wide">
          {card.label}
        </span>
        <div
          className={`w-8 h-8 rounded-lg ${card.bg} flex items-center justify-center`}
        >
          <card.icon className={`w-4 h-4 ${card.color}`} />
        </div>
      </div>
      {statsLoading ? (
        <Skeleton
          className="h-8 w-16"
          data-ocid="dashboard.loading_state"
        />
      ) : (
        <p className={`text-3xl font-bold ${card.color}`}>
          {card.value}
        </p>
      )}
    </motion.div>
  ))}
</div>

{/* Cases Table */}
<div className="bg-card rounded-xl border border-border shadow-card overflow-hidden">
  <div className="px-6 py-4 border-b border-border flex items-center justify-between">
    <h2 className="text-base font-semibold text-foreground">
      Recently Registered Children
    </h2>
    <Badge variant="secondary">{filteredCases.length} cases</Badge>
```

```
</div>

{casesLoading ? (
  <div className="p-6 space-y-3" data-ocid="dashboard.loading_state">
    {[1, 2, 3].map((i) => (
      <Skeleton key={i} className="h-14 w-full" />
    ))}
  </div>
) : filteredCases.length === 0 ? (
  <div
    className="py-16 text-center"
    data-ocid="dashboard.empty_state"
  >
    <Users className="w-10 h-10 text-muted-foreground mx-auto mb-3" />
    <p className="text-muted-foreground">
      {search
        ? "No matching cases found."
        : "No cases registered yet."}
    </p>
    {!search && (
      <Button
        variant="outline"
        className="mt-4"
        onClick={() => navigate({ to: "/register" })}
        data-ocid="dashboard.secondary_button"
      >
        Register First Case
      </Button>
    )}
  </div>
) : (
  <div className="overflow-x-auto">
    <table className="w-full text-sm">
      <thead>
        <tr className="border-b border-border bg-muted/30">
          <th className="text-left px-6 py-3 text-xs font-semibold text-muted-foreground u
            Child
          </th>
          <th className="text-left px-4 py-3 text-xs font-semibold text-muted-foreground u
            Age
          </th>
          <th className="text-left px-4 py-3 text-xs font-semibold text-muted-foreground u
            Last Location
          </th>
          <th className="text-left px-4 py-3 text-xs font-semibold text-muted-foreground u
            Last Seen Place
          </th>
          <th className="text-left px-4 py-3 text-xs font-semibold text-muted-foreground u
            Status
          </th>
          <th className="text-right px-6 py-3 text-xs font-semibold text-muted-foreground
            Action
          </th>
        </tr>
      </thead>
```

```
<tbody>
  {filteredCases.map((c, idx) => (
    <tr
      key={c.contactNumber}
      className="border-b border-border/50 hover:bg-muted/20 transition-colors"
      data-ocid={`dashboard.item.${idx + 1}`}
    >
      <td className="px-6 py-4">
        <div className="flex items-center gap-3">
          <div className="w-9 h-9 rounded-full overflow-hidden bg-primary/10 flex it
            {c.photoId ? (
              <img
                src={c.photoId}
                alt={c.name}
                className="w-full h-full object-cover"
                onError={(e) => {
                  (e.target as HTMLImageElement).style.display =
                    "none";
                }}
              />
            ) : (
              <span className="text-primary font-bold text-sm">
                {c.name.charAt(0)}
              </span>
            )}
          </div>
          <div>
            <p className="font-semibold text-foreground">
              {c.name}
            </p>
            <p className="text-xs text-muted-foreground">
              {c.contactNumber}
            </p>
          </div>
        </div>
      </td>
      <td className="px-4 py-4 text-muted-foreground">
        {Number(c.age)} yrs
      </td>
      <td className="px-4 py-4 text-muted-foreground hidden md:table-cell max-w-[140
        {c.lastLocation}
      </td>
      <td className="px-4 py-4 text-muted-foreground hidden lg:table-cell max-w-[140
        {c.lastSeenPlace}
      </td>
      <td className="px-4 py-4">
        {c.status === Status.active ? (
          <span className="inline-flex items-center px-2.5 py-0.5 rounded-full text-
            Active
          </span>
        ) : (
          <span className="inline-flex items-center px-2.5 py-0.5 rounded-full text-
            Found
          </span>
        )}
```

```
                    </td>
                    <td className="px-6 py-4 text-right">
                      <Button
                        size="sm"
                        variant="outline"
                        onClick={() => navigate({ to: "/search" })}
                        data-ocid="dashboard.edit_button"
                        className="gap-1 text-xs"
                      >
                        <Eye className="w-3 h-3" />
                        View
                      </Button>
                    </td>
                  </tr>
                ))}
              </tbody>
            </table>
          </div>
        )}
      </div>
    </main>

    {/* Footer */}
    <footer className="bg-footer text-footer-foreground mt-auto">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-6">
        <p className="text-sm text-center">
          &copy; {new Date().getFullYear()} Active Child Finder. Built with
          love using{" "}
          <a
            href={`https://caffeine.ai?utm_source=caffeine-footer&utm_medium=referral&utm_content=
            target="_blank"
            rel="noopener noreferrer"
            className="underline hover:opacity-80"
          >
            caffeine.ai
          </a>
        </p>
      </div>
    </footer>
  </div>
);
}


// ============================================================
// FILE: src/frontend/src/pages/RegisterPage.tsx
// ============================================================
import { Alert, AlertDescription } from "@/components/ui/alert";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import {
  Select,
  SelectContent,
  SelectItem,
```

```
  SelectTrigger,
  SelectValue,
} from "@/components/ui/select";
import { useNavigate } from "@tanstack/react-router";
import {
  AlertCircle,
  CheckCircle,
  Loader2,
  RefreshCw,
  Upload,
  X,
} from "lucide-react";
import { AnimatePresence, motion } from "motion/react";
import { useRef, useState } from "react";
import { Status } from "../backend";
import Navbar from "../components/Navbar";
import { useActor } from "../hooks/useActor";
import { useInternetIdentity } from "../hooks/useInternetIdentity";
import { useRegisterCase } from "../hooks/useQueries";

interface FormData {
  name: string;
  age: string;
  lastLocation: string;
  contactNumber: string;
  lastSeenPlace: string;
  status: string;
}

const INITIAL_FORM: FormData = {
  name: "",
  age: "",
  lastLocation: "",
  contactNumber: "",
  lastSeenPlace: "",
  status: "active",
};

function mapStatus(val: string): Status {
  switch (val) {
    case "underReview":
      return Status.underReview;
    case "found":
      return Status.found;
    case "closed":
      return Status.closed;
    default:
      return Status.active;
  }
}

function isCanisterStoppedError(msg: string): boolean {
  return (
    msg.includes("IC0508") ||
    msg.includes("is stopped") ||
```

```
    msg.includes("canister is stopping")
  );
}

async function compressImage(dataUrl: string): Promise<string> {
  return new Promise((resolve) => {
    const img = new Image();
    img.onload = () => {
      const MAX_DIM = 600;
      let { width, height } = img;
      if (width > MAX_DIM || height > MAX_DIM) {
        if (width > height) {
          height = Math.round((height * MAX_DIM) / width);
          width = MAX_DIM;
        } else {
          width = Math.round((width * MAX_DIM) / height);
          height = MAX_DIM;
        }
      }
      const canvas = document.createElement("canvas");
      canvas.width = width;
      canvas.height = height;
      const ctx = canvas.getContext("2d");
      if (!ctx) {
        resolve(dataUrl);
        return;
      }
      ctx.drawImage(img, 0, 0, width, height);
      resolve(canvas.toDataURL("image/jpeg", 0.65));
    };
    img.onerror = () => resolve(dataUrl);
    img.src = dataUrl;
  });
}

export default function RegisterPage() {
  const navigate = useNavigate();
  const {
    actor,
    isFetching: actorLoading,
    isError: actorError,
    refetch: refetchActor,
  } = useActor();
  const { identity } = useInternetIdentity();
  const { mutateAsync: registerCase, isPending } = useRegisterCase();

  const [form, setForm] = useState<FormData>(INITIAL_FORM);
  const [errors, setErrors] = useState<Partial<FormData>>({});
  const [photoDataUrl, setPhotoDataUrl] = useState<string | null>(null);
  const [photoFileName, setPhotoFileName] = useState<string | null>(null);
  const [isCompressing, setIsCompressing] = useState(false);
  const [isDragging, setIsDragging] = useState(false);
  const [successInfo, setSuccessInfo] = useState<{
    name: string;
    contact: string;
```

```
} | null>(null);
const [submitError, setSubmitError] = useState("");
const fileInputRef = useRef<HTMLInputElement>(null);

const handleField = (key: keyof FormData, val: string) => {
  setForm((p) => ({ ...p, [key]: val }));
  setErrors((p) => ({ ...p, [key]: undefined }));
};

const handlePhoto = async (file: File) => {
  setPhotoFileName(file.name);
  setIsCompressing(true);
  const reader = new FileReader();
  reader.onload = async () => {
    const raw = reader.result as string;
    const compressed = await compressImage(raw);
    setPhotoDataUrl(compressed);
    setIsCompressing(false);
  };
  reader.readAsDataURL(file);
};

const handleDrop = (e: React.DragEvent) => {
  e.preventDefault();
  setIsDragging(false);
  const file = e.dataTransfer.files[0];
  if (file?.type.startsWith("image/")) {
    handlePhoto(file);
  }
};

const validate = (): boolean => {
  const errs: Partial<FormData> = {};
  if (!form.name.trim()) errs.name = "Name is required";
  if (
    !form.age.trim() ||
    Number.isNaN(Number(form.age)) ||
    Number(form.age) <= 0
  )
    errs.age = "Valid age is required";
  if (!form.lastLocation.trim())
    errs.lastLocation = "Last location is required";
  if (!form.contactNumber.trim())
    errs.contactNumber = "Contact number is required";
  if (!form.lastSeenPlace.trim())
    errs.lastSeenPlace = "Last seen place is required";
  setErrors(errs);
  return Object.keys(errs).length === 0;
};

const attemptRegister = async (retries = 3): Promise<void> => {
  try {
    await registerCase({
      name: form.name.trim(),
      age: BigInt(Math.round(Number(form.age))),
```

```
      lastLocation: form.lastLocation.trim(),
      contactNumber: form.contactNumber.trim(),
      lastSeenPlace: form.lastSeenPlace.trim(),
      status: mapStatus(form.status),
      photoId: photoDataUrl ?? "",
    });
    setSuccessInfo({
      name: form.name.trim(),
      contact: form.contactNumber.trim(),
    });
    setForm(INITIAL_FORM);
    setPhotoDataUrl(null);
    setPhotoFileName(null);
  } catch (err: unknown) {
    const msg: string = (err as { message?: string })?.message ?? "";
    if (isCanisterStoppedError(msg) && retries > 0) {
      await new Promise((r) => setTimeout(r, 3000));
      return attemptRegister(retries - 1);
    }
    throw err;
  }
};

const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  setSubmitError("");

  if (!identity) {
    setSubmitError(
      "You must be logged in to register a case. Please log in first.",
    );
    return;
  }

  if (!validate()) return;

  if (!actor) {
    setSubmitError(
      "Backend connection failed. Please use the Retry button above or refresh the page.",
    );
    return;
  }

  if (isCompressing) {
    setSubmitError("Please wait for the photo to finish processing.");
    return;
  }

  try {
    await attemptRegister(3);
  } catch (err: unknown) {
    const msg: string = (err as { message?: string })?.message ?? "";
    if (isCanisterStoppedError(msg)) {
      setSubmitError(
        "The backend is temporarily restarting. Please wait 30 seconds and try again.",
```

```
      );
    } else if (msg.includes("already exists")) {
      setSubmitError(
        "A case with this contact number already exists. Please use a different contact number.",
      );
    } else if (
      msg.includes("Not authenticated") ||
      msg.includes("Unauthorized")
    ) {
      setSubmitError(
        "Your session expired. Please log out and log in again.",
      );
    } else {
      setSubmitError(msg || "Registration failed. Please try again.");
    }
  }
};

const isSubmitDisabled =
  isPending || actorLoading || !identity || isCompressing;

return (
  <div className="min-h-screen bg-background flex flex-col">
    <Navbar />

    <div className="bg-card border-b border-border">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-6">
        <h1 className="text-2xl font-bold text-foreground">
          Register Missing Child
        </h1>
        <p className="text-muted-foreground text-sm mt-1">
          Fill in all details to register a missing child case
        </p>
      </div>
    </div>

    <main className="flex-1 max-w-3xl mx-auto w-full px-4 sm:px-6 lg:px-8 py-8">
      {/* Not logged in warning */}
      {!identity && !actorLoading && (
        <Alert
          variant="destructive"
          className="mb-6"
          data-ocid="register.error_state"
        >
          <AlertCircle className="h-4 w-4" />
          <AlertDescription className="flex items-center justify-between gap-4">
            <span>You must be logged in to register a case.</span>
            <Button
              size="sm"
              variant="outline"
              onClick={() => navigate({ to: "/login" })}
              className="shrink-0"
              data-ocid="register.link"
            >
              Go to Login
```

```
      </Button>
    </AlertDescription>
  </Alert>
)}

{/* Actor loading indicator */}
{actorLoading && identity && (
  <Alert
    className="mb-6 border-blue-200 bg-blue-50 dark:border-blue-800 dark:bg-blue-950"
    data-ocid="register.loading_state"
  >
    <Loader2 className="h-4 w-4 animate-spin text-blue-500" />
    <AlertDescription className="text-blue-700 dark:text-blue-300">
      Connecting to backend   please wait a moment...
    </AlertDescription>
  </Alert>
)}

{/* Actor error   failed after all retries */}
{actorError && !actorLoading && identity && (
  <Alert
    variant="destructive"
    className="mb-6"
    data-ocid="register.error_state"
  >
    <AlertCircle className="h-4 w-4" />
    <AlertDescription className="flex items-center justify-between gap-4">
      <span>
        Backend connection failed. The canister may be temporarily
        unavailable.
      </span>
      <Button
        size="sm"
        variant="outline"
        onClick={() => refetchActor()}
        className="shrink-0"
        data-ocid="register.secondary_button"
      >
        <RefreshCw className="w-3 h-3 mr-1" />
        Retry
      </Button>
    </AlertDescription>
  </Alert>
)}

<AnimatePresence>
  {successInfo && (
    <motion.div
      initial={{ opacity: 0, y: -10 }}
      animate={{ opacity: 1, y: 0 }}
      exit={{ opacity: 0, y: -10 }}
      className="mb-6"
      data-ocid="register.success_state"
    >
      <Alert className="border-success/30 bg-success/5">
```

```
        <CheckCircle className="h-4 w-4 text-success" />
        <AlertDescription className="text-success">
          <strong>{successInfo.name}</strong> has been registered
          successfully. Contact: <strong>{successInfo.contact}</strong>.
          Authorities have been notified.
        </AlertDescription>
      </Alert>
      <div className="flex gap-3 mt-3">
        <Button
          variant="outline"
          onClick={() => setSuccessInfo(null)}
          data-ocid="register.secondary_button"
        >
          Register Another
        </Button>
        <Button
          onClick={() => navigate({ to: "/dashboard" })}
          className="bg-primary text-primary-foreground"
          data-ocid="register.primary_button"
        >
          Go to Dashboard
        </Button>
      </div>
    </motion.div>
  )}
</AnimatePresence>

{submitError && (
  <Alert
    variant="destructive"
    className="mb-6"
    data-ocid="register.error_state"
  >
    <AlertCircle className="h-4 w-4" />
    <AlertDescription className="flex items-center justify-between gap-4">
      <span>{submitError}</span>
      <Button
        size="sm"
        variant="outline"
        onClick={() => setSubmitError("")}
        className="shrink-0"
        data-ocid="register.close_button"
      >
        Dismiss
      </Button>
    </AlertDescription>
  </Alert>
)}

<form onSubmit={handleSubmit}>
  <div className="bg-card rounded-xl border border-border shadow-card p-6 space-y-6">
    <h2 className="text-base font-semibold text-foreground border-b border-border pb-4">
      Child Information
    </h2>
```

```
<div className="grid grid-cols-1 sm:grid-cols-2 gap-5">
  {/* Name */}
  <div className="space-y-1.5">
    <Label htmlFor="child-name">
      Full Name <span className="text-destructive">*</span>
    </Label>
    <Input
      id="child-name"
      data-ocid="register.input"
      placeholder="Enter child's full name"
      value={form.name}
      onChange={(e) => handleField("name", e.target.value)}
      className={errors.name ? "border-destructive" : ""}
    />
    {errors.name && (
      <p
        className="text-xs text-destructive"
        data-ocid="register.error_state"
      >
        {errors.name}
      </p>
    )}
  </div>

  {/* Age */}
  <div className="space-y-1.5">
    <Label htmlFor="child-age">
      Age <span className="text-destructive">*</span>
    </Label>
    <Input
      id="child-age"
      data-ocid="register.input"
      type="number"
      min="0"
      max="18"
      placeholder="Age in years"
      value={form.age}
      onChange={(e) => handleField("age", e.target.value)}
      className={errors.age ? "border-destructive" : ""}
    />
    {errors.age && (
      <p
        className="text-xs text-destructive"
        data-ocid="register.error_state"
      >
        {errors.age}
      </p>
    )}
  </div>

  {/* Last Location */}
  <div className="space-y-1.5">
    <Label htmlFor="last-location">
      Last Known Location{" "}
      <span className="text-destructive">*</span>
```

```
  </Label>
  <Input
    id="last-location"
    data-ocid="register.input"
    placeholder="e.g. Central Park, New York"
    value={form.lastLocation}
    onChange={(e) => handleField("lastLocation", e.target.value)}
    className={errors.lastLocation ? "border-destructive" : ""}
  />
  {errors.lastLocation && (
    <p
      className="text-xs text-destructive"
      data-ocid="register.error_state"
    >
      {errors.lastLocation}
    </p>
  )}
</div>

{/* Contact Number */}
<div className="space-y-1.5">
  <Label htmlFor="contact-number">
    Contact Number <span className="text-destructive">*</span>
  </Label>
  <Input
    id="contact-number"
    data-ocid="register.input"
    type="tel"
    placeholder="+1 (555) 000-0000"
    value={form.contactNumber}
    onChange={(e) => handleField("contactNumber", e.target.value)}
    className={errors.contactNumber ? "border-destructive" : ""}
  />
  {errors.contactNumber && (
    <p
      className="text-xs text-destructive"
      data-ocid="register.error_state"
    >
      {errors.contactNumber}
    </p>
  )}
</div>

{/* Last Seen Place */}
<div className="space-y-1.5">
  <Label htmlFor="last-seen">
    Last Seen Place <span className="text-destructive">*</span>
  </Label>
  <Input
    id="last-seen"
    data-ocid="register.input"
    placeholder="e.g. School playground, Mall entrance"
    value={form.lastSeenPlace}
    onChange={(e) => handleField("lastSeenPlace", e.target.value)}
    className={errors.lastSeenPlace ? "border-destructive" : ""}
```

```
      />
      {errors.lastSeenPlace && (
        <p
          className="text-xs text-destructive"
          data-ocid="register.error_state"
        >
          {errors.lastSeenPlace}
        </p>
      )}
    </div>

    {/* Status */}
    <div className="space-y-1.5">
      <Label>Status</Label>
      <Select
        value={form.status}
        onValueChange={(v) => handleField("status", v)}
      >
        <SelectTrigger data-ocid="register.select">
          <SelectValue />
        </SelectTrigger>
        <SelectContent>
          <SelectItem value="active">Active</SelectItem>
          <SelectItem value="underReview">Under Review</SelectItem>
          <SelectItem value="found">Found</SelectItem>
          <SelectItem value="closed">Closed</SelectItem>
        </SelectContent>
      </Select>
    </div>
  </div>

  {/* Photo Upload */}
  <div className="space-y-2">
    <Label>Photo Upload</Label>
    <p className="text-xs text-muted-foreground">
      Accepts all image formats (JPG, PNG, WebP, HEIC, TIFF, GIF, BMP,
      etc.)
    </p>

    {isCompressing && (
      <div className="flex items-center gap-2 text-sm text-muted-foreground">
        <Loader2 className="w-4 h-4 animate-spin" />
        Processing photo...
      </div>
    )}

    {photoDataUrl && !isCompressing ? (
      <div className="relative inline-block">
        <img
          src={photoDataUrl}
          alt={form.name || "Child"}
          className="w-32 h-32 rounded-xl object-cover border-2 border-secondary shadow"
        />
        <button
          type="button"
```

```
          onClick={() => {
            setPhotoDataUrl(null);
            setPhotoFileName(null);
          }}
          className="absolute -top-2 -right-2 w-6 h-6 bg-destructive text-destructive-fore
          data-ocid="register.close_button"
        >
          <X className="w-3 h-3" />
        </button>
        <p className="text-xs text-muted-foreground mt-1 truncate max-w-[8rem]">
          {photoFileName}
        </p>
      </div>
    ) : !isCompressing ? (
      <button
        type="button"
        data-ocid="register.dropzone"
        onDragOver={(e) => {
          e.preventDefault();
          setIsDragging(true);
        }}
        onDragLeave={() => setIsDragging(false)}
        onDrop={handleDrop}
        onClick={() => fileInputRef.current?.click()}
        className={`w-full border-2 border-dashed rounded-xl p-8 text-center cursor-pointe
          isDragging
            ? "border-secondary bg-secondary/5"
            : "border-border hover:border-secondary hover:bg-secondary/5"
        }`}
      >
        <Upload className="w-8 h-8 text-muted-foreground mx-auto mb-2" />
        <p className="text-sm font-medium text-foreground">
          Drop photo here or click to browse
        </p>
        <p className="text-xs text-muted-foreground mt-1">
          All image formats supported
        </p>
        <input
          ref={fileInputRef}
          type="file"
          accept="image/*"
          className="hidden"
          data-ocid="register.upload_button"
          onChange={(e) => {
            const file = e.target.files?.[0];
            if (file) handlePhoto(file);
          }}
        />
      </button>
    ) : null}
  </div>
</div>

<div className="mt-6 flex gap-3 justify-end">
  <Button
```

```
            type="button"
            variant="outline"
            onClick={() => navigate({ to: "/dashboard" })}
            data-ocid="register.cancel_button"
          >
            Cancel
          </Button>
          <Button
            type="submit"
            disabled={isSubmitDisabled}
            className="bg-secondary text-secondary-foreground hover:bg-secondary/90 min-w-[140px]"
            data-ocid="register.submit_button"
          >
            {isPending ? (
              <span className="flex items-center gap-2">
                <Loader2 className="w-4 h-4 animate-spin" />
                Registering...
              </span>
            ) : actorLoading ? (
              <span className="flex items-center gap-2">
                <Loader2 className="w-4 h-4 animate-spin" />
                Connecting...
              </span>
            ) : isCompressing ? (
              <span className="flex items-center gap-2">
                <Loader2 className="w-4 h-4 animate-spin" />
                Processing photo...
              </span>
            ) : (
              "Register Case"
            )}
          </Button>
        </div>
      </form>
    </main>

    <footer className="bg-footer text-footer-foreground mt-auto">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-6">
        <p className="text-sm text-center">
          &copy; {new Date().getFullYear()} Missing Child Finder. Built with
          love using{" "}
          <a
            href={`https://caffeine.ai?utm_source=caffeine-footer&utm_medium=referral&utm_content=
            target="_blank"
            rel="noopener noreferrer"
            className="underline hover:opacity-80"
          >
            caffeine.ai
          </a>
        </p>
      </div>
    </footer>
  </div>
);
}
```

```
// ============================================================
// FILE: src/frontend/src/pages/SearchPage.tsx
// ============================================================
import { Alert, AlertDescription } from "@/components/ui/alert";
import { Button } from "@/components/ui/button";
import { Progress } from "@/components/ui/progress";
import { Tabs, TabsContent, TabsList, TabsTrigger } from "@/components/ui/tabs";
import {
  AlertTriangle,
  Camera,
  CheckCircle,
  Eye,
  EyeOff,
  Loader2,
  Search,
  ShieldAlert,
  SwitchCamera,
  Upload,
  Video,
  X,
} from "lucide-react";
import { AnimatePresence, motion } from "motion/react";
import { useCallback, useEffect, useRef, useState } from "react";
import type { ChildRecord } from "../backend";
import { Status } from "../backend";
import { useCamera } from "../camera/useCamera";
import Navbar from "../components/Navbar";
import {
  useAddAlert,
  useGetAllCases,
  useUpdateStatusToFound,
} from "../hooks/useQueries";
import {
  areModelsLoaded,
  clearSearchFaceCache,
  computeFrameVariance,
  computeMatchScore,
  detectFaceInImage,
  ensureModelsLoaded,
  extractHistogram,
  loadImageToCanvas,
} from "../utils/imageAnalysis";

interface MatchResult {
  record: ChildRecord;
  score: number;
}

type SearchPhase =
  | "idle"
  | "loading-models"
  | "preprocessing"
  | "analyzing"
```

```
  | "age-progression"
  | "fetching"
  | "extracting-frames"
  | "analyzing-frames"
  | "done";

const PHASE_MESSAGES: Partial<Record<SearchPhase, string>> = {
  "loading-models": "Loading face recognition models...",
  preprocessing: "Preprocessing image...",
  analyzing: "Extracting face descriptors...",
  "age-progression": "Applying bi-directional age progression model...",
  fetching: "Matching against registered cases...",
  "extracting-frames": "Extracting frames from video...",
  "analyzing-frames": "Analyzing frames with face recognition...",
};

const extractFramesFromVideo = async (
  file: File,
  numFrames = 20,
): Promise<string[]> => {
  return new Promise((resolve) => {
    const video = document.createElement("video");
    const url = URL.createObjectURL(file);
    video.src = url;
    video.muted = true;
    const frames: string[] = [];
    video.addEventListener("loadedmetadata", async () => {
      const duration = video.duration;
      const canvas = document.createElement("canvas");
      canvas.width = 640;
      canvas.height = 480;
      const ctx = canvas.getContext("2d")!;

      // Skip first/last 5% of duration
      const startTime = duration * 0.05;
      const endTime = duration * 0.95;
      const usableDuration = endTime - startTime;

      let prevHist: number[] | null = null;
      const collected: string[] = [];

      for (let i = 0; i < numFrames * 2 && collected.length < numFrames; i++) {
        const time = startTime + (usableDuration / (numFrames * 2 - 1)) * i;
        if (time > endTime) break;
        video.currentTime = time;
        await new Promise((r) =>
          video.addEventListener("seeked", r, { once: true }),
        );
        ctx.drawImage(video, 0, 0, 640, 480);
        const frameDataUrl = canvas.toDataURL("image/jpeg", 0.9);

        // Skip near-identical frames (histogram diff < 0.05)
        const imgData = ctx.getImageData(0, 0, 640, 480);
        const hist = extractHistogram(imgData);
        if (prevHist !== null) {
```

```
          const diff = hist.reduce(
            (sum, v, idx) => sum + Math.abs(v - prevHist![idx]),
            0,
          );
          if (diff < 0.05) continue;
        }
        prevHist = hist;
        collected.push(frameDataUrl);
      }

      // Ensure we always get at least some frames even if all were skipped
      if (collected.length === 0) {
        for (let i = 0; i < Math.min(numFrames, 5); i++) {
          const time = startTime + (usableDuration / 4) * i;
          video.currentTime = time;
          await new Promise((r) =>
            video.addEventListener("seeked", r, { once: true }),
          );
          ctx.drawImage(video, 0, 0, 640, 480);
          collected.push(canvas.toDataURL("image/jpeg", 0.9));
        }
      }

      frames.push(...collected);
      URL.revokeObjectURL(url);
      resolve(frames);
    });
    video.load();
  });
};

export default function SearchPage() {
  const { data: allCases } = useGetAllCases();
  const { mutateAsync: markFound } = useUpdateStatusToFound();
  const { mutateAsync: addAlert } = useAddAlert();

  const [photo, setPhoto] = useState<File | null>(null);
  const [photoPreview, setPhotoPreview] = useState<string | null>(null);
  const [isDragging, setIsDragging] = useState(false);
  const [phase, setPhase] = useState<SearchPhase>("idle");
  const [progress, setProgress] = useState(0);
  const [results, setResults] = useState<MatchResult[]>([]);
  const [markingFound, setMarkingFound] = useState<string | null>(null);
  const [ageProgressedUrl, setAgeProgressedUrl] = useState<string | null>(null);
  const [alertMessage, setAlertMessage] = useState<string | null>(null);
  const [modelsLoaded, setModelsLoaded] = useState(false);
  const fileInputRef = useRef<HTMLInputElement>(null);

  // Video upload state
  const [videoFile, setVideoFile] = useState<File | null>(null);
  const [videoFrameCount, setVideoFrameCount] = useState(0);
  const [isVideoDragging, setIsVideoDragging] = useState(false);
  const videoInputRef = useRef<HTMLInputElement>(null);
  const [videoAnalysisMessage, setVideoAnalysisMessage] = useState<
    string | null
```

```
>(null);

// Load face-api models in background on mount
useEffect(() => {
  ensureModelsLoaded()
    .then(() => {
      setModelsLoaded(areModelsLoaded());
    })
    .catch(() => {
      // Silent fail -- will fall back to histogram matching
    });
}, []);

// Face detection state
const [faceVariance, setFaceVariance] = useState(0);
const [faceDetectedByApi, setFaceDetectedByApi] = useState(false);
const faceRafRef = useRef<number | null>(null);
const offscreenCanvasRef = useRef<HTMLCanvasElement | null>(null);

const camera = useCamera({ facingMode: "user" });

// Variance-based face detection loop (always runs as fallback)
const runFaceDetectionLoop = useCallback(() => {
  if (!camera.videoRef.current || !camera.isActive) return;

  const video = camera.videoRef.current;

  if (!offscreenCanvasRef.current) {
    offscreenCanvasRef.current = document.createElement("canvas");
    offscreenCanvasRef.current.width = 64;
    offscreenCanvasRef.current.height = 64;
  }

  const canvas = offscreenCanvasRef.current;
  const ctx = canvas.getContext("2d");
  if (!ctx || video.readyState < 2) {
    faceRafRef.current = requestAnimationFrame(runFaceDetectionLoop);
    return;
  }

  const vw = video.videoWidth;
  const vh = video.videoHeight;
  const cx = vw * 0.25;
  const cy = vh * 0.15;
  const cw = vw * 0.5;
  const ch = vh * 0.7;

  ctx.drawImage(video, cx, cy, cw, ch, 0, 0, 64, 64);
  const imageData = ctx.getImageData(0, 0, 64, 64);
  const variance = computeFrameVariance(imageData);
  setFaceVariance(variance);

  faceRafRef.current = requestAnimationFrame(runFaceDetectionLoop);
}, [camera.isActive, camera.videoRef]);
```

```
useEffect(() => {
  if (camera.isActive) {
    faceRafRef.current = requestAnimationFrame(runFaceDetectionLoop);
  } else {
    if (faceRafRef.current !== null) {
      cancelAnimationFrame(faceRafRef.current);
      faceRafRef.current = null;
    }
    setFaceVariance(0);
    setFaceDetectedByApi(false);
  }
  return () => {
    if (faceRafRef.current !== null) {
      cancelAnimationFrame(faceRafRef.current);
      faceRafRef.current = null;
    }
  };
}, [camera.isActive, runFaceDetectionLoop]);

// face-api.js based face detection every 800ms when camera is active
useEffect(() => {
  if (!camera.isActive || !modelsLoaded) return;

  const interval = setInterval(async () => {
    if (!camera.videoRef.current || camera.videoRef.current.readyState < 2)
      return;
    const video = camera.videoRef.current;

    // Capture a frame to canvas
    const captureCanvas = document.createElement("canvas");
    captureCanvas.width = video.videoWidth || 320;
    captureCanvas.height = video.videoHeight || 240;
    const ctx = captureCanvas.getContext("2d");
    if (!ctx) return;
    ctx.drawImage(video, 0, 0, captureCanvas.width, captureCanvas.height);

    const img = new Image();
    img.src = captureCanvas.toDataURL("image/jpeg", 0.7);
    await new Promise<void>((r) => {
      img.onload = () => r();
    });

    const detected = await detectFaceInImage(img);
    setFaceDetectedByApi(detected);
  }, 800);

  return () => clearInterval(interval);
}, [camera.isActive, modelsLoaded, camera.videoRef]);

// Combine variance + API detection for final status
const faceStatus: "detected" | "partial" | "none" = modelsLoaded
  ? faceDetectedByApi
    ? "detected"
    : faceVariance > 300
      ? "partial"
```

```
      : "none"
  : faceVariance > 800
    ? "detected"
    : faceVariance > 300
      ? "partial"
      : "none";

const handlePhoto = (file: File) => {
  setPhoto(file);
  setPhotoPreview(URL.createObjectURL(file));
  setResults([]);
  setPhase("idle");
  setProgress(0);
  setAgeProgressedUrl(null);
  setAlertMessage(null);
};

const handleDrop = (e: React.DragEvent) => {
  e.preventDefault();
  setIsDragging(false);
  const file = e.dataTransfer.files[0];
  if (file?.type.startsWith("image/")) handlePhoto(file);
};

const runSearch = async (overrideFile?: File) => {
  const activePhoto = overrideFile ?? photo;
  if (!activePhoto) return;

  setResults([]);
  setAgeProgressedUrl(null);
  setAlertMessage(null);
  setPhase("loading-models");
  setProgress(5);

  await ensureModelsLoaded().catch(() => {});
  setModelsLoaded(areModelsLoaded());
  setProgress(15);

  const dataUrl = await new Promise<string>((resolve, reject) => {
    const reader = new FileReader();
    reader.onload = () => resolve(reader.result as string);
    reader.onerror = reject;
    reader.readAsDataURL(activePhoto);
  });

  setPhase("preprocessing");
  setProgress(25);
  await loadImageToCanvas(dataUrl);

  setPhase("analyzing");
  setProgress(35);

  setPhase("age-progression");
  await new Promise<void>((resolve) => setTimeout(resolve, 400));
  setProgress(55);
```

```
setPhase("fetching");

const cases = allCases ?? [];
const casesWithPhotos = cases.filter(
  (c) => c.photoId && c.photoId.trim() !== "",
);
const casesWithoutPhotos = cases.filter(
  (c) => !c.photoId || c.photoId.trim() === "",
);

let agedUrl: string | null = null;

const scoredWithPhotos: MatchResult[] = await Promise.all(
  casesWithPhotos.map(async (record) => {
    const score = await computeMatchScore(
      dataUrl,
      record.photoId,
      (url) => {
        if (!agedUrl) {
          agedUrl = url;
          setAgeProgressedUrl(url);
        }
      },
    );
    return { record, score };
  }),
);

const scoredWithoutPhotos: MatchResult[] = casesWithoutPhotos.map(
  (record) => ({ record, score: 0 }),
);

const allScored = [...scoredWithPhotos, ...scoredWithoutPhotos];
allScored.sort((a, b) => b.score - a.score);

// Always show the single best-scored result (even if score is low/zero)
const withPhotos = allScored.filter(
  (r) => r.record.photoId && r.record.photoId.trim() !== "",
);
const topResult = withPhotos.length > 0 ? [withPhotos[0]] : [];

setProgress(100);
setPhase("done");
setResults(topResult);

if (topResult.length > 0) {
  const match = topResult[0];
  const msg = `Match found for ${match.record.name} (Age: ${Number(match.record.age)}, Last seen
  setAlertMessage(`Alert sent to ${match.record.contactNumber}: ${msg}`);
  try {
    await addAlert({
      contactNumber: match.record.contactNumber,
      message: msg,
    });
```

```
    } catch {
      // best-effort
    }
  }
};

const runVideoSearch = async () => {
  if (!videoFile) return;

  setResults([]);
  setAgeProgressedUrl(null);
  setAlertMessage(null);
  setVideoAnalysisMessage(null);
  setPhase("loading-models");
  setProgress(5);

  await ensureModelsLoaded().catch(() => {});
  setModelsLoaded(areModelsLoaded());
  setProgress(10);

  setPhase("extracting-frames");
  setVideoAnalysisMessage("Extracting frames from video...");
  const frames = await extractFramesFromVideo(videoFile, 15);
  setVideoFrameCount(frames.length);
  setProgress(25);

  setPhase("analyzing-frames");

  const cases = allCases ?? [];
  const casesWithPhotos = cases.filter(
    (c) => c.photoId && c.photoId.trim() !== "",
  );
  const casesWithoutPhotos = cases.filter(
    (c) => !c.photoId || c.photoId.trim() === "",
  );

  // For each case, find the best score across all frames
  const caseScores: Map<string, number> = new Map();

  for (let fi = 0; fi < frames.length; fi++) {
    const frameDataUrl = frames[fi];
    const faceMsg = `Analyzing frame ${fi + 1} of ${frames.length}...`;
    setVideoAnalysisMessage(faceMsg);
    setProgress(25 + Math.round(((fi + 1) / frames.length) * 65));

    await Promise.all(
      casesWithPhotos.map(async (record) => {
        const score = await computeMatchScore(frameDataUrl, record.photoId);
        const caseKey = String(record.name) + String(record.age);
        const existing = caseScores.get(caseKey) ?? 0;
        if (score > existing) {
          caseScores.set(caseKey, score);
        }
      }),
    );
```

```
  }

  const scoredWithPhotos: MatchResult[] = casesWithPhotos.map((record) => ({
    record,
    score: caseScores.get(String(record.name) + String(record.age)) ?? 0,
  }));

  const scoredWithoutPhotos: MatchResult[] = casesWithoutPhotos.map(
    (record) => ({ record, score: 0 }),
  );

  const allScored = [...scoredWithPhotos, ...scoredWithoutPhotos];
  allScored.sort((a, b) => b.score - a.score);

  // Always show the single best-scored result (even if score is low/zero)
  const withPhotos = allScored.filter(
    (r) => r.record.photoId && r.record.photoId.trim() !== "",
  );
  const topResult = withPhotos.length > 0 ? [withPhotos[0]] : [];

  setProgress(100);
  setPhase("done");
  setResults(topResult);
  setVideoAnalysisMessage(null);

  if (topResult.length > 0) {
    const match = topResult[0];
    const msg = `Match found for ${match.record.name} (Age: ${Number(match.record.age)}, Last seen
    setAlertMessage(`Alert sent to ${match.record.contactNumber}: ${msg}`);
    try {
      await addAlert({
        contactNumber: match.record.contactNumber,
        message: msg,
      });
    } catch {
      // best-effort
    }
  }
};

const handleCaptureAndSearch = async () => {
  if (!camera.videoRef.current || !camera.isActive) return;
  clearSearchFaceCache();
  const video = camera.videoRef.current;

  const capturedFrames: File[] = [];
  for (let i = 0; i < 5; i++) {
    const w = video.videoWidth || 640;
    const h = video.videoHeight || 480;
    const cap = document.createElement("canvas");
    cap.width = w;
    cap.height = h;
    const ctx = cap.getContext("2d");
    if (ctx) {
      ctx.drawImage(video, 0, 0, w, h);
```

```
      const blob = await new Promise<Blob | null>((r) =>
        cap.toBlob(r, "image/jpeg", 0.92),
      );
      if (blob)
        capturedFrames.push(
          new File([blob], `frame${i}.jpg`, { type: "image/jpeg" }),
        );
    }
    if (i < 4) await new Promise((r) => setTimeout(r, 100));
  }

  if (capturedFrames.length === 0) return;

  // Pick the best frame: first one with a detected face
  let bestFrame = capturedFrames[Math.floor(capturedFrames.length / 2)];
  for (const frame of capturedFrames) {
    const dataUrl = await new Promise<string>((resolve) => {
      const reader = new FileReader();
      reader.onload = () => resolve(reader.result as string);
      reader.readAsDataURL(frame);
    });
    const img = new Image();
    img.src = dataUrl;
    await new Promise<void>((r) => {
      img.onload = () => r();
    });
    const hasFace = await detectFaceInImage(img);
    if (hasFace) {
      bestFrame = frame;
      break;
    }
  }

  handlePhoto(bestFrame);
  await runSearch(bestFrame);
};
const handleTabChange = (value: string) => {
  if (value !== "camera" && camera.isActive) {
    camera.stopCamera();
  }
  if (value !== "video") {
    setVideoFile(null);
    setVideoFrameCount(0);
    setVideoAnalysisMessage(null);
  }
};

const handleMarkFound = async (contactNumber: string) => {
  setMarkingFound(contactNumber);
  try {
    await markFound(contactNumber);
  } finally {
    setMarkingFound(null);
  }
};
```

```
const isSearching =
  phase === "loading-models" ||
  phase === "preprocessing" ||
  phase === "analyzing" ||
  phase === "age-progression" ||
  phase === "fetching" ||
  phase === "extracting-frames" ||
  phase === "analyzing-frames";
const topMatch = results[0];
const hasRegisteredCasesWithPhotos = (allCases ?? []).some(
  (c) => c.photoId && c.photoId.trim() !== "",
);

return (
  <div className="min-h-screen bg-background flex flex-col">
    <Navbar />

    <div className="bg-card border-b border-border">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-6">
        <h1 className="text-2xl font-bold text-foreground">
          Search by Photo
        </h1>
        <p className="text-muted-foreground text-sm mt-1">
          Upload a photo, video, or use live camera to match against
          registered missing children using CNN face recognition with age
          progression
        </p>
      </div>
    </div>

    <main className="flex-1 max-w-4xl mx-auto w-full px-4 sm:px-6 lg:px-8 py-8 space-y-6">
      <Tabs defaultValue="photo" onValueChange={handleTabChange}>
        <TabsList className="mb-4">
          <TabsTrigger value="photo" data-ocid="search.tab">
            <Upload className="w-4 h-4 mr-2" />
            Photo Upload
          </TabsTrigger>
          <TabsTrigger value="video" data-ocid="search.tab">
            <Video className="w-4 h-4 mr-2" />
            Upload Video
          </TabsTrigger>
          <TabsTrigger value="camera" data-ocid="search.tab">
            <Camera className="w-4 h-4 mr-2" />
            Live Camera
          </TabsTrigger>
        </TabsList>

        <TabsContent value="photo">
          <div className="bg-card rounded-xl border border-border shadow-card p-6">
            <h2 className="text-base font-semibold text-foreground mb-4">
              Upload Search Photo
            </h2>

            {photoPreview ? (
```

```
    <div className="flex items-start gap-6">
      <div className="relative shrink-0">
        <img
          src={photoPreview}
          alt="Selected for search"
          className="w-32 h-32 rounded-xl object-cover border-2 border-secondary shadow"
        />
        <button
          type="button"
          onClick={() => {
            setPhoto(null);
            setPhotoPreview(null);
            setResults([]);
            setPhase("idle");
            setAgeProgressedUrl(null);
            setAlertMessage(null);
          }}
          className="absolute -top-2 -right-2 w-6 h-6 bg-destructive text-destructive-fo
          data-ocid="search.close_button"
        >
          <X className="w-3 h-3" />
        </button>
      </div>
      <div className="flex-1">
        <p className="font-medium text-foreground">{photo?.name}</p>
        <p className="text-xs text-muted-foreground mt-1">
          {photo ? `${(photo.size / 1024).toFixed(1)} KB` : ""}
        </p>
        <Button
          onClick={() => runSearch()}
          disabled={isSearching}
          className="mt-4 bg-primary text-primary-foreground hover:bg-primary/90 gap-2"
          data-ocid="search.primary_button"
        >
          {isSearching ? (
            <>
              <Loader2 className="w-4 h-4 animate-spin" />{" "}
              Analyzing...
            </>
          ) : (
            <>
              <Search className="w-4 h-4" /> Search for Match
            </>
          )}
        </Button>
      </div>
    </div>
  ) : (
    <button
      type="button"
      data-ocid="search.dropzone"
      onDragOver={(e) => {
        e.preventDefault();
        setIsDragging(true);
      }}
```

```
        onDragLeave={() => setIsDragging(false)}
        onDrop={handleDrop}
        onClick={() => fileInputRef.current?.click()}
        className={`w-full border-2 border-dashed rounded-xl p-12 text-center cursor-point
          isDragging
            ? "border-secondary bg-secondary/5"
            : "border-border hover:border-secondary hover:bg-secondary/5"
        }`}
      >
        <Upload className="w-10 h-10 text-muted-foreground mx-auto mb-3" />
        <p className="font-medium text-foreground">
          Drop a photo here or click to upload
        </p>
        <p className="text-xs text-muted-foreground mt-1">
          All image formats supported (JPG, PNG, WebP, HEIC, etc.)
        </p>
        <input
          ref={fileInputRef}
          type="file"
          accept="image/*"
          className="hidden"
          data-ocid="search.upload_button"
          onChange={(e) => {
            const file = e.target.files?.[0];
            if (file) {
              clearSearchFaceCache();
              handlePhoto(file);
            }
          }}
        />
      </button>
    )}
  </div>
</TabsContent>

<TabsContent value="video">
  <div className="bg-card rounded-xl border border-border shadow-card p-6">
    <h2 className="text-base font-semibold text-foreground mb-1">
      Upload Video for Face Detection
    </h2>
    <p className="text-xs text-muted-foreground mb-4">
      Upload a video file to extract frames and cross-check faces
      against registered cases
    </p>

    {videoFile ? (
      <div className="flex items-start gap-6">
        <div className="relative shrink-0 w-32 h-32 rounded-xl bg-muted border-2 border-se
          <Video className="w-8 h-8 text-secondary" />
          <span className="text-xs text-muted-foreground text-center px-2 leading-tight li
            {videoFile.name}
          </span>
          <button
            type="button"
            onClick={() => {
```

```
            setVideoFile(null);
            setVideoFrameCount(0);
            setResults([]);
            setPhase("idle");
            setAlertMessage(null);
            setVideoAnalysisMessage(null);
          }}
          className="absolute -top-2 -right-2 w-6 h-6 bg-destructive text-destructive-fo
          data-ocid="search.close_button"
        >
          <X className="w-3 h-3" />
        </button>
      </div>
      <div className="flex-1">
        <p className="font-medium text-foreground truncate">
          {videoFile.name}
        </p>
        <p className="text-xs text-muted-foreground mt-1">
          {(videoFile.size / (1024 * 1024)).toFixed(1)} MB
        </p>
        {videoFrameCount > 0 && (
          <p className="text-xs text-muted-foreground mt-0.5">
            {videoFrameCount} frames extracted
          </p>
        )}
        {videoAnalysisMessage && (
          <p className="text-xs text-secondary mt-0.5 font-medium">
            {videoAnalysisMessage}
          </p>
        )}
        <Button
          onClick={runVideoSearch}
          disabled={isSearching}
          className="mt-4 bg-primary text-primary-foreground hover:bg-primary/90 gap-2"
          data-ocid="search.primary_button"
        >
          {isSearching ? (
            <>
              <Loader2 className="w-4 h-4 animate-spin" />
              Analyzing...
            </>
          ) : (
            <>
              <Search className="w-4 h-4" /> Analyze Video
            </>
          )}
        </Button>
      </div>
    </div>
  ) : (
    <button
      type="button"
      data-ocid="search.dropzone"
      onDragOver={(e) => {
        e.preventDefault();
```

```
          setIsVideoDragging(true);
        }}
        onDragLeave={() => setIsVideoDragging(false)}
        onDrop={(e) => {
          e.preventDefault();
          setIsVideoDragging(false);
          const file = e.dataTransfer.files[0];
          if (file?.type.startsWith("video/")) {
            setVideoFile(file);
            setResults([]);
            setPhase("idle");
            setAlertMessage(null);
          }
        }}
        onClick={() => videoInputRef.current?.click()}
        className={`w-full border-2 border-dashed rounded-xl p-12 text-center cursor-point
          isVideoDragging
            ? "border-secondary bg-secondary/5"
            : "border-border hover:border-secondary hover:bg-secondary/5"
        }`}
      >
        <Video className="w-10 h-10 text-muted-foreground mx-auto mb-3" />
        <p className="font-medium text-foreground">
          Drop a video here or click to upload
        </p>
        <p className="text-xs text-muted-foreground mt-1">
          Supported formats: MP4, MOV, WebM
        </p>
        <p className="text-xs text-muted-foreground mt-0.5">
          Up to 15 frames extracted and analyzed for face matches
        </p>
        <input
          ref={videoInputRef}
          type="file"
          accept="video/mp4,video/quicktime,video/webm,video/*"
          className="hidden"
          data-ocid="search.upload_button"
          onChange={(e) => {
            const file = e.target.files?.[0];
            if (file) {
              clearSearchFaceCache();
              setVideoFile(file);
              setResults([]);
              setPhase("idle");
              setAlertMessage(null);
            }
          }}
        />
      </button>
    )}
  </div>
</TabsContent>

<TabsContent value="camera">
  <div className="bg-card rounded-xl border border-border shadow-card p-6">
```

```
<h2 className="text-base font-semibold text-foreground mb-4">
  Live Face Detection
</h2>

<div className="relative aspect-video w-full rounded-xl overflow-hidden bg-black mb-4"
  <video
    ref={camera.videoRef}
    autoPlay
    playsInline
    muted
    className="w-full h-full object-cover"
    style={{ transform: "scaleX(-1)" }}
  />
  <canvas ref={camera.canvasRef} className="hidden" />

  {camera.isActive && (
    <div className="absolute inset-0 flex items-center justify-center pointer-events-n
      <div
        className={`w-48 h-56 border-2 rounded-2xl flex items-end justify-center pb-3
          faceStatus === "detected"
            ? "border-green-400 animate-pulse shadow-[0_0_16px_2px_rgba(74,222,128,0.4
            : faceStatus === "partial"
              ? "border-yellow-400/70 border-dashed"
              : "border-white/40 border-dashed"
        }`}
      >
        <span
          className={`text-xs font-medium px-2 py-1 rounded-full ${
            faceStatus === "detected"
              ? "bg-green-500/80 text-white"
              : faceStatus === "partial"
                ? "bg-yellow-500/70 text-white"
                : "text-white/80 bg-black/40"
          }`}
        >
          {faceStatus === "detected"
            ? "Face Detected"
            : faceStatus === "partial"
              ? "Searching for face..."
              : "Position face in frame"}
        </span>
      </div>
    </div>
  )}

  {!camera.isActive && !camera.isLoading && (
    <div className="absolute inset-0 flex flex-col items-center justify-center gap-2">
      <Camera className="w-12 h-12 text-white/30" />
      <p className="text-white/50 text-sm">Camera not started</p>
    </div>
  )}

  {camera.isLoading && (
    <div className="absolute inset-0 flex items-center justify-center">
      <Loader2 className="w-8 h-8 text-white/60 animate-spin" />
```

```
    </div>
  )}
</div>

{camera.isActive && (
  <div className="mb-4 p-3 rounded-lg bg-muted/40 border border-border">
    <div className="flex items-center justify-between mb-2">
      <div className="flex items-center gap-2">
        {faceStatus === "detected" ? (
          <Eye className="w-4 h-4 text-green-500" />
        ) : (
          <EyeOff className="w-4 h-4 text-muted-foreground" />
        )}
        <span className="text-sm font-medium text-foreground">
          Face Detection
          {modelsLoaded && (
            <span className="ml-1 text-xs text-muted-foreground font-normal">
              (CNN active)
            </span>
          )}
        </span>
      </div>
      <span
        className={`text-xs font-semibold px-2 py-0.5 rounded-full ${
          faceStatus === "detected"
            ? "bg-green-500/15 text-green-600"
            : faceStatus === "partial"
              ? "bg-yellow-500/15 text-yellow-600"
              : "bg-muted text-muted-foreground"
        }`}
      >
        {faceStatus === "detected"
          ? "Face Detected"
          : faceStatus === "partial"
            ? "Searching..."
            : "No Face"}
      </span>
    </div>
    <div className="w-full h-2 rounded-full bg-muted overflow-hidden">
      <div
        className={`h-full rounded-full transition-all duration-200 ${
          faceStatus === "detected"
            ? "bg-green-500"
            : faceStatus === "partial"
              ? "bg-yellow-500"
              : "bg-muted-foreground/30"
        }`}
        style={{
          width: `${Math.min(100, (faceVariance / 1200) * 100)}%`,
        }}
      />
    </div>
  </div>
)}
```

```
{camera.error?.type === "permission" && (
  <div
    className="bg-card border border-destructive/30 rounded-xl p-6 space-y-4 mb-4"
    data-ocid="search.error_state"
  >
    <div className="flex items-center gap-3">
      <div className="w-10 h-10 rounded-full bg-destructive/10 flex items-center justi
        <ShieldAlert className="w-5 h-5 text-destructive" />
      </div>
      <div>
        <h3 className="font-semibold text-foreground">
          Camera Permission Required
        </h3>
        <p className="text-sm text-muted-foreground">
          Allow camera access to use face detection
        </p>
      </div>
    </div>
    <div className="space-y-3">
      <p className="text-sm font-medium text-foreground">
        How to enable camera access:
      </p>
      <ol className="space-y-2 text-sm text-muted-foreground">
        <li className="flex gap-2">
          <span className="font-medium text-foreground shrink-0">
            1.
          </span>
          Click the camera or lock icon in your browser&apos;s
          address bar
        </li>
        <li className="flex gap-2">
          <span className="font-medium text-foreground shrink-0">
            2.
          </span>
          Find &quot;Camera&quot; in the permissions list
        </li>
        <li className="flex gap-2">
          <span className="font-medium text-foreground shrink-0">
            3.
          </span>
          Change it from &quot;Blocked&quot; to &quot;Allow&quot;
        </li>
        <li className="flex gap-2">
          <span className="font-medium text-foreground shrink-0">
            4.
          </span>
          Click &quot;Try Again&quot; below to restart the camera
        </li>
      </ol>
    </div>
    <div className="bg-muted/50 rounded-lg p-3 text-xs text-muted-foreground space-y-1
      <p>
        <strong>Chrome/Edge:</strong> Click the lock icon ’ Site
        settings ’ Camera ’ Allow
      </p>
```

```
      <p>
        <strong>Firefox:</strong> Click the shield icon ’
        Permissions ’ Camera ’ Allow
      </p>
      <p>
        <strong>Safari:</strong> Safari menu ’ Settings for this
        website ’ Camera ’ Allow
      </p>
    </div>
    <Button
      onClick={() => camera.retry()}
      className="gap-2"
      data-ocid="search.primary_button"
    >
      <Camera className="w-4 h-4" />
      Try Again
    </Button>
  </div>
)}

{camera.error && camera.error.type !== "permission" && (
  <Alert
    className="mb-4 border-destructive/40 bg-destructive/5"
    data-ocid="search.error_state"
  >
    <AlertTriangle className="h-4 w-4 text-destructive" />
    <AlertDescription className="text-destructive">
      {camera.error.message}
    </AlertDescription>
  </Alert>
)}

<div className="flex gap-3">
  {!camera.isActive ? (
    <Button
      onClick={() => camera.startCamera()}
      disabled={camera.isLoading}
      className="gap-2"
      data-ocid="search.primary_button"
    >
      {camera.isLoading ? (
        <Loader2 className="w-4 h-4 animate-spin" />
      ) : (
        <Camera className="w-4 h-4" />
      )}
      Start Camera
    </Button>
  ) : (
    <>
      <Button
        onClick={handleCaptureAndSearch}
        disabled={isSearching}
        className="gap-2 bg-primary text-primary-foreground hover:bg-primary/90"
        data-ocid="search.primary_button"
      >
```

```
              {isSearching ? (
                <Loader2 className="w-4 h-4 animate-spin" />
              ) : (
                <Search className="w-4 h-4" />
              )}
              Capture & Search
            </Button>
            <Button
              variant="outline"
              onClick={() => camera.stopCamera()}
              className="gap-2"
              data-ocid="search.secondary_button"
            >
              <X className="w-4 h-4" />
              Stop Camera
            </Button>
            <Button
              variant="outline"
              onClick={() => camera.switchCamera()}
              disabled={camera.isLoading}
              className="gap-2"
              title={
                camera.currentFacingMode === "user"
                  ? "Switch to back camera"
                  : "Switch to front camera"
              }
            >
              {camera.isLoading ? (
                <Loader2 className="w-4 h-4 animate-spin" />
              ) : (
                <SwitchCamera className="w-4 h-4" />
              )}
              {camera.currentFacingMode === "user"
                ? "Back Camera"
                : "Front Camera"}
            </Button>
          </>
        )}
      </div>
    </div>
  </TabsContent>
</Tabs>

<AnimatePresence>
  {isSearching && (
    <motion.div
      initial={{ opacity: 0, y: 10 }}
      animate={{ opacity: 1, y: 0 }}
      exit={{ opacity: 0 }}
      className="bg-card rounded-xl border border-border shadow-card p-6"
      data-ocid="search.loading_state"
    >
      <div className="flex items-center gap-3 mb-4">
        <div className="w-8 h-8 rounded-full bg-secondary/20 flex items-center justify-cente
          <Loader2 className="w-4 h-4 text-secondary animate-spin" />
```

```
      </div>
      <p className="font-medium text-foreground">
        {videoAnalysisMessage ??
          PHASE_MESSAGES[phase] ??
          "Processing..."}
      </p>
    </div>
    <Progress value={progress} className="h-2" />
    <p className="text-xs text-muted-foreground mt-2">
      {progress}% complete
    </p>

    {phase === "age-progression" && photoPreview && (
      <motion.div
        initial={{ opacity: 0, y: 6 }}
        animate={{ opacity: 1, y: 0 }}
        className="mt-4 flex items-center gap-4"
      >
        <div className="flex flex-col items-center gap-1">
          <img
            src={photoPreview}
            alt="Original"
            className="w-20 h-20 rounded-lg object-cover border border-border"
          />
          <span className="text-xs text-muted-foreground">
            Original
          </span>
        </div>
        <div className="flex flex-col items-center gap-1 text-muted-foreground">
          <div className="flex items-center gap-1">
            <div className="w-6 h-px bg-border" />
            <span className="text-xs">Age ?</span>
            <div className="w-6 h-px bg-border" />
          </div>
        </div>
        <div className="flex flex-col items-center gap-1">
          {ageProgressedUrl ? (
            <img
              src={ageProgressedUrl}
              alt="Age Progressed"
              className="w-20 h-20 rounded-lg object-cover border border-secondary/60"
            />
          ) : (
            <div className="w-20 h-20 rounded-lg bg-muted flex items-center justify-center
              <Loader2 className="w-5 h-5 text-muted-foreground animate-spin" />
            </div>
          )}
          <span className="text-xs text-muted-foreground">
            Age Progressed
          </span>
        </div>
      </motion.div>
    )}
  </motion.div>
)}
```

```
</AnimatePresence>

<AnimatePresence>
  {alertMessage && (
    <motion.div
      initial={{ opacity: 0, y: -8 }}
      animate={{ opacity: 1, y: 0 }}
      exit={{ opacity: 0 }}
    >
      <Alert
        className="border-destructive/40 bg-destructive/5"
        data-ocid="search.toast"
      >
        <AlertTriangle className="h-4 w-4 text-destructive" />
        <AlertDescription className="text-destructive font-medium">
          {alertMessage}
        </AlertDescription>
      </Alert>
    </motion.div>
  )}
</AnimatePresence>

<AnimatePresence>
  {phase === "done" && results.length > 0 && !modelsLoaded && (
    <motion.div
      initial={{ opacity: 0, y: 12 }}
      animate={{ opacity: 1, y: 0 }}
      className="rounded-lg border border-amber-500/40 bg-amber-500/10 px-4 py-3 text-sm tex
      data-ocid="search.error_state"
    >
        Face detection models could not load   results are based on
      basic color matching only and may be less accurate. Check your
      internet connection and try again.
    </motion.div>
  )}

  {phase === "done" && results.length > 0 && (
    <motion.div
      initial={{ opacity: 0, y: 12 }}
      animate={{ opacity: 1, y: 0 }}
      className="space-y-4"
    >
      {topMatch && topMatch.score >= 65 && (
        <Alert
          className="border-destructive/40 bg-destructive/5"
          data-ocid="search.toast"
        >
          <AlertTriangle className="h-4 w-4 text-destructive" />
          <AlertDescription>
            <strong className="text-destructive">MATCH FOUND!</strong>{" "}
            Contact authorities immediately and alert the registered
            number:{" "}
            <strong className="text-destructive">
              {topMatch.record.contactNumber}
            </strong>
```

```
      . Best match: <strong>{topMatch.record.name}</strong>
    </AlertDescription>
  </Alert>
)}

<h2 className="text-base font-semibold text-foreground">
  Best Match Found
</h2>

{topMatch && topMatch.score < 40 && (
  <Alert className="border-blue-500/30 bg-blue-500/5">
    <AlertDescription className="text-blue-700 dark:text-blue-400">
      <strong>Tip:</strong> For best matching accuracy, use a
      clear frontal photo with good lighting.
    </AlertDescription>
  </Alert>
)}

{results.map((result) => {
  return (
    <motion.div
      key={result.record.contactNumber}
      initial={{ opacity: 0, scale: 0.97 }}
      animate={{ opacity: 1, scale: 1 }}
      className={
        result.score >= 65
          ? "rounded-xl border border-success/40 bg-success/5 shadow-card p-6"
          : result.score >= 40
            ? "rounded-xl border border-amber-500/40 bg-amber-500/5 shadow-card p-6"
            : "rounded-xl border border-muted bg-muted/30 shadow-card p-6"
      }
      data-ocid="search.item.1"
    >
      <div className="flex items-start gap-5">
        {/* Large prominent photo */}
        <div
          className={
            result.score >= 65
              ? "relative shrink-0 w-40 h-40 rounded-xl overflow-hidden bg-muted borde
              : result.score >= 40
                ? "relative shrink-0 w-40 h-40 rounded-xl overflow-hidden bg-muted bor
                : "relative shrink-0 w-40 h-40 rounded-xl overflow-hidden bg-muted bor
          }
        >
          {result.record.photoId ? (
            <img
              src={result.record.photoId}
              alt={result.record.name}
              className="w-full h-full object-cover"
              onError={(e) => {
                (e.target as HTMLImageElement).style.display =
                  "none";
              }}
            />
          ) : (
```

```
    <div className="w-full h-full flex items-center justify-center">
      <span className="text-3xl font-bold text-muted-foreground">
        {result.record.name.charAt(0)}
      </span>
    </div>
  )}
  <div
    className={
      result.score >= 65
        ? "absolute bottom-0 left-0 right-0 bg-success/90 text-white text-[11p
        : result.score >= 40
          ? "absolute bottom-0 left-0 right-0 bg-amber-500/90 text-white text-
          : "absolute bottom-0 left-0 right-0 bg-muted-foreground/70 text-whit
    }
  >
    {result.score >= 65
      ? "  MATCH FOUND"
      : result.score >= 40
        ? "  POSSIBLE MATCH"
        : result.score >= 1
          ? "~ LOW SIMILARITY"
          : "Ë CLOSEST VISUAL MATCH"}
  </div>
</div>

<div className="flex-1 min-w-0">
  <div className="flex items-start justify-between gap-2">
    <div>
      <div className="flex items-center gap-2 flex-wrap">
        <p className="font-bold text-lg text-foreground">
          {result.record.name}
        </p>
        <span
          className={
            result.score >= 65
              ? "inline-flex items-center px-2 py-0.5 rounded-full text-xs fon
              : result.score >= 40
                ? "inline-flex items-center px-2 py-0.5 rounded-full text-xs f
                : "inline-flex items-center px-2 py-0.5 rounded-full text-xs f
          }
        >
          {result.score >= 65
            ? "High Confidence Match"
            : result.score >= 40
              ? "Possible Match"
              : result.score >= 1
                ? "Low Similarity"
                : "Closest Visual Match"}
        </span>
      </div>
      <p className="text-xs text-muted-foreground">
        Age {Number(result.record.age)} &bull;{" "}
        {result.record.lastLocation}
      </p>
      <p className="text-xs text-muted-foreground mt-0.5">
```

```
                Contact:{" "}
                <strong>{result.record.contactNumber}</strong>
              </p>
              {result.record.lastSeenPlace && (
                <p className="text-xs text-muted-foreground mt-0.5">
                  Last seen:{" "}
                  <strong>{result.record.lastSeenPlace}</strong>
                </p>
              )}
            </div>
          </div>

          <div className="flex items-center justify-between mt-3">
            <div className="flex items-center gap-2">
              {result.record.status === Status.active ? (
                <span className="inline-flex items-center px-2 py-0.5 rounded-full tex
                  Active
                </span>
              ) : result.record.status === Status.found ? (
                <span className="inline-flex items-center gap-1 px-2 py-0.5 rounded-fu
                  <CheckCircle className="w-3 h-3" /> Found
                </span>
              ) : (
                <span className="inline-flex items-center px-2 py-0.5 rounded-full tex
                  {String(result.record.status)}
                </span>
              )}
            </div>
            {result.record.status === Status.active && (
              <Button
                size="sm"
                variant="outline"
                onClick={() =>
                  handleMarkFound(result.record.contactNumber)
                }
                disabled={
                  markingFound === result.record.contactNumber
                }
                data-ocid="search.confirm_button"
                className="text-xs gap-1 border-success/40 text-success hover:bg-succe
              >
                {markingFound === result.record.contactNumber ? (
                  <Loader2 className="w-3 h-3 animate-spin" />
                ) : (
                  <CheckCircle className="w-3 h-3" />
                )}
                Mark as Found
              </Button>
            )}
          </div>
        </div>
      </div>
    </motion.div>
  );
})}
```

```
            </motion.div>
          )}
        </AnimatePresence>

        {phase === "done" && results.length === 0 && (
          <div
            className="bg-card rounded-xl border border-border shadow-card p-12 text-center"
            data-ocid="search.empty_state"
          >
            <Search className="w-10 h-10 text-muted-foreground mx-auto mb-3" />
            <p className="font-medium text-foreground">
              {hasRegisteredCasesWithPhotos
                ? "No confident match found"
                : "No registered cases with photos"}
            </p>
            <p className="text-sm text-muted-foreground mt-1">
              {hasRegisteredCasesWithPhotos
                ? "The face recognition system did not find a high-confidence match. Use a clear, we
                : "Register missing children with photos first to enable face matching."}
            </p>
          </div>
        )}
      </main>

      <footer className="bg-footer text-footer-foreground mt-auto">
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-6">
          <p className="text-sm text-center">
            &copy; {new Date().getFullYear()} Missing Child Finder. Built with
            love using{" "}
            <a
              href={`https://caffeine.ai?utm_source=caffeine-footer&utm_medium=referral&utm_content=
              target="_blank"
              rel="noopener noreferrer"
              className="underline hover:opacity-80"
            >
              caffeine.ai
            </a>
          </p>
        </div>
      </footer>
    </div>
  );
}


// ============================================================
// FILE: src/frontend/src/pages/AdminPage.tsx
// ============================================================
import {
  AlertDialog,
  AlertDialogAction,
  AlertDialogCancel,
  AlertDialogContent,
  AlertDialogDescription,
  AlertDialogFooter,
```

```
  AlertDialogHeader,
  AlertDialogTitle,
  AlertDialogTrigger,
} from "@/components/ui/alert-dialog";
import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import {
  Select,
  SelectContent,
  SelectItem,
  SelectTrigger,
  SelectValue,
} from "@/components/ui/select";
import { Skeleton } from "@/components/ui/skeleton";
import {
  Table,
  TableBody,
  TableCell,
  TableHead,
  TableHeader,
  TableRow,
} from "@/components/ui/table";
import { useQueryClient } from "@tanstack/react-query";
import { Link, useNavigate } from "@tanstack/react-router";
import { Bell, LogOut, Search, Shield, Trash2 } from "lucide-react";
import { motion } from "motion/react";
import { useState } from "react";
import { Status } from "../backend";
import { useInternetIdentity } from "../hooks/useInternetIdentity";
import {
  useDeleteCase,
  useGetAlerts,
  useGetAllCases,
  useUpdateCaseStatus,
} from "../hooks/useQueries";

function statusBadge(status: Status) {
  switch (status) {
    case Status.active:
      return (
        <Badge className="bg-destructive/10 text-destructive border-destructive/20">
          Active
        </Badge>
      );
    case Status.underReview:
      return (
        <Badge className="bg-yellow-500/10 text-yellow-700 border-yellow-400/20">
          Under Review
        </Badge>
      );
    case Status.found:
      return (
        <Badge className="bg-green-500/10 text-green-700 border-green-400/20">
          Found
        </Badge>
```

```
      );
    case Status.closed:
      return <Badge className="bg-muted text-muted-foreground">Closed</Badge>;
    default:
      return <Badge>{String(status)}</Badge>;
  }
}

export default function AdminPage() {
  const navigate = useNavigate();
  const { clear } = useInternetIdentity();
  const queryClient = useQueryClient();

  const { data: cases, isLoading: casesLoading } = useGetAllCases();
  const { data: alerts, isLoading: alertsLoading } = useGetAlerts();
  const updateStatus = useUpdateCaseStatus();
  const deleteCase = useDeleteCase();

  const [updatingContact, setUpdatingContact] = useState<string | null>(null);

  const handleStatusChange = async (contactNumber: string, status: Status) => {
    setUpdatingContact(contactNumber);
    try {
      await updateStatus.mutateAsync({ contactNumber, status });
    } finally {
      setUpdatingContact(null);
    }
  };

  const handleDelete = async (contactNumber: string) => {
    await deleteCase.mutateAsync(contactNumber);
  };

  const handleLogout = async () => {
    await clear();
    queryClient.clear();
    navigate({ to: "/" });
  };

  return (
    <div className="min-h-screen bg-background flex flex-col">
      {/* Admin Navbar */}
      <header className="sticky top-0 z-50 bg-card border-b border-border shadow-xs">
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
          <div className="flex items-center justify-between h-16">
            <Link
              to="/admin"
              className="flex items-center gap-3"
              data-ocid="admin.link"
            >
              <div className="w-9 h-9 rounded-full bg-primary flex items-center justify-center shado
                <Shield className="w-4 h-4 text-primary-foreground" />
              </div>
              <div className="leading-tight">
                <p className="text-sm font-bold text-foreground leading-none">
```

```
            Admin Panel
          </p>
          <p className="text-xs text-muted-foreground leading-none mt-0.5">
            Missing Child Finder
          </p>
        </div>
      </Link>
      <div className="flex items-center gap-3">
        <Link to="/dashboard">
          <Button
            variant="outline"
            size="sm"
            className="gap-2"
            data-ocid="admin.link"
          >
            <Search className="w-4 h-4" />
            User View
          </Button>
        </Link>
        <Button
          variant="ghost"
          size="sm"
          onClick={handleLogout}
          className="gap-2"
          data-ocid="admin.link"
        >
          <LogOut className="w-4 h-4" />
          Logout
        </Button>
      </div>
    </div>
  </div>
</header>

<main className="flex-1 max-w-7xl mx-auto w-full px-4 sm:px-6 lg:px-8 py-8 space-y-10">
  {/* Cases Management */}
  <section data-ocid="admin.panel">
    <motion.div
      initial={{ opacity: 0, y: 10 }}
      animate={{ opacity: 1, y: 0 }}
      transition={{ duration: 0.3 }}
    >
      <h2 className="text-xl font-bold text-foreground mb-1">
        Case Management
      </h2>
      <p className="text-sm text-muted-foreground mb-5">
        Review, update status, and delete missing child cases.
      </p>

      <div className="bg-card rounded-xl border border-border shadow-card overflow-hidden">
        {casesLoading ? (
          <div className="p-6 space-y-3" data-ocid="admin.loading_state">
            {["a", "b", "c", "d"].map((k) => (
              <Skeleton key={k} className="h-10 w-full rounded" />
            ))}
```

```
    </div>
  ) : !cases || cases.length === 0 ? (
    <div className="p-12 text-center" data-ocid="admin.empty_state">
      <p className="text-muted-foreground">
        No cases registered yet.
      </p>
    </div>
  ) : (
    <Table data-ocid="admin.table">
      <TableHeader>
        <TableRow>
          <TableHead>Name</TableHead>
          <TableHead>Age</TableHead>
          <TableHead>Last Location</TableHead>
          <TableHead>Contact</TableHead>
          <TableHead>Status</TableHead>
          <TableHead className="text-right">Actions</TableHead>
        </TableRow>
      </TableHeader>
      <TableBody>
        {cases.map((c, idx) => (
          <TableRow
            key={c.contactNumber}
            data-ocid={`admin.item.${idx + 1}`}
          >
            <TableCell className="font-medium">{c.name}</TableCell>
            <TableCell>{Number(c.age)}</TableCell>
            <TableCell className="max-w-[180px] truncate">
              {c.lastLocation}
            </TableCell>
            <TableCell>{c.contactNumber}</TableCell>
            <TableCell>
              <Select
                value={c.status}
                onValueChange={(v) =>
                  handleStatusChange(c.contactNumber, v as Status)
                }
                disabled={updatingContact === c.contactNumber}
              >
                <SelectTrigger
                  className="w-36 h-8 text-xs"
                  data-ocid="admin.select"
                >
                  <SelectValue>{statusBadge(c.status)}</SelectValue>
                </SelectTrigger>
                <SelectContent>
                  <SelectItem value={Status.active}>
                    Active
                  </SelectItem>
                  <SelectItem value={Status.underReview}>
                    Under Review
                  </SelectItem>
                  <SelectItem value={Status.found}>
                    Found
                  </SelectItem>
```

```
                <SelectItem value={Status.closed}>
                  Closed
                </SelectItem>
              </SelectContent>
            </Select>
          </TableCell>
          <TableCell className="text-right">
            <AlertDialog>
              <AlertDialogTrigger asChild>
                <Button
                  size="sm"
                  variant="ghost"
                  className="text-destructive hover:bg-destructive/10 hover:text-destr
                  data-ocid={`admin.delete_button.${idx + 1}`}
                >
                  <Trash2 className="w-4 h-4" />
                </Button>
              </AlertDialogTrigger>
              <AlertDialogContent data-ocid="admin.dialog">
                <AlertDialogHeader>
                  <AlertDialogTitle>Delete Case</AlertDialogTitle>
                  <AlertDialogDescription>
                    Are you sure you want to delete the case for{" "}
                    <strong>{c.name}</strong>? This action cannot
                    be undone.
                  </AlertDialogDescription>
                </AlertDialogHeader>
                <AlertDialogFooter>
                  <AlertDialogCancel data-ocid="admin.cancel_button">
                    Cancel
                  </AlertDialogCancel>
                  <AlertDialogAction
                    className="bg-destructive text-destructive-foreground hover:bg-des
                    onClick={() => handleDelete(c.contactNumber)}
                    data-ocid="admin.confirm_button"
                  >
                    Delete
                  </AlertDialogAction>
                </AlertDialogFooter>
              </AlertDialogContent>
            </AlertDialog>
          </TableCell>
        </TableRow>
      ))}
    </TableBody>
  </Table>
)}
</div>
</motion.div>
</section>

{/* Alerts Section */}
<section data-ocid="admin.panel">
<motion.div
  initial={{ opacity: 0, y: 10 }}
```

```
      animate={{ opacity: 1, y: 0 }}
      transition={{ duration: 0.3, delay: 0.1 }}
    >
      <div className="flex items-center gap-2 mb-1">
        <Bell className="w-5 h-5 text-primary" />
        <h2 className="text-xl font-bold text-foreground">
          Match Alerts
        </h2>
      </div>
      <p className="text-sm text-muted-foreground mb-5">
        Alerts generated when a face match was found during a search.
      </p>

      <div className="bg-card rounded-xl border border-border shadow-card overflow-hidden">
        {alertsLoading ? (
          <div className="p-6 space-y-3" data-ocid="admin.loading_state">
            {["a", "b", "c"].map((k) => (
              <Skeleton key={k} className="h-10 w-full rounded" />
            ))}
          </div>
        ) : !alerts || alerts.length === 0 ? (
          <div className="p-12 text-center" data-ocid="admin.empty_state">
            <Bell className="w-8 h-8 text-muted-foreground mx-auto mb-2" />
            <p className="text-muted-foreground">
              No alerts have been sent yet.
            </p>
          </div>
        ) : (
          <div class="divide-y divide-border">
            {alerts.map((alert, idx) => (
              <div
                key={`${alert.contactNumber}-${idx}`}
                className="flex items-start gap-4 p-4"
                data-ocid={`admin.item.${idx + 1}`}
              >
                <div class="w-8 h-8 rounded-full bg-destructive/10 flex items-center justi
                  <Bell class="w-4 h-4 text-destructive" />
                </div>
                <div class="flex-1 min-w-0">
                  <p class="text-sm font-medium text-foreground">
                    {alert.message}
                  </p>
                  <p class="text-xs text-muted-foreground mt-0.5">
                    Sent to: <strong>{alert.contactNumber}</strong>
                  </p>
                </div>
              </div>
            ))}
          </div>
        )}
      </div>
    </motion.div>
  </section>
</main>
```

```
      <footer className="py-6 text-center text-xs text-muted-foreground">
        &copy; {new Date().getFullYear()}. Built with love using{" "}
        <a
          href={`https://caffeine.ai?utm_source=caffeine-footer&utm_medium=referral&utm_content=${en
          target="_blank"
          rel="noopener noreferrer"
          className="underline hover:opacity-80"
        >
          caffeine.ai
        </a>
      </footer>
    </div>
  );
}


// ============================================================
// FILE: src/frontend/src/pages/AccountPage.tsx
// ============================================================
import { Badge } from "@/components/ui/badge";
import { Skeleton } from "@/components/ui/skeleton";
import { Database, ShieldCheck, User } from "lucide-react";
import { UserRole } from "../backend";
import Navbar from "../components/Navbar";
import { useInternetIdentity } from "../hooks/useInternetIdentity";
import { useGetCallerProfile, useGetCallerRole } from "../hooks/useQueries";

function roleBadge(role: UserRole | undefined) {
  if (role === UserRole.admin)
    return (
      <Badge className="bg-primary/10 text-primary border-primary/20">
        Admin
      </Badge>
    );
  if (role === UserRole.user)
    return (
      <Badge className="bg-secondary/10 text-secondary border-secondary/20">
        User
      </Badge>
    );
  return <Badge variant="outline">Guest</Badge>;
}

export default function AccountPage() {
  const { identity } = useInternetIdentity();
  const { data: profile, isLoading: profileLoading } = useGetCallerProfile();
  const { data: role, isLoading: roleLoading } = useGetCallerRole();

  const principalId = identity?.getPrincipal().toString() ?? " ";

  return (
    <div className="min-h-screen bg-background flex flex-col">
      <Navbar />

      <div className="bg-card border-b border-border">
```

```
  <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-6">
    <h1 className="text-2xl font-bold text-foreground">My Account</h1>
    <p className="text-muted-foreground text-sm mt-1">
      Your account details and storage information
    </p>
  </div>
</div>

<main className="flex-1 max-w-2xl mx-auto w-full px-4 sm:px-6 lg:px-8 py-8">
  <div className="bg-card rounded-xl border border-border shadow-card p-6 space-y-5">
    {/* Username */}
    <div className="flex items-center justify-between py-3 border-b border-border">
      <div className="flex items-center gap-3">
        <div className="w-8 h-8 rounded-full bg-primary/10 flex items-center justify-center">
          <User className="w-4 h-4 text-primary" />
        </div>
        <span className="text-sm text-muted-foreground font-medium">
          Username
        </span>
      </div>
      {profileLoading ? (
        <Skeleton className="h-5 w-32" />
      ) : (
        <span
          className="text-sm font-semibold text-foreground"
          data-ocid="account.panel"
        >
          {profile?.name ?? " "}
        </span>
      )}
    </div>

    {/* Principal ID */}
    <div className="flex items-start justify-between py-3 border-b border-border gap-4">
      <div className="flex items-center gap-3 shrink-0">
        <div className="w-8 h-8 rounded-full bg-secondary/10 flex items-center justify-center"
          <ShieldCheck className="w-4 h-4 text-secondary" />
        </div>
        <span className="text-sm text-muted-foreground font-medium">
          Principal ID
        </span>
      </div>
      <span className="text-xs font-mono text-foreground break-all text-right">
        {principalId}
      </span>
    </div>

    {/* Role */}
    <div className="flex items-center justify-between py-3 border-b border-border">
      <div className="flex items-center gap-3">
        <div className="w-8 h-8 rounded-full bg-success/10 flex items-center justify-center">
          <ShieldCheck className="w-4 h-4 text-success" />
        </div>
        <span className="text-sm text-muted-foreground font-medium">
          Role
```

```
              </span>
            </div>
            {roleLoading ? (
              <Skeleton className="h-5 w-16" />
            ) : (
              <span data-ocid="account.card">{roleBadge(role)}</span>
            )}
          </div>

          {/* Storage */}
          <div className="flex items-center justify-between py-3">
            <div className="flex items-center gap-3">
              <div className="w-8 h-8 rounded-full bg-primary/10 flex items-center justify-center">
                <Database className="w-4 h-4 text-primary" />
              </div>
              <span className="text-sm text-muted-foreground font-medium">
                Storage
              </span>
            </div>
            <Badge className="bg-primary/10 text-primary border-primary/20 gap-1.5">
              <Database className="w-3 h-3" />
              512 GB Storage Allocated
            </Badge>
          </div>
        </div>
      </main>

      <footer className="bg-footer text-footer-foreground mt-auto">
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-6">
          <p className="text-sm text-center">
            &copy; {new Date().getFullYear()} Missing Child Finder. Built with
            love using{" "}
            <a
              href={`https://caffeine.ai?utm_source=caffeine-footer&utm_medium=referral&utm_content=
              target="_blank"
              rel="noopener noreferrer"
              className="underline hover:opacity-80"
            >
              caffeine.ai
            </a>
          </p>
        </div>
      </footer>
    </div>
  );
}


// ============================================================
// FILE: src/frontend/src/pages/AdminLoginPage.tsx
// ============================================================
import { Alert, AlertDescription } from "@/components/ui/alert";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
```

```
import { useNavigate } from "@tanstack/react-router";
import {
  AlertCircle,
  CheckCircle,
  Eye,
  EyeOff,
  Loader2,
  RefreshCw,
  Shield,
} from "lucide-react";
import { motion } from "motion/react";
import { useState } from "react";
import { useActor } from "../hooks/useActor";
import { useInternetIdentity } from "../hooks/useInternetIdentity";

export default function AdminLoginPage() {
  const navigate = useNavigate();
  const { login, loginStatus, identity } = useInternetIdentity();
  const {
    actor,
    isFetching: actorLoading,
    isError: actorError,
    refetch: refetchActor,
  } = useActor();

  const [password, setPassword] = useState("");
  const [showPassword, setShowPassword] = useState(false);
  const [error, setError] = useState("");
  const [claiming, setClaiming] = useState(false);
  const [success, setSuccess] = useState(false);

  const isLoggingIn = loginStatus === "logging-in";

  const handleIdentityLogin = async () => {
    setError("");
    try {
      await login();
    } catch (_err: unknown) {
      setError("Login failed. Please try again.");
    }
  };

  const handleClaimAdmin = async (e: React.FormEvent) => {
    e.preventDefault();
    setError("");

    if (!password.trim()) {
      setError("Admin password is required");
      return;
    }
    if (!actor) {
      setError("Backend not connected. Please click Retry.");
      return;
    }
```

```
  setClaiming(true);
  try {
    await actor.claimAdminRole(password);
    setSuccess(true);
    setTimeout(() => navigate({ to: "/admin" }), 800);
  } catch (err: unknown) {
    const msg = String((err as { message?: string })?.message ?? err);
    if (
      msg.includes("Invalid password") ||
      msg.includes("invalid password") ||
      msg.includes("wrong password")
    ) {
      setError("Incorrect admin password. Please use the correct password.");
    } else if (
      msg.includes("IC0508") ||
      msg.includes("is stopped") ||
      msg.includes("temporarily unavailable")
    ) {
      setError(
        "Backend is temporarily restarting. Please wait 30 seconds and try again.",
      );
    } else {
      setError(msg || "Failed to claim admin role. Please try again.");
    }
  } finally {
    setClaiming(false);
  }
};

return (
  <div className="min-h-screen bg-background flex items-center justify-center px-4 py-12">
    <motion.div
      initial={{ opacity: 0, y: 20 }}
      animate={{ opacity: 1, y: 0 }}
      transition={{ duration: 0.4 }}
      className="w-full max-w-md"
    >
      {/* Brand */}
      <div className="text-center mb-8">
        <div className="inline-flex items-center justify-center w-16 h-16 rounded-full bg-primary
          <Shield className="w-7 h-7 text-primary-foreground" />
        </div>
        <h1 className="text-2xl font-bold text-foreground">Admin Login</h1>
        <p className="text-muted-foreground text-sm mt-1">
          Restricted access   authorized administrators only
        </p>
      </div>

      <div className="bg-card rounded-xl shadow-card border border-border p-8 space-y-6">
        {/* Backend connection error */}
        {actorError && !actorLoading && (
          <Alert variant="destructive">
            <AlertCircle className="h-4 w-4" />
            <AlertDescription className="flex items-center justify-between">
              <span>Backend not connected. Click Retry to reconnect.</span>
```

```
      <Button
        variant="outline"
        size="sm"
        onClick={() => refetchActor()}
        className="ml-2 h-7 text-xs"
      >
        <RefreshCw className="w-3 h-3 mr-1" /> Retry
      </Button>
    </AlertDescription>
  </Alert>
)}

{/* Success message */}
{success && (
  <Alert className="border-green-200 bg-green-50 dark:border-green-800 dark:bg-green-950">
    <CheckCircle className="h-4 w-4 text-green-500" />
    <AlertDescription className="text-green-700 dark:text-green-300">
      Admin access granted! Redirecting...
    </AlertDescription>
  </Alert>
)}

{/* General error */}
{error && (
  <Alert variant="destructive" data-ocid="admin-login.error_state">
    <AlertCircle className="h-4 w-4" />
    <AlertDescription>{error}</AlertDescription>
  </Alert>
)}

{/* Step 1: Internet Identity */}
{!identity ? (
  <div className="space-y-4">
    <div className="p-4 bg-muted/40 rounded-lg text-sm text-muted-foreground">
      <p className="font-medium text-foreground mb-1">
        Step 1: Authenticate with Internet Identity
      </p>
      <p>
        Sign in with your Internet Identity to verify your identity
        before entering the admin password.
      </p>
    </div>
    <Button
      className="w-full bg-primary text-primary-foreground hover:bg-primary/90"
      onClick={handleIdentityLogin}
      disabled={isLoggingIn}
      data-ocid="admin-login.primary_button"
    >
      {isLoggingIn ? (
        <span className="flex items-center gap-2">
          <Loader2 className="w-4 h-4 animate-spin" />
          Authenticating...
        </span>
      ) : (
        "Login with Internet Identity"
```

```
      )}
    </Button>
  </div>
) : (
  <div className="space-y-4">
    <div className="p-4 bg-muted/40 rounded-lg text-sm">
      <p className="font-medium text-foreground mb-1">
        Step 2: Enter Admin Password
      </p>
      <p className="text-muted-foreground">
        Enter the admin passcode to claim admin access.
      </p>
    </div>

    {actorLoading && (
      <div className="flex items-center gap-2 text-sm text-muted-foreground">
        <Loader2 className="w-4 h-4 animate-spin" />
        Connecting to backend...
      </div>
    )}

    <form onSubmit={handleClaimAdmin} className="space-y-4">
      <div className="space-y-1.5">
        <Label htmlFor="admin-password">Admin Password</Label>
        <div className="relative">
          <Input
            id="admin-password"
            type={showPassword ? "text" : "password"}
            placeholder="Enter admin password"
            value={password}
            onChange={(e) => setPassword(e.target.value)}
            className="pr-10"
            data-ocid="admin-login.input"
            autoComplete="current-password"
            disabled={claiming || success}
          />
          <button
            type="button"
            className="absolute right-3 top-1/2 -translate-y-1/2 text-muted-foreground hov
            onClick={() => setShowPassword((v) => !v)}
          >
            {showPassword ? (
              <EyeOff className="w-4 h-4" />
            ) : (
              <Eye className="w-4 h-4" />
            )}
          </button>
        </div>
      </div>
      <Button
        type="submit"
        className="w-full bg-primary text-primary-foreground hover:bg-primary/90"
        disabled={claiming || actorLoading || !actor || success}
        data-ocid="admin-login.submit_button"
      >
```

```
                  {claiming ? (
                    <span className="flex items-center gap-2">
                      <Loader2 className="w-4 h-4 animate-spin" />
                      Verifying...
                    </span>
                  ) : actorLoading ? (
                    <span className="flex items-center gap-2">
                      <Loader2 className="w-4 h-4 animate-spin" />
                      Connecting...
                    </span>
                  ) : (
                    "Access Admin Panel"
                  )}
                </Button>
              </form>
            </div>
          )}
        </div>

        <button
          type="button"
          className="mt-4 w-full text-center text-sm text-muted-foreground hover:text-foreground tra
          onClick={() => navigate({ to: "/" })}
          data-ocid="admin-login.link"
        >
          • Back to Home
        </button>
      </motion.div>
    </div>
  );
}


// ============================================================
// FILE: src/frontend/src/backend.d.ts
// ============================================================
import type { Principal } from "@icp-sdk/core/principal";
export interface Some<T> {
    __kind__: "Some";
    value: T;
}
export interface None {
    __kind__: "None";
}
export type Option<T> = Some<T> | None;
export interface ChildRecord {
    age: bigint;
    status: Status;
    lastSeenPlace: string;
    lastLocation: string;
    name: string;
    contactNumber: string;
    photoId: string;
}
export interface Alert {
```

```
    message: string;
    contactNumber: string;
}
export interface DashboardStats {
    totalCases: bigint;
    activeCases: bigint;
    foundCases: bigint;
}
export interface UserProfile {
    name: string;
}
export enum Status {
    found = "found",
    closed = "closed",
    active = "active",
    underReview = "underReview"
}
export enum UserRole {
    admin = "admin",
    user = "user",
    guest = "guest"
}
export interface backendInterface {
    addAlert(input: [string, string]): Promise<void>;
    assignCallerUserRole(user: Principal, role: UserRole): Promise<void>;
    claimAdminRole(password: string): Promise<void>;
    deleteCase(contactNumber: string): Promise<void>;
    getAlerts(): Promise<Array<Alert>>;
    getAllCases(): Promise<Array<ChildRecord>>;
    getCallerUserProfile(): Promise<UserProfile | null>;
    getCallerUserRole(): Promise<UserRole>;
    getCase(contactNumber: string): Promise<ChildRecord>;
    getDashboardStats(): Promise<DashboardStats>;
    getUserProfile(user: Principal): Promise<UserProfile | null>;
    isCallerAdmin(): Promise<boolean>;
    registerCase(newRecord: ChildRecord): Promise<void>;
    saveCallerUserProfile(profile: UserProfile): Promise<void>;
    updateCaseStatus(input: [string, Status]): Promise<void>;
    updateStatusToFound(contactNumber: string): Promise<void>;
}


// ============================================================
// FILE: src/frontend/src/backend.ts
// ============================================================
/* eslint-disable */

// @ts-nocheck

// This file was automatically generated by @caffeinelabs/bindgen@0.1.0.
// You should NOT make any changes in this file as it will be overwritten.
// Additionally, you should also exclude this file from your linter and/or formatter to prevent it f

import { Actor, HttpAgent, type HttpAgentOptions, type ActorConfig, type Agent, type ActorSubclass }
import type { Principal } from "@icp-sdk/core/principal";
```

```
import { idlFactory, type _SERVICE } from "./declarations/backend.did";
export interface Some<T> {
    __kind__: "Some";
    value: T;
}
export interface None {
    __kind__: "None";
}
export type Option<T> = Some<T> | None;
function some<T>(value: T): Some<T> {
    return {
        __kind__: "Some",
        value: value
    };
}
function none(): None {
    return {
        __kind__: "None"
    };
}
function isNone<T>(option: Option<T>): option is None {
    return option.__kind__ === "None";
}
function isSome<T>(option: Option<T>): option is Some<T> {
    return option.__kind__ === "Some";
}
function unwrap<T>(option: Option<T>): T {
    if (isNone(option)) {
        throw new Error("unwrap: none");
    }
    return option.value;
}
function candid_some<T>(value: T): [T] {
    return [
        value
    ];
}
function candid_none<T>(): [] {
    return [];
}
function record_opt_to_undefined<T>(arg: T | null): T | undefined {
    return arg == null ? undefined : arg;
}
export class ExternalBlob {
    _blob?: Uint8Array<ArrayBuffer> | null;
    directURL: string;
    onProgress?: (percentage: number) => void = undefined;
    private constructor(directURL: string, blob: Uint8Array<ArrayBuffer> | null){
        if (blob) {
            this._blob = blob;
        }
        this.directURL = directURL;
    }
    static fromURL(url: string): ExternalBlob {
        return new ExternalBlob(url, null);
```

```
    }
    static fromBytes(blob: Uint8Array<ArrayBuffer>): ExternalBlob {
        const url = URL.createObjectURL(new Blob([
            new Uint8Array(blob)
        ], {
            type: 'application/octet-stream'
        }));
        return new ExternalBlob(url, blob);
    }
    public async getBytes(): Promise<Uint8Array<ArrayBuffer>> {
        if (this._blob) {
            return this._blob;
        }
        const response = await fetch(this.directURL);
        const blob = await response.blob();
        this._blob = new Uint8Array(await blob.arrayBuffer());
        return this._blob;
    }
    public getDirectURL(): string {
        return this.directURL;
    }
    public withUploadProgress(onProgress: (percentage: number) => void): ExternalBlob {
        this.onProgress = onProgress;
        return this;
    }
}
export interface ChildRecord {
    age: bigint;
    status: Status;
    lastSeenPlace: string;
    lastLocation: string;
    name: string;
    contactNumber: string;
    photoId: string;
}
export interface Alert {
    message: string;
    contactNumber: string;
}
export interface _CaffeineStorageRefillInformation {
    proposed_top_up_amount?: bigint;
}
export interface _CaffeineStorageCreateCertificateResult {
    method: string;
    blob_hash: string;
}
export interface DashboardStats {
    totalCases: bigint;
    activeCases: bigint;
    foundCases: bigint;
}
export interface UserProfile {
    name: string;
}
export interface _CaffeineStorageRefillResult {
```

```
    success?: boolean;
    topped_up_amount?: bigint;
}
export enum Status {
    found = "found",
    closed = "closed",
    active = "active",
    underReview = "underReview"
}
export enum UserRole {
    admin = "admin",
    user = "user",
    guest = "guest"
}
export interface backendInterface {
    _caffeineStorageBlobIsLive(hash: Uint8Array): Promise<boolean>;
    _caffeineStorageBlobsToDelete(): Promise<Array<Uint8Array>>;
    _caffeineStorageConfirmBlobDeletion(blobs: Array<Uint8Array>): Promise<void>;
    _caffeineStorageCreateCertificate(blobHash: string): Promise<_CaffeineStorageCreateCertificateRe
    _caffeineStorageRefillCashier(refillInformation: _CaffeineStorageRefillInformation | null): Prom
    _caffeineStorageUpdateGatewayPrincipals(): Promise<void>;
    _initializeAccessControlWithSecret(userSecret: string): Promise<void>;
    addAlert(input: [string, string]): Promise<void>;
    assignCallerUserRole(user: Principal, role: UserRole): Promise<void>;
    claimAdminRole(password: string): Promise<void>;
    deleteCase(contactNumber: string): Promise<void>;
    getAlerts(): Promise<Array<Alert>>;
    getAllCases(): Promise<Array<ChildRecord>>;
    getCallerUserProfile(): Promise<UserProfile | null>;
    getCallerUserRole(): Promise<UserRole>;
    getCase(contactNumber: string): Promise<ChildRecord>;
    getDashboardStats(): Promise<DashboardStats>;
    getUserProfile(user: Principal): Promise<UserProfile | null>;
    isCallerAdmin(): Promise<boolean>;
    registerCase(newRecord: ChildRecord): Promise<void>;
    saveCallerUserProfile(profile: UserProfile): Promise<void>;
    updateCaseStatus(input: [string, Status]): Promise<void>;
    updateStatusToFound(contactNumber: string): Promise<void>;
}
import type { ChildRecord as _ChildRecord, Status as _Status, UserProfile as _UserProfile, UserRole
export class Backend implements backendInterface {
    constructor(private actor: ActorSubclass<_SERVICE>, private _uploadFile: (file: ExternalBlob) =>
    async _caffeineStorageBlobIsLive(arg0: Uint8Array): Promise<boolean> {
        if (this.processError) {
            try {
                const result = await this.actor._caffeineStorageBlobIsLive(arg0);
                return result;
            } catch (e) {
                this.processError(e);
                throw new Error("unreachable");
            }
        } else {
            const result = await this.actor._caffeineStorageBlobIsLive(arg0);
            return result;
        }
```

```
}
async _caffeineStorageBlobsToDelete(): Promise<Array<Uint8Array>> {
    if (this.processError) {
        try {
            const result = await this.actor._caffeineStorageBlobsToDelete();
            return result;
        } catch (e) {
            this.processError(e);
            throw new Error("unreachable");
        }
    } else {
        const result = await this.actor._caffeineStorageBlobsToDelete();
        return result;
    }
}
async _caffeineStorageConfirmBlobDeletion(arg0: Array<Uint8Array>): Promise<void> {
    if (this.processError) {
        try {
            const result = await this.actor._caffeineStorageConfirmBlobDeletion(arg0);
            return result;
        } catch (e) {
            this.processError(e);
            throw new Error("unreachable");
        }
    } else {
        const result = await this.actor._caffeineStorageConfirmBlobDeletion(arg0);
        return result;
    }
}
async _caffeineStorageCreateCertificate(arg0: string): Promise<_CaffeineStorageCreateCertificate
    if (this.processError) {
        try {
            const result = await this.actor._caffeineStorageCreateCertificate(arg0);
            return result;
        } catch (e) {
            this.processError(e);
            throw new Error("unreachable");
        }
    } else {
        const result = await this.actor._caffeineStorageCreateCertificate(arg0);
        return result;
    }
}
async _caffeineStorageRefillCashier(arg0: _CaffeineStorageRefillInformation | null): Promise<_Ca
    if (this.processError) {
        try {
            const result = await this.actor._caffeineStorageRefillCashier(to_candid_opt_n1(this.
            return from_candid__CaffeineStorageRefillResult_n4(this._uploadFile, this._downloadF
        } catch (e) {
            this.processError(e);
            throw new Error("unreachable");
        }
    } else {
        const result = await this.actor._caffeineStorageRefillCashier(to_candid_opt_n1(this._upl
        return from_candid__CaffeineStorageRefillResult_n4(this._uploadFile, this._downloadFile,
```

```
    }
}
async _caffeineStorageUpdateGatewayPrincipals(): Promise<void> {
    if (this.processError) {
        try {
            const result = await this.actor._caffeineStorageUpdateGatewayPrincipals();
            return result;
        } catch (e) {
            this.processError(e);
            throw new Error("unreachable");
        }
    } else {
        const result = await this.actor._caffeineStorageUpdateGatewayPrincipals();
        return result;
    }
}
async _initializeAccessControlWithSecret(arg0: string): Promise<void> {
    if (this.processError) {
        try {
            const result = await this.actor._initializeAccessControlWithSecret(arg0);
            return result;
        } catch (e) {
            this.processError(e);
            throw new Error("unreachable");
        }
    } else {
        const result = await this.actor._initializeAccessControlWithSecret(arg0);
        return result;
    }
}
async addAlert(arg0: [string, string]): Promise<void> {
    if (this.processError) {
        try {
            const result = await this.actor.addAlert(arg0);
            return result;
        } catch (e) {
            this.processError(e);
            throw new Error("unreachable");
        }
    } else {
        const result = await this.actor.addAlert(arg0);
        return result;
    }
}
async assignCallerUserRole(arg0: Principal, arg1: UserRole): Promise<void> {
    if (this.processError) {
        try {
            const result = await this.actor.assignCallerUserRole(arg0, to_candid_UserRole_n8(thi
            return result;
        } catch (e) {
            this.processError(e);
            throw new Error("unreachable");
        }
    } else {
        const result = await this.actor.assignCallerUserRole(arg0, to_candid_UserRole_n8(this._u
```

```
        return result;
    }
}
async claimAdminRole(arg0: string): Promise<void> {
    if (this.processError) {
        try {
            const result = await this.actor.claimAdminRole(arg0);
            return result;
        } catch (e) {
            this.processError(e);
            throw new Error("unreachable");
        }
    } else {
        const result = await this.actor.claimAdminRole(arg0);
        return result;
    }
}
async deleteCase(arg0: string): Promise<void> {
    if (this.processError) {
        try {
            const result = await this.actor.deleteCase(arg0);
            return result;
        } catch (e) {
            this.processError(e);
            throw new Error("unreachable");
        }
    } else {
        const result = await this.actor.deleteCase(arg0);
        return result;
    }
}
async getAlerts(): Promise<Array<Alert>> {
    if (this.processError) {
        try {
            const result = await this.actor.getAlerts();
            return result;
        } catch (e) {
            this.processError(e);
            throw new Error("unreachable");
        }
    } else {
        const result = await this.actor.getAlerts();
        return result;
    }
}
async getAllCases(): Promise<Array<ChildRecord>> {
    if (this.processError) {
        try {
            const result = await this.actor.getAllCases();
            return from_candid_vec_n10(this._uploadFile, this._downloadFile, result);
        } catch (e) {
            this.processError(e);
            throw new Error("unreachable");
        }
    } else {
```

```
        const result = await this.actor.getAllCases();
        return from_candid_vec_n10(this._uploadFile, this._downloadFile, result);
    }
}
async getCallerUserProfile(): Promise<UserProfile | null> {
    if (this.processError) {
        try {
            const result = await this.actor.getCallerUserProfile();
            return from_candid_opt_n15(this._uploadFile, this._downloadFile, result);
        } catch (e) {
            this.processError(e);
            throw new Error("unreachable");
        }
    } else {
        const result = await this.actor.getCallerUserProfile();
        return from_candid_opt_n15(this._uploadFile, this._downloadFile, result);
    }
}
async getCallerUserRole(): Promise<UserRole> {
    if (this.processError) {
        try {
            const result = await this.actor.getCallerUserRole();
            return from_candid_UserRole_n16(this._uploadFile, this._downloadFile, result);
        } catch (e) {
            this.processError(e);
            throw new Error("unreachable");
        }
    } else {
        const result = await this.actor.getCallerUserRole();
        return from_candid_UserRole_n16(this._uploadFile, this._downloadFile, result);
    }
}
async getCase(arg0: string): Promise<ChildRecord> {
    if (this.processError) {
        try {
            const result = await this.actor.getCase(arg0);
            return from_candid_ChildRecord_n11(this._uploadFile, this._downloadFile, result);
        } catch (e) {
            this.processError(e);
            throw new Error("unreachable");
        }
    } else {
        const result = await this.actor.getCase(arg0);
        return from_candid_ChildRecord_n11(this._uploadFile, this._downloadFile, result);
    }
}
async getDashboardStats(): Promise<DashboardStats> {
    if (this.processError) {
        try {
            const result = await this.actor.getDashboardStats();
            return result;
        } catch (e) {
            this.processError(e);
            throw new Error("unreachable");
        }
```

```
    } else {
        const result = await this.actor.getDashboardStats();
        return result;
    }
}
async getUserProfile(arg0: Principal): Promise<UserProfile | null> {
    if (this.processError) {
        try {
            const result = await this.actor.getUserProfile(arg0);
            return from_candid_opt_n15(this._uploadFile, this._downloadFile, result);
        } catch (e) {
            this.processError(e);
            throw new Error("unreachable");
        }
    } else {
        const result = await this.actor.getUserProfile(arg0);
        return from_candid_opt_n15(this._uploadFile, this._downloadFile, result);
    }
}
async isCallerAdmin(): Promise<boolean> {
    if (this.processError) {
        try {
            const result = await this.actor.isCallerAdmin();
            return result;
        } catch (e) {
            this.processError(e);
            throw new Error("unreachable");
        }
    } else {
        const result = await this.actor.isCallerAdmin();
        return result;
    }
}
async registerCase(arg0: ChildRecord): Promise<void> {
    if (this.processError) {
        try {
            const result = await this.actor.registerCase(to_candid_ChildRecord_n18(this._uploadF
            return result;
        } catch (e) {
            this.processError(e);
            throw new Error("unreachable");
        }
    } else {
        const result = await this.actor.registerCase(to_candid_ChildRecord_n18(this._uploadFile,
        return result;
    }
}
async saveCallerUserProfile(arg0: UserProfile): Promise<void> {
    if (this.processError) {
        try {
            const result = await this.actor.saveCallerUserProfile(arg0);
            return result;
        } catch (e) {
            this.processError(e);
            throw new Error("unreachable");
```

```
            }
        } else {
            const result = await this.actor.saveCallerUserProfile(arg0);
            return result;
        }
    }
    async updateCaseStatus(arg0: [string, Status]): Promise<void> {
        if (this.processError) {
            try {
                const result = await this.actor.updateCaseStatus(to_candid_tuple_n22(this._uploadFil
                return result;
            } catch (e) {
                this.processError(e);
                throw new Error("unreachable");
            }
        } else {
            const result = await this.actor.updateCaseStatus(to_candid_tuple_n22(this._uploadFile, t
            return result;
        }
    }
    async updateStatusToFound(arg0: string): Promise<void> {
        if (this.processError) {
            try {
                const result = await this.actor.updateStatusToFound(arg0);
                return result;
            } catch (e) {
                this.processError(e);
                throw new Error("unreachable");
            }
        } else {
            const result = await this.actor.updateStatusToFound(arg0);
            return result;
        }
    }
}
function from_candid_ChildRecord_n11(_uploadFile: (file: ExternalBlob) => Promise<Uint8Array>, _down
    return from_candid_record_n12(_uploadFile, _downloadFile, value);
}
function from_candid_Status_n13(_uploadFile: (file: ExternalBlob) => Promise<Uint8Array>, _downloadF
    return from_candid_variant_n14(_uploadFile, _downloadFile, value);
}
function from_candid_UserRole_n16(_uploadFile: (file: ExternalBlob) => Promise<Uint8Array>, _downloa
    return from_candid_variant_n17(_uploadFile, _downloadFile, value);
}
function from_candid__CaffeineStorageRefillResult_n4(_uploadFile: (file: ExternalBlob) => Promise<Ui
    return from_candid_record_n5(_uploadFile, _downloadFile, value);
}
function from_candid_opt_n15(_uploadFile: (file: ExternalBlob) => Promise<Uint8Array>, _downloadFile
    return value.length === 0 ? null : value[0];
}
function from_candid_opt_n6(_uploadFile: (file: ExternalBlob) => Promise<Uint8Array>, _downloadFile:
    return value.length === 0 ? null : value[0];
}
function from_candid_opt_n7(_uploadFile: (file: ExternalBlob) => Promise<Uint8Array>, _downloadFile:
    return value.length === 0 ? null : value[0];
```

```
}
function from_candid_record_n12(_uploadFile: (file: ExternalBlob) => Promise<Uint8Array>, _downloadF
    age: bigint;
    status: _Status;
    lastSeenPlace: string;
    lastLocation: string;
    name: string;
    contactNumber: string;
    photoId: string;
}): {
    age: bigint;
    status: Status;
    lastSeenPlace: string;
    lastLocation: string;
    name: string;
    contactNumber: string;
    photoId: string;
} {
    return {
        age: value.age,
        status: from_candid_Status_n13(_uploadFile, _downloadFile, value.status),
        lastSeenPlace: value.lastSeenPlace,
        lastLocation: value.lastLocation,
        name: value.name,
        contactNumber: value.contactNumber,
        photoId: value.photoId
    };
}
function from_candid_record_n5(_uploadFile: (file: ExternalBlob) => Promise<Uint8Array>, _downloadFi
    success: [] | [boolean];
    topped_up_amount: [] | [bigint];
}): {
    success?: boolean;
    topped_up_amount?: bigint;
} {
    return {
        success: record_opt_to_undefined(from_candid_opt_n6(_uploadFile, _downloadFile, value.succes
        topped_up_amount: record_opt_to_undefined(from_candid_opt_n7(_uploadFile, _downloadFile, val
    };
}
function from_candid_variant_n14(_uploadFile: (file: ExternalBlob) => Promise<Uint8Array>, _download
    found: null;
} | {
    closed: null;
} | {
    active: null;
} | {
    underReview: null;
}): Status {
    return "found" in value ? Status.found : "closed" in value ? Status.closed : "active" in value ?
}
function from_candid_variant_n17(_uploadFile: (file: ExternalBlob) => Promise<Uint8Array>, _download
    admin: null;
} | {
    user: null;
```

```
} | {
    guest: null;
}): UserRole {
    return "admin" in value ? UserRole.admin : "user" in value ? UserRole.user : "guest" in value ?
}
function from_candid_vec_n10(_uploadFile: (file: ExternalBlob) => Promise<Uint8Array>, _downloadFile
    return value.map((x)=>from_candid_ChildRecord_n11(_uploadFile, _downloadFile, x));
}
function to_candid_ChildRecord_n18(_uploadFile: (file: ExternalBlob) => Promise<Uint8Array>, _downlo
    return to_candid_record_n19(_uploadFile, _downloadFile, value);
}
function to_candid_Status_n20(_uploadFile: (file: ExternalBlob) => Promise<Uint8Array>, _downloadFil
    return to_candid_variant_n21(_uploadFile, _downloadFile, value);
}
function to_candid_UserRole_n8(_uploadFile: (file: ExternalBlob) => Promise<Uint8Array>, _downloadFi
    return to_candid_variant_n9(_uploadFile, _downloadFile, value);
}
function to_candid__CaffeineStorageRefillInformation_n2(_uploadFile: (file: ExternalBlob) => Promise
    return to_candid_record_n3(_uploadFile, _downloadFile, value);
}
function to_candid_opt_n1(_uploadFile: (file: ExternalBlob) => Promise<Uint8Array>, _downloadFile: (
    return value === null ? candid_none() : candid_some(to_candid__CaffeineStorageRefillInformation_
}
function to_candid_record_n19(_uploadFile: (file: ExternalBlob) => Promise<Uint8Array>, _downloadFil
    age: bigint;
    status: Status;
    lastSeenPlace: string;
    lastLocation: string;
    name: string;
    contactNumber: string;
    photoId: string;
}): {
    age: bigint;
    status: _Status;
    lastSeenPlace: string;
    lastLocation: string;
    name: string;
    contactNumber: string;
    photoId: string;
} {
    return {
        age: value.age,
        status: to_candid_Status_n20(_uploadFile, _downloadFile, value.status),
        lastSeenPlace: value.lastSeenPlace,
        lastLocation: value.lastLocation,
        name: value.name,
        contactNumber: value.contactNumber,
        photoId: value.photoId
    };
}
function to_candid_record_n3(_uploadFile: (file: ExternalBlob) => Promise<Uint8Array>, _downloadFile
    proposed_top_up_amount?: bigint;
}): {
    proposed_top_up_amount: [] | [bigint];
} {
```

```
    return {
        proposed_top_up_amount: value.proposed_top_up_amount ? candid_some(value.proposed_top_up_amo
    };
}
function to_candid_tuple_n22(_uploadFile: (file: ExternalBlob) => Promise<Uint8Array>, _downloadFile
    return [
        value[0],
        to_candid_Status_n20(_uploadFile, _downloadFile, value[1])
    ];
}
function to_candid_variant_n21(_uploadFile: (file: ExternalBlob) => Promise<Uint8Array>, _downloadFi
    found: null;
} | {
    closed: null;
} | {
    active: null;
} | {
    underReview: null;
} {
    return value == Status.found ? {
        found: null
    } : value == Status.closed ? {
        closed: null
    } : value == Status.active ? {
        active: null
    } : value == Status.underReview ? {
        underReview: null
    } : value;
}
function to_candid_variant_n9(_uploadFile: (file: ExternalBlob) => Promise<Uint8Array>, _downloadFil
    admin: null;
} | {
    user: null;
} | {
    guest: null;
} {
    return value == UserRole.admin ? {
        admin: null
    } : value == UserRole.user ? {
        user: null
    } : value == UserRole.guest ? {
        guest: null
    } : value;
}
export interface CreateActorOptions {
    agent?: Agent;
    agentOptions?: HttpAgentOptions;
    actorOptions?: ActorConfig;
    processError?: (error: unknown) => never;
}
export function createActor(canisterId: string, _uploadFile: (file: ExternalBlob) => Promise<Uint8Ar
    const agent = options.agent || HttpAgent.createSync({
        ...options.agentOptions
    });
    if (options.agent && options.agentOptions) {
```

```
        console.warn("Detected both agent and agentOptions passed to createActor. Ignoring agentOpti
    }
    const actor = Actor.createActor<_SERVICE>(idlFactory, {
        agent,
        canisterId: canisterId,
        ...options.actorOptions
    });
    return new Backend(actor, _uploadFile, _downloadFile, options.processError);
}
```